

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**PRISM GUARD: VIDEOJUEGO
3D TIPO RTS DONDE EL
JUGADOR HA DE
DEFENDERSE ANTE OLEADAS
DE ENEMIGOS**

AUTOR: MARCOS MORALES MÁRQUEZ

Puerto Real, junio 2026

TRABAJO FIN DE GRADO

GRADO EN INGENIERÍA INFORMÁTICA

**PRISM GUARD: VIDEOJUEGO
3D TIPO RTS DONDE EL
JUGADOR HA DE
DEFENDERSE ANTE OLEADAS
DE ENEMIGOS**

DIRECTOR: KEVIN JESÚS VALLE GÓMEZ
CODIRECTOR: JOSE MIGUEL ARAGÓN JURADO
AUTOR: MARCOS MORALES MÁRQUEZ

Puerto Real, junio 2026

Agradecimientos

A mis padres, por apoyarme siempre, por todos los consejos y ayudas que me han dado.

A mis amigos de la universidad, especialmente Ángel y Mario, por hacer esta experiencia más divertida, incluso en los malos momentos.

A mis profesores, tanto de la universidad como del instituto, por enseñarme tanto.

A mi tutor, Kevin, y a mi co-tutor, José Miguel, por apoyarme desde el principio y por sus ideas para mejorar y complementar el proyecto.

Resumen

Este Trabajo Fin de Grado presenta el desarrollo de un prototipo de videojuego de estrategia y gestión en 3D, implementado en Unity, en el que el jugador administra una base mediante sistemas de construcción, producción, investigación, economía y combate. El proyecto se ha diseñado con una arquitectura modular orientada a separar claramente los datos, la lógica de juego, la interfaz de usuario y los sistemas de apoyo, de manera que el comportamiento global pueda escalar de forma ordenada y mantener una base de código extensible.

El juego incorpora un sistema de construcción con previsualización, validación de colocación y transición entre planos y estructuras terminadas. A ello se suman sistemas de recursos e inventario, con almacenes capaces de gestionar distintos tipos de objetos y procesadores de recursos que convierten materiales de entrada en productos de salida según recetas configurables. Para coordinar el trabajo de las unidades, se ha implementado un sistema de tareas centralizado que incluye transporte de objetos, construcción y otras actividades, junto con reservas internas para evitar conflictos entre trabajadores concurrentes.

La jugabilidad también incluye un sistema de investigación que desbloquea contenido y aplica modificadores de atributos, así como un sistema general de atributos basado en datos que permite ajustar los atributos de unidades y estructuras de manera flexible. Sobre esta base se han desarrollado unidades controladas por inteligencia artificial (IA) y un sistema de órdenes de estrategia en tiempo real (RTS) con selección múltiple, movimiento en formación y asignación de destinos. El combate se apoya en un sistema de objetivos compartidos, sensores de detección, proyectiles, torres defensivas y retroalimentación visual del daño y la curación.

El resultado es una base jugable que integra gestión, construcción, investigación, producción y combate dentro de un mismo marco técnico, dejando una arquitectura preparada para seguir ampliando contenido, balance y funcionalidades en iteraciones futuras.

Palabras clave: videojuego 3D, Unity, C#, RTS, gestión, construcción, recursos, investigación, ML-Agents, combate, State Pattern, Singleton Pattern, Flyweight Pattern, SOLID.

Índice general

Índice de figuras	11
Índice de tablas	13
I Prolegómeno	1
1. Introducción	2
1.1. Motivación	2
1.2. Alcance y objetivos	3
1.3. Glosario de términos	4
1.4. Organización del documento	6
2. Antecedentes	9
2.1. Contexto	9
2.2. Estado de la técnica	9
3. Plan de gestión de proyecto	15
3.1. Metodología de desarrollo	15
3.2. Tecnologías	15
3.3. Planificación del proyecto	16
3.4. Costes	18
3.4.1. Costes humanos	19
3.4.2. Costes materiales	19
3.4.3. Costes indirectos	19
3.4.4. Coste total del desarrollo	19
3.4.5. Presupuesto estimado del proyecto	20
3.5. Riesgos	20
3.6. Gestión de la configuración	21
3.7. Aseguramiento de calidad	23
II Desarrollo	24
4. Requisitos del Sistema	25
4.1. Objetivos de Negocio	25
4.2. Objetivos del Sistema	25
4.3. Requisitos funcionales	26
4.4. Requisitos no funcionales	28
4.5. Requisitos de información	28
4.6. Reglas de negocio	29
5. Análisis del Sistema	33

5.1.	Modelo Conceptual	33
5.2.	Modelo de Casos de Uso	34
5.3.	Modelo de Interfaz de Usuario	43
5.4.	Modelo de Comportamiento	44
6.	Diseño del Sistema	51
6.1.	Arquitectura del Sistema	51
6.1.1.	Diseño de alto nivel	52
6.1.2.	Diseño detallado	53
6.2.	Diseño Físico de Datos	54
6.3.	Diseño de la Interfaz de Usuario	55
7.	Construcción del Sistema	59
7.1.	Entorno de Construcción	59
7.2.	Código Fuente	60
7.2.1.	Archivos sueltos	60
7.2.2.	Directorio Game	61
7.2.3.	Directorio Inventory	62
7.2.4.	Directorio ItemConsumer	63
7.2.5.	Directorio Items	64
7.2.6.	Directorio Research	65
7.2.7.	Directorio Resources	66
7.2.8.	Directorio Score	67
7.2.9.	Directorio Selectable & Order	67
7.2.10.	Directorio Stat	68
7.2.11.	Directorio Structure	69
7.2.12.	Directorio Target	74
7.2.13.	Directorio UI	74
7.2.14.	Directorio Units	76
7.2.15.	Directorio Wave	82
7.2.16.	Extractos significativos de códigos	82
7.3.	Scripts de Base de Datos	90
8.	Pruebas del Sistema	93
8.1.	Estrategia	93
8.2.	Pruebas Unitarias	94
8.3.	Pruebas de Integración	96
8.4.	Pruebas de Sistema	98
8.5.	Pruebas de Aceptación	103
9.	Despliegue del Sistema	105
9.1.	Arquitectura Física	105
9.2.	Instrucciones de despliegue	105
9.2.1.	Requisitos previos	106
9.2.2.	Inventario de componentes	106
9.2.3.	Procedimientos de instalación	106
9.2.4.	Pruebas de implantación	107

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio	107
III Epílogo	109
10. Conclusiones	110
10.1. Objetivos alcanzados	110
10.2. Lecciones aprendidas	110
10.3. Trabajo futuro	111
Información sobre Licencia	113
Bibliografía	115
IV Anexos	119
A. Manual del desarrollador	120
A.1. Introducción	120
A.2. Preparación del entorno de trabajo	120
A.3. Consideraciones generales sobre el desarrollo	121
A.4. Instrucciones para construcción	121
B. Manual de usuario	125
B.1. Introducción	125
B.2. Instalación	125
B.3. Uso del sistema	125

Índice de figuras

2.1. Imágenes de RimWorld y Age of Empires I	11
2.2. Imágenes de Oxygen Not Included y Diplomacy is Not an Option . . .	11
2.3. Imágenes de Factorio y Dwarf Fortress	11
3.1. Desarrollo iterativo e incremental	15
3.2. Iconos de Unity y C Sharp	16
3.3. Iconos de Git y GitHub	17
3.4. Iconos de Fork y Obsidian	17
3.5. Iconos de Overleaf y Paint.NET	17
3.6. Diagrama de Gantt	18
3.7. Diagrama de flujo de Gitflow	22
3.8. Ejemplo de Pull Request	22
5.1. Modelo conceptual	33
5.2. Diagrama de caso de uso	34
5.3. Mockup interfaz principal	44
5.4. Mockup interfaz de construcción	45
5.5. Mockup interfaz de investigación	46
5.6. Mockup interfaz de producción de unidades	47
5.7. Mockup interfaz de la tienda	48
5.8. Diagrama de navegación	48
5.9. Diagrama de secuencia para construir una estructura	49
5.10. Diagrama de secuencia para investigar un proyecto	49
5.11. Diagrama de secuencia para procesar recursos	50
5.12. Diagrama de secuencia para cambiar la velocidad del juego	50
6.1. Diagrama de contexto	52
6.2. Diagrama de componentes de alto nivel	52
6.3. Prototipo de la interfaz principal	56
6.4. Prototipo de la interfaz de construcción	57
6.5. Prototipo de la interfaz de investigación	57
6.6. Prototipo de la interfaz de producción de unidades	57
6.7. Prototipo de la interfaz de la tienda	58
7.1. Organización general de los directorios	60
7.2. Scripts sueltos	61
7.3. Organización del directorio Game	62
7.4. Organización del directorio Inventory	63
7.5. Organización del directorio ItemConsumer	64
7.6. Organización del directorio Items	64
7.7. Organización del directorio Research	65
7.8. Organización del directorio Resources	66
7.9. Organización del directorio Score	67

7.10. Organización del directorio Selectable & Order	68
7.11. Organización del directorio Stat	68
7.12. Scripts sueltos del directorio Structure	69
7.13. Organización del subdirectorío Construction	70
7.14. Organización del subdirectorío Defenses	70
7.15. Organización del subdirectorío Resource	71
7.16. Organización del subdirectorío Shop	72
7.17. Organización del subdirectorío UnitFactory	72
7.18. Organización del subdirectorío Warehouses	73
7.19. Organización del directorío Target	74
7.20. Organización del directorío UI	75
7.21. Scripts sueltos del directorío Unit	77
7.22. Organización del subdirectorío Agent	77
7.23. Organización del subdirectorío Enemy	78
7.24. Organización del subdirectorío Soldier	79
7.25. Organización del subdirectorío Worker	80
7.26. Organización del directorío Wave	82
B.1. Previsualización de la estructura a construir	126
B.2. Plano y construcción	127
B.3. Trabajador completando tareas	127
B.4. Inventario y umbrales	128
B.5. Interfaz de investigación	129
B.6. Combate y retroalimentación visual	130
B.7. Movimiento en formación	131
B.8. Interfaz de derrota	131

Índice de tablas

3.1. Presupuesto estimado del proyecto	20
4.1. Requisitos funcionales del sistema	26
4.2. Requisitos no funcionales del sistema	28

Índice de extractos de código

7.1. Extracto de SoldierIdleState	83
7.2. Extracto de SoldierChaseState	83
7.3. Extracto de SoldierAttackState	84
7.4. Extracto de TaskManager mostrando como se garantiza que solo exista una única instancia	85
7.5. Extracto de TaskManager mostrando el método RequestTask	86
7.6. Extracto de StatModifierManager mostrando el uso del evento OnModifiersChanged	87
7.7. Extracto de Enemy mostrando la suscripción al evento OnModifiersChanged	88
7.8. Extracto de BaseData	89
7.9. Extracto de ResearchData	89

Parte I

Prolegómeno

1. Introducción

A lo largo de esta memoria se presenta el desarrollo de un prototipo de videojuego de estrategia y gestión 3D, así como el proceso seguido para su análisis, diseño, construcción, validación y despliegue. El trabajo aborda la creación de un sistema jugable completo, con especial atención a la organización interna del software, a la coordinación entre subsistemas y a la construcción de una base técnica preparada para evolucionar en futuras iteraciones.

El proyecto combina mecánicas propias de un videojuego de estrategia en tiempo real con técnicas habituales de la ingeniería del software, por lo que constituye un caso de estudio adecuado para aplicar principios de arquitectura modular, diseño orientado a componentes, gestión de estados, control de versiones y validación mediante pruebas. Sobre esa base, el resto de este capítulo expone la motivación del proyecto, su alcance, un glosario de términos relevantes y la organización general de la memoria.

1.1. Motivación

Este proyecto se enmarca en el desarrollo de un videojuego de estrategia y gestión 3D, implementado con el motor gráfico Unity, que integra mecánicas habituales en los juegos de estrategia en tiempo real (RTS), como la construcción de estructuras, la gestión de recursos, la asignación de tareas a unidades, la investigación de mejoras y el combate en tiempo real, además del uso de *Machine Learning* (ML) [35] para optimizar la toma de decisiones de unidades especiales.

Desde el punto de vista técnico, el proyecto permite aplicar de forma conjunta múltiples áreas de la ingeniería del software: modelado de datos, arquitectura modular, diseño orientado a componentes, sistemas de inteligencia artificial, control de versiones, gestión de estados, interfaz de usuario y validación del comportamiento mediante pruebas. Además, el desarrollo de un videojuego de este tipo exige resolver problemas de coordinación entre subsistemas, como la reserva de recursos, la sincronización entre trabajadores o la comunicación entre la lógica del juego y la interfaz.

Otro de los motivos del proyecto es la posibilidad de construir una base técnica flexible y extensible. El juego no se plantea únicamente como una demostración visual, sino como un prototipo funcional sobre el que se pueden incorporar nuevas mecánicas y contenido en futuras iteraciones. Por ello, una parte importante del esfuerzo se ha centrado en crear sistemas reutilizables y desacoplados, aplicando patrones de diseño como *State Pattern* [36], *Singleton Pattern* [12], *Flyweight Pattern* [33], *Command Pattern* y un enfoque basado en eventos, de forma que la evolución futura del proyecto sea más sencilla y controlada.

Del mismo modo, se han tenido en cuenta los principios de diseño orientado a objetos recogidos en *SOLID* [7], con el objetivo de mantener responsabilidades bien definidas,

reducir dependencias innecesarias y facilitar tanto la mantenibilidad como la ampliación del código fuente. Esta orientación resulta especialmente relevante en un proyecto de estas características, donde numerosos sistemas deben convivir e interactuar sin comprometer la estabilidad general del conjunto.

1.2. Alcance y objetivos

El objetivo principal del proyecto es desarrollar un prototipo jugable de estrategia y gestión que permita al jugador construir una base, producir y organizar unidades, investigar mejoras, recolectar y procesar recursos, defenderse de oleadas enemigas e interactuar con distintos sistemas de interfaz y control.

Dentro de ese objetivo general, el alcance del proyecto se concreta en los siguientes subobjetivos:

- O1. **Sistema de construcción.** Permitir la colocación de estructuras con previsualización, validación de posición y transición entre plano y estructura terminada.
- O2. **Sistema de recursos e inventario.** Gestionar recursos, inventarios, reservas y almacenes de forma coherente, incluyendo categorías de objetos y capacidades limitadas.
- O3. **Sistema de procesamiento de recursos.** Transformar recursos de entrada en productos de salida mediante recetas configurables y procesos definidos.
- O4. **Sistema de investigación.** Desbloquear contenido nuevo y aplicar modificadores sobre atributos, unidades y estructuras mediante proyectos de investigación.
- O5. **Sistema de tareas.** Centralizar la asignación y ejecución de tareas para trabajadores, como recolección y transporte de recursos, construcción e investigación.
- O6. **Sistema de selección y órdenes RTS.** Permitir al jugador seleccionar unidades y emitir órdenes de tipo RTS para controlarlas de forma directa.
- O7. **Sistema de combate y defensa.** Integrar combate, defensa y retroalimentación visual para representar de forma clara los enfrentamientos en partida.
- O8. **Sistema de atributos.** Disponer de un sistema de atributos basado en datos, capaz de modificar de forma flexible las características de unidades y estructuras.
- O9. **Interfaz de usuario.** Proporcionar paneles y elementos de interfaz para la construcción, para facilitar la interacción del jugador con los sistemas principales, como investigación, producción, inventario y órdenes.

El proyecto se ha planteado como un prototipo técnico y funcional, por lo que no pretende incluir una campaña completa, un balance final de contenido ni una cantidad extensa de arte final. El alcance se centra en dejar implementadas las bases jugables y arquitectónicas del sistema, con especial atención a la robustez, la escalabilidad y la mantenibilidad del código.

1.3. Glosario de términos

- **Addressable Asset**: recurso gestionado mediante el sistema Addressables de Unity, cargado dinámicamente durante la ejecución.
- **Addressables**: sistema de Unity para la carga dinámica de recursos y datos.
- **Agente**: entidad controlada por inteligencia artificial o aprendizaje automático, capaz de percibir el entorno y ejecutar acciones de forma autónoma.
- **AI / IA**: *Artificial Intelligence* / Inteligencia Artificial. Conjunto de técnicas utilizadas para dotar de comportamiento automático a unidades o sistemas del juego.
- **Blueprint / Plano**: plano de construcción que representa una estructura pendiente de ser terminada.
- **Combat Feedback**: retroalimentación visual asociada al combate, como textos de daño o curación.
- **Command Pattern**: patrón de diseño que encapsula una petición como un objeto, permitiendo parametrizar acciones y desacoplar el emisor de la ejecución.
- **Consumidor / Estructura consumidora**: elemento del juego capaz de recibir, almacenar o consumir ítems o recursos, con una capacidad limitada.
- **Event-driven**: enfoque de diseño basado en eventos, en el que los sistemas reaccionan a cambios o sucesos mediante notificaciones desacopladas.
- **Flyweight Pattern**: patrón de diseño que comparte estado común entre múltiples instancias para reducir el consumo de memoria.
- **Gitflow**: modelo de trabajo con Git basado en ramas diferenciadas para desarrollo, integración y versiones estables.
- **Input System**: sistema de gestión de entradas de Unity que permite abstraer dispositivos de control y acciones del jugador.
- **Instancia**: objeto concreto generado en tiempo de ejecución a partir de un prefab o una definición de datos.
- **Inventory / Inventario**: sistema encargado del almacenamiento y gestión de recursos e ítems.
- **Item / Recurso**: objeto intercambiable del juego que puede almacenarse, transportarse, comprarse, venderse o consumirse.
- **ML**: *Machine Learning*. Rama de la inteligencia artificial basada en el aprendizaje a partir de datos.
- **NavMesh**: sistema de navegación de Unity basado en superficies transitables utilizadas para el cálculo de rutas.
- **Nodo de investigación**: elemento individual dentro del árbol de investigación que puede desbloquear mejoras o contenido.

- **Oleada:** conjunto de enemigos generados periódicamente como parte de la progresión y dificultad del juego.
- **Pool de objetos:** técnica de optimización basada en reutilizar objetos previamente creados para evitar instanciaciones frecuentes.
- **Prefab:** activo de Unity que almacena una entidad reutilizable junto con su configuración, componentes y jerarquía.
- **Pull Request:** solicitud de integración de cambios desde una rama de desarrollo hacia otra dentro de un flujo de trabajo basado en Git.
- **Recipe / Receta:** definición que especifica los recursos de entrada, salida y las condiciones necesarias para ejecutar un proceso de producción.
- **Reserva:** mecanismo mediante el cual un recurso o espacio de almacenamiento se marca temporalmente para evitar conflictos entre tareas concurrentes.
- **RTS:** *Real-Time Strategy*. Género de estrategia en tiempo real basado en el control de unidades y la gestión de bases.
- **Scene / Escena:** entorno de Unity que agrupa objetos, sistemas y configuración de juego para una fase concreta o un menú.
- **ScriptableObject:** tipo de dato de Unity utilizado para definir información configurable, como atributos, unidades, estructuras, recursos o recetas.
- **Singleton Pattern:** patrón de diseño que garantiza la existencia de una única instancia de una clase y proporciona un punto de acceso global a ella.
- **SOLID:** conjunto de cinco principios de diseño orientado a objetos que favorecen la mantenibilidad, escalabilidad y extensibilidad del código.
- **State Pattern:** patrón de diseño que permite modificar el comportamiento de un objeto cuando cambia su estado interno.
- **StatKey:** identificador único de un atributo concreto dentro del sistema de atributos.
- **Stats / Atributos:** valores numéricos que describen las características de una unidad o estructura, como salud, daño, velocidad o alcance.
- **Threshold / Umbral:** valor límite a partir del cual se detiene o modifica una acción, como producir o recolectar más recursos.
- **Unity:** motor de desarrollo de videojuegos utilizado para implementar el proyecto.
- **Worker / Trabajador:** unidad encargada de ejecutar tareas de construcción, transporte, investigación y otras acciones de apoyo.

1.4. Organización del documento

La presente memoria se estructura en tres bloques principales: *Prolegómeno*, *Desarrollo* y *Epílogo*, además de los anexos finales.

Prolegómeno

- **Capítulo 1: Introducción.** Presenta el contexto general del proyecto, su motivación, los objetivos perseguidos, el alcance del sistema y la organización de la memoria.
- **Capítulo 2: Antecedentes.** Analiza el estado de la técnica y las soluciones existentes que han servido como referencia para el desarrollo del proyecto.
- **Capítulo 3: Plan de gestión del proyecto.** Describe la metodología de desarrollo utilizada, la planificación temporal, los recursos empleados, el análisis de costes, la gestión de riesgos y las medidas de calidad aplicadas.

Desarrollo

- **Capítulo 4: Requisitos del sistema.** Recoge los requisitos funcionales y no funcionales que debe satisfacer la solución desarrollada.
- **Capítulo 5: Análisis del sistema.** Presenta el análisis conceptual del sistema mediante casos de uso y modelos que describen el comportamiento esperado.
- **Capítulo 6: Diseño del sistema.** Describe la arquitectura del software, la organización de los subsistemas, los patrones de diseño empleados y el diseño de la interfaz de usuario.
- **Capítulo 7: Construcción del sistema.** Detalla las tecnologías utilizadas, la estructura del código fuente y la implementación de los principales componentes y funcionalidades.
- **Capítulo 8: Pruebas del sistema.** Expone la estrategia de pruebas seguida, los casos evaluados y los resultados obtenidos durante la validación del proyecto.
- **Capítulo 9: Despliegue del sistema.** Describe el proceso de compilación, distribución y puesta en funcionamiento del videojuego.

Epílogo

- **Capítulo 10: Conclusiones.** Resume los objetivos alcanzados, las principales aportaciones del proyecto, las limitaciones encontradas y las posibles líneas de trabajo futuro.

Anexos

- **Anexo A: Manual del desarrollador.** Proporciona información técnica sobre la estructura interna del proyecto, su configuración y las pautas necesarias para modificar o ampliar sus funcionalidades.
- **Anexo B: Manual del usuario.** Describe el funcionamiento del videojuego desde el punto de vista del usuario final, incluyendo controles, mecánicas principales e instrucciones de uso.

2. Antecedentes

En este capítulo se describe el contexto que justifica el desarrollo del proyecto y se analizan algunas de las soluciones y enfoques existentes que han servido como referencia.

2.1. Contexto

El presente Trabajo Fin de Grado se enmarca en el desarrollo de un videojuego 3D de estrategia y gestión, implementado con Unity. La propuesta combina mecánicas propias de los juegos de construcción de bases, administración de recursos, asignación de tareas, investigación tecnológica, defensa frente a oleadas y control táctico de unidades.

Este tipo de proyectos plantea retos técnicos relevantes, ya que obliga a coordinar sistemas que suelen estar estrechamente relacionados entre sí: generación y consumo de recursos, almacenamiento, transporte, producción, construcción, combate, interfaz de usuario e inteligencia artificial. A diferencia de otros desarrollos de software más lineales, un videojuego de estas características requiere una arquitectura modular capaz de evolucionar de forma ordenada sin comprometer la estabilidad del conjunto.

La motivación del proyecto no responde a una necesidad organizativa externa, sino al interés por construir un prototipo jugable con una base técnica sólida, capaz de integrar sistemas complejos de manera coherente. En este sentido, el valor del trabajo no reside únicamente en el resultado visual o en la jugabilidad inicial, sino también en la calidad de la estructura interna del código, en la reutilización de componentes y en la posibilidad de ampliar el proyecto en iteraciones futuras.

Desde el punto de vista tecnológico, Unity proporciona un entorno adecuado para este objetivo por su integración con sistemas de animación, física, navegación, interfaz de usuario, carga de recursos y depuración. Además, el uso de *ScriptableObjects*, *Addressables*, *Input System* y *NavMesh* permite organizar el proyecto de forma flexible, separar los datos de la lógica de ejecución y facilitar el mantenimiento de los distintos subsistemas.

2.2. Estado de la técnica

Antes de definir la solución propuesta, resulta conveniente revisar algunos sistemas existentes que abordan problemas similares. En el ámbito de los videojuegos de gestión y estrategia en tiempo real, existen títulos que resuelven por separado varias de las mecánicas que este proyecto pretende integrar: construcción de estructuras, asignación de tareas a unidades, gestión de inventario, producción de unidades, investigación de mejoras, defensa de la base y control táctico del combate. Las referencias principales utilizadas son:

- **RimWorld** [19] (Figura 2.1a) es un simulador de colonias guiado por un narrador de inteligencia artificial. Su principal aportación es el enfoque en la automatización de tareas y en la gestión de prioridades por parte de colonos autónomos, lo que genera situaciones emergentes a partir de la interacción entre necesidades, recursos y eventos dinámicos. Cada colono actúa en función de un conjunto de trabajos ordenados por prioridad, lo que permite al sistema distribuir actividades de forma flexible y adaptativa sin intervención del jugador. Este planteamiento resulta especialmente relevante para el presente proyecto, ya que inspira la organización de tareas, la asignación de trabajos a unidades, la coordinación entre distintos sistemas internos y el sistema de investigación.
- **Age of Empires I** [43] (Figura 2.1b) es el primer juego de una saga de estrategia en tiempo real centrada en construir, expandir y defender imperios. Su estructura clásica combina recolección de recursos, construcción de edificios, desarrollo tecnológico y combate táctico. En este caso, el interés como referencia reside en la claridad de sus bucles jugables y en la relación directa entre economía, producción y combate.
- **Oxygen Not Included** [16] (Figura 2.2a) es una simulación de colonia en la que el jugador gestiona colonos en un entorno interdependiente. Su diseño destaca por la profundidad de sus sistemas de simulación, donde el flujo de gases, temperatura, energía, logística y recursos afecta de forma continua a la colonia. Esta referencia resulta útil para el proyecto por su tratamiento de sistemas complejos interrelacionados y por la importancia que concede a la organización interna y a la automatización progresiva.
- **Diplomacy is Not an Option** [6] (Figura 2.2b) es un RTS de construcción de bases ambientado en un mundo medieval de fantasía, donde el objetivo es construir defensas y resistir oleadas constantes de enemigos. Su interés para este proyecto radica en la combinación entre construcción, defensa de la base y presión táctica sostenida, así como en la necesidad de responder de forma rápida a amenazas crecientes. Este enfoque conecta directamente con la parte de combate, oleadas y estructuras defensivas del sistema desarrollado.
- **Factorio** [42] (Figura 2.3a) es un videojuego centrado en la automatización industrial y la optimización de cadenas de producción. Gira en torno a la extracción de recursos, su transformación mediante máquinas interconectadas y la construcción de infraestructuras capaces de escalar de forma eficiente conforme aumenta la complejidad del sistema. Esta referencia resulta especialmente útil para el proyecto por su tratamiento de la logística, el transporte entre nodos de producción y la necesidad de organizar de manera coherente los flujos de entrada y salida de materiales. Además, muestra con claridad cómo la automatización progresiva reduce la carga de trabajo manual del jugador y convierte la planificación en el eje principal de la experiencia.
- **Dwarf Fortress** [24] (Figura 2.3b) es una simulación de gestión extremadamente profunda en la que múltiples sistemas interactúan de forma continua para producir resultados emergentes. El control de las unidades se basa en la asignación de labores, la priorización de trabajos y la convivencia entre necesidades, riesgos y recursos limitados. Su interés como referencia reside en la complejidad sistémica,

la granularidad de la simulación y la capacidad de generar situaciones no previstas directamente por el jugador. Para este proyecto, constituye una referencia valiosa a la hora de justificar la interacción entre subsistemas y la autonomía de las unidades.



(a) RimWorld



(b) Age of Empires I

Figura 2.1: Imágenes de RimWorld y Age of Empires I

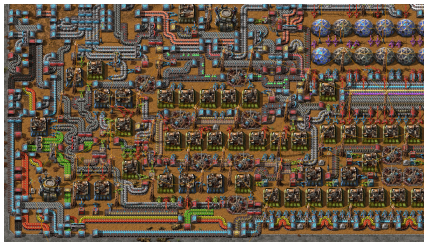


(a) Oxygen Not Included



(b) Diplomacy is Not an Option

Figura 2.2: Imágenes de Oxygen Not Included y Diplomacy is Not an Option



(a) Factorio



(b) Dwarf Fortress

Figura 2.3: Imágenes de Factorio y Dwarf Fortress

Estas propuestas muestran enfoques distintos y complementarios. *RimWorld* destaca por la automatización de tareas y la asignación de prioridades a colonos autónomos, lo que reduce el control directo del jugador, pero exige una planificación cuidadosa de los sistemas internos. *Age of Empires* prioriza, en cambio, el control táctico inmediato sobre la economía y el combate, con una progresión más orientada a la expansión territorial y a la toma de decisiones en tiempo real. *Oxygen Not Included* apuesta por una simulación profunda, donde la interdependencia entre sistemas incrementa de forma notable la complejidad de gestión. *Diplomacy is Not an Option* combina construcción de base, formación de ejércitos y defensa frente a hordas, lo que refuerza

la presión táctica y la necesidad de coordinar economía y combate de forma constante. Por su parte, *Factorio* pone el foco en la automatización industrial, la optimización de cadenas de producción y la organización eficiente de flujos de recursos, mientras que *Dwarf Fortress* representa un enfoque de simulación extremadamente detallado, en el que la interacción y coordinación entre subsistemas adquieren un papel central.

Desde el punto de vista del desarrollo de software, estas soluciones muestran patrones útiles para este trabajo: uso de datos configurables para unidades y estructuras, reglas de desbloqueo mediante investigación, sistemas de órdenes basados en selección múltiple y mecanismos de automatización progresiva. También evidencian ventajas y desventajas claras. Los enfoques más automatizados y sistémicos favorecen la profundidad jugable y la aparición de situaciones emergentes, pero complican la depuración, el equilibrio y el control de estados. En cambio, los modelos más tácticos resultan más directos para el jugador, aunque suelen repartir menos responsabilidad entre los subsistemas y ofrecen menor complejidad a largo plazo. Estas ideas han servido como referencia para estructurar la solución propuesta, adaptándolas a una arquitectura propia y a los requisitos concretos del proyecto.

En consecuencia, el estado de la técnica confirma que el problema está bien delimitado y que existen aproximaciones previas útiles, pero también pone de manifiesto la oportunidad de integrar en un único prototipo una serie de subsistemas que habitualmente aparecen separados. Esta combinación constituye uno de los principales aportes del proyecto, tanto desde el punto de vista funcional como desde el punto de vista de ingeniería del software.

3. Plan de gestión de proyecto

En esta sección se describen todos los aspectos relativos a la gestión del proyecto: metodología, organización, costes, planificación, riesgos y aseguramiento de la calidad.

3.1. Metodología de desarrollo

El proyecto se ha abordado mediante una metodología iterativa e incremental, orientada a construir primero una base funcional estable y, posteriormente, ampliar el sistema con nuevas mecánicas y subsistemas. Este enfoque resulta adecuado en el desarrollo de videojuegos, ya que permite validar de forma temprana la jugabilidad, detectar problemas de integración entre sistemas y corregir decisiones de diseño antes de que el coste de modificación sea elevado.

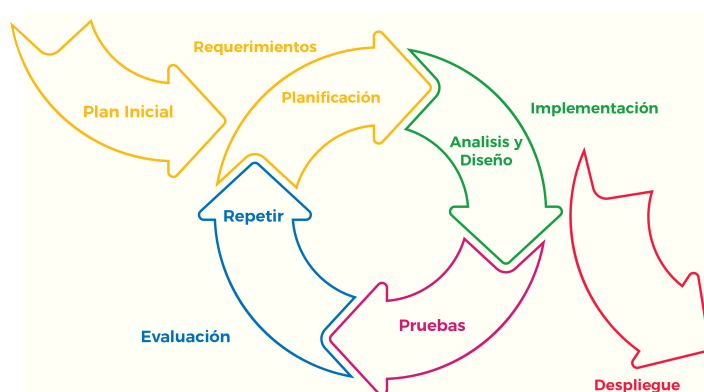


Figura 3.1: Desarrollo iterativo e incremental

La organización del trabajo se ha basado en la metodología ágil mostrada en la Figura 3.1. En cada iteración se ha seguido, de forma general, la secuencia planificación, análisis, diseño e implementación, pruebas y ajuste, revisando continuamente la interacción entre los distintos módulos del juego. Esta estrategia ha permitido evolucionar el proyecto desde sistemas básicos hasta subsistemas más complejos, manteniendo criterios propios de ingeniería del software: separación entre datos y lógica, modularidad, reutilización de componentes, revisión de dependencias y validación continua del comportamiento en escenas de prueba.

3.2. Tecnologías

El desarrollo se ha realizado con *Unity 6.4* [25] como motor principal, por ser una plataforma especialmente adecuada para la creación de videojuegos 3D y por integrar herramientas nativas para animación, navegación, interfaz de usuario y despliegue. El

lenguaje de programación empleado ha sido *C#* [20], utilizado para implementar la lógica del juego.

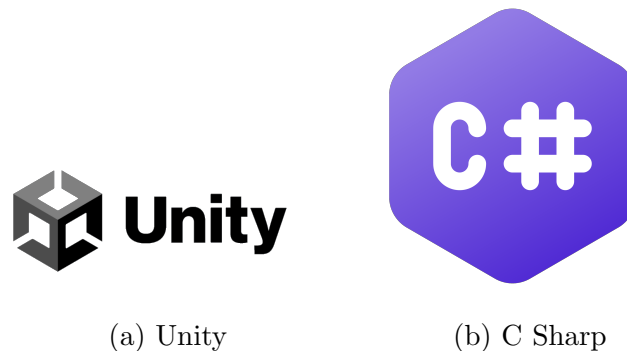


Figura 3.2: Iconos de Unity y C Sharp

Para la organización de datos dentro de Unity se han utilizado *ScriptableObjects* [39], que permiten separar la información configurable de la lógica de ejecución. Gracias a este enfoque, elementos como unidades, recursos, estructuras, recetas, investigaciones, atributos o parámetros de construcción pueden definirse como activos reutilizables y fácilmente editables desde el editor de Unity. Este modelo de trabajo resulta especialmente adecuado para un proyecto en el que gran parte del contenido debe poder ajustarse sin modificar la lógica interna del sistema.

También se han empleado otros subsistemas del motor, como *Input System* [34] para la gestión del control del jugador, *NavMesh* [32] para la navegación de unidades, *Addressables* [38] para la carga flexible de contenido y *Unity UI / TextMeshPro* [37] para la interfaz. En determinadas partes del proyecto se ha utilizado *ML-Agents* [35] para experimentar con agentes controlados por políticas de aprendizaje automático.

En cuanto a las herramientas de desarrollo, el proyecto se ha apoyado en *Git* [18] como sistema de control de versiones y en *GitHub* [13] como repositorio remoto principal. Para la gestión de ramas y revisiones, se ha utilizado *Fork* [5], lo que ha facilitado la revisión de cambios, el trabajo con ramas de funcionalidad y la integración progresiva de nuevas características. Como IDE se ha usado Microsoft Visual Studio Community 2026 [21]. Para la toma de notas y la organización de ideas, se ha empleado *Obsidian* [22], mientras que la elaboración de la memoria se ha realizado en *L^AT_EX* [17] mediante *Overleaf*. Por otra parte, para la edición de iconos, capturas e imágenes del proyecto, se ha utilizado *Paint.NET*, debido a su sencillez.

3.3. Planificación del proyecto

En la Figura 3.6 se puede observar el diagrama de Gantt; el desarrollo del proyecto se ha dividido en las siguientes partes:

- **Investigación inicial:** primero se recopilaban juegos con mecánicas similares a las deseadas, con el objetivo de aprender por qué las mecánicas más significativas de cada juego se han hecho de esa forma, para seguir un camino similar a la hora



Figura 3.3: Iconos de Git y GitHub

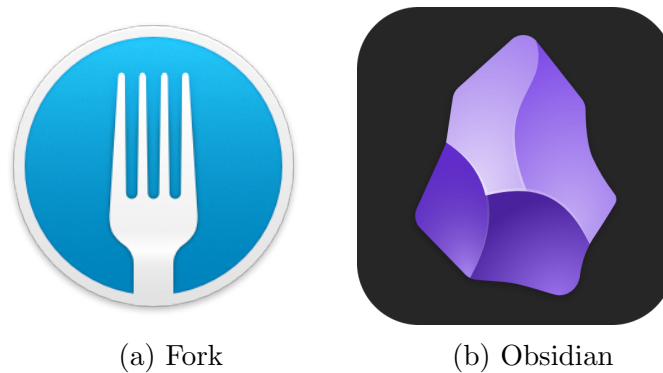


Figura 3.4: Iconos de Fork y Obsidian

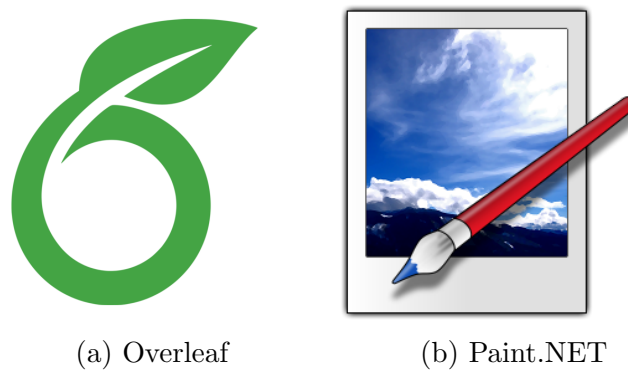


Figura 3.5: Iconos de Overleaf y Paint.NET

de implementar esas mecánicas en este proyecto. También se investigó qué motor gráfico usar, priorizando aquellos gratuitos como Unity [25], Unreal [9] o Godot [15], eligiendo al final Unity ya que tenía más experiencia con Unity que con el resto de opciones.

- **Análisis y diseño:** primero se analizó cómo debe ser el proyecto a partir de las ideas e investigaciones iniciales y qué funcionalidades principales debe tener. A continuación se elaboran diagramas y modelos de la arquitectura y se hacen mockups de la interfaz del usuario.
- **Implementación:** en esta fase se programan e integran las funcionalidades ana-

lizadas anteriormente, dividiéndose en hitos o funcionalidades principales, mostradas como subtarefas de la sección de implementación en la Figura 3.6.

- **Pruebas e integración:** a lo largo de la implementación, gracias al editor de Unity, es posible comprobar tanto las nuevas funcionalidades implementadas como la integración con el resto de los sistemas simulando una partida, facilitando mucho la detección y depuración de errores.
- **Documentación:** al final de la implementación de cada hito, se actualiza la memoria. Una vez acabada la implementación, se integra todo en la memoria, dándole la forma que tiene ahora.

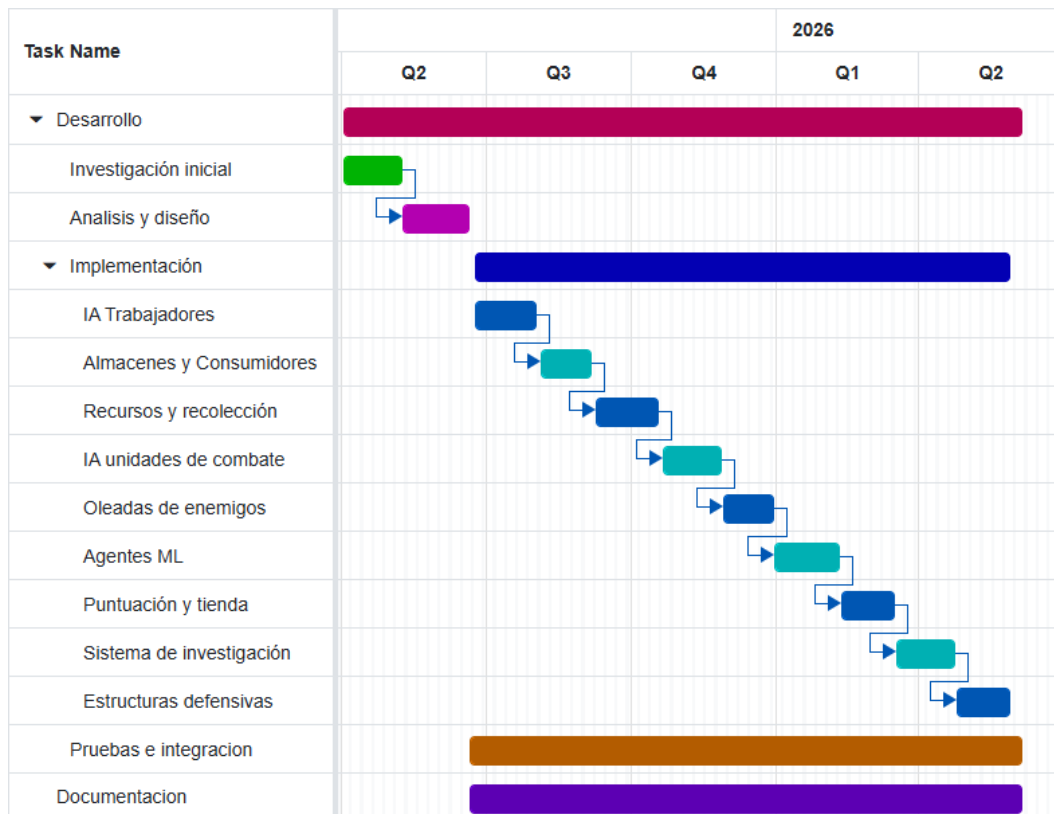


Figura 3.6: Diagrama de Gantt

Como conclusión, la investigación, el análisis y el diseño se completaron un poco más rápido de lo estimado; sin embargo, la implementación de ciertas partes tardó más de lo estimado.

3.4. Costes

Para estimar los costes del proyecto, se distinguen tres bloques: costes humanos, costes materiales y costes indirectos. Además, se incluye un presupuesto orientativo del sistema como producto software, con el fin de diferenciar el coste real de desarrollo del valor aproximado que podría tener en un entorno comercial.

3.4.1. Costes humanos

Basándose en la planificación temporal del proyecto y en el diagrama de Gantt de la Figura 3.6, se estima una dedicación total aproximada de 900 horas de trabajo. Esta cifra se obtiene a partir de unos 300 días de desarrollo con una media de 3 horas diarias. Realmente ha habido días en los que no se pudo dedicar prácticamente nada de tiempo, y días (especialmente sábados y domingos) en los que se dedicó alrededor de 7 horas.

Tomando como referencia un salario medio aproximado de 13,30 € por hora para un ingeniero informático [23], el coste humano estimado asciende a:

$$13,30 \text{ €/h} \times 900 \text{ h} = 11.970 \text{ €}$$

3.4.2. Costes materiales

El desarrollo se ha realizado empleando herramientas de uso gratuito o previamente disponibles antes del inicio del proyecto, como Unity, Visual Studio, Git, GitHub, Overleaf y distintos paquetes oficiales del motor. Por tanto, no se ha considerado un coste material adicional relevante asociado a licencias o adquisición de software.

3.4.3. Costes indirectos

El principal coste indirecto corresponde al consumo eléctrico del equipo utilizado durante el desarrollo. Este coste puede estimarse mediante la expresión:

$$\text{Coste} = \text{Potencia} \times \text{Horas} \times \text{Precio}$$

Si se considera una potencia media aproximada del ordenador de 300 W (0,30 kW), un tiempo total de trabajo de 900 horas y un precio de 0,05954 €/kWh [8], el coste eléctrico estimado es:

$$0,30 \text{ kW} \times 900 \text{ h} \times 0,05954 \text{ €/kWh} = 16,08 \text{ €}$$

3.4.4. Coste total del desarrollo

El coste total estimado del desarrollo, sumando costes humanos, materiales e indirectos, asciende a:

$$11.970 + 0 + 16,08 = 11.986,08 \text{ €}$$

3.4.5. Presupuesto estimado del proyecto

El coste calculado en la sección anterior representa únicamente el esfuerzo real invertido durante el desarrollo del proyecto. Sin embargo, en un contexto profesional, el presupuesto ofertado a un cliente debe incluir otros conceptos además del coste directo de producción.

Entre estos conceptos se encuentran los gastos generales de la empresa, los costes administrativos, las actividades de análisis y gestión, las posibles correcciones posteriores a la entrega, así como un margen de beneficio que garantice la viabilidad económica del proyecto.

Una práctica habitual consiste en aplicar un porcentaje adicional sobre el coste de desarrollo para cubrir dichos conceptos. Considerando un incremento conservador del 30 %, el presupuesto estimado sería:

Tabla 3.1

Presupuesto estimado del proyecto

Concepto	Importe (€)
Coste humano	11.970,00
Costes indirectos	16,08
Coste de desarrollo	11.986,08
Gastos generales y beneficio (30 %)	3.595,82
Presupuesto estimado	15.581,90

3.5. Riesgos

A continuación se explican los riesgos detectados:

- **Interdependencia de los sistemas (alto):** muchos subsistemas interactúan entre sí y dependen de estados compartidos. Un error en uno de ellos puede propagarse al resto y provocar comportamientos inesperados.
- **Complejidad de integración (alto):** a medida que se incorporan nuevas mecánicas, aumenta el riesgo de que la integración entre construcción, tareas, inventario, investigación, combate e interfaz introduzca errores difíciles de localizar.
- **Errores de sincronización (alto):** al tratarse de un juego en tiempo real, es necesario coordinar reservas, cancelaciones y cambios de estado para evitar condiciones de carrera, estados bloqueados o recursos duplicados.
- **Rendimiento insuficiente (medio):** la acumulación de unidades, estructuras, sensores, tareas y comprobaciones periódicas puede afectar al rendimiento si no se controlan adecuadamente las actualizaciones y los cálculos por fotograma.
- **Problemas de navegación y colisiones (medio):** el sistema de movimiento de la IA puede generar comportamientos erróneos si el terreno, los obstáculos o los radios de interacción no están correctamente configurados.

- **Dependencia de paquetes externos (medio):** el proyecto utiliza paquetes y herramientas externas de Unity. Cambios de versión, incompatibilidades o actualizaciones de API pueden obligar a modificar parte del código o del flujo de trabajo.
- **Obsolescencia tecnológica (medio):** algunas herramientas o paquetes empleados en el proyecto pueden quedar desactualizados con el tiempo, lo que puede requerir migraciones o refactorizaciones futuras.
- **Pérdida de referencias en Unity (alto):** dado que el proyecto depende de *prefabs*, escenas, *ScriptableObjects* y componentes serializados, existe el riesgo de que una modificación en el editor rompa referencias y provoque errores en tiempo de ejecución.
- **Errores de configuración de datos (alto):** un valor incorrecto en un *ScriptableObject*, en una receta, en un atributo o en un requisito de investigación puede provocar fallos lógicos difíciles de detectar.
- **Sobrecarga en la planificación (medio):** algunas funcionalidades pueden requerir más tiempo del previsto, lo que puede afectar a las fases finales del proyecto y obligar a recortar o simplificar contenido.
- **Cambio de alcance (medio):** durante el desarrollo pueden aparecer nuevas ideas o mejoras que incrementen el alcance inicial del proyecto y desvíen tiempo de las tareas prioritarias.
- **Problemas de compatibilidad entre sistemas operativos (bajo):** aunque el proyecto está orientado a Windows, pueden aparecer diferencias de comportamiento o problemas de ejecución en otras plataformas.

Para mitigar estos riesgos se ha priorizado una implementación incremental, con pruebas frecuentes, uso de componentes modulares desacoplados y validación de cada sistema antes de integrarlo con el resto. A su vez, cada vez que una parte de la implementación se completa, se sube al repositorio en Git. Para evitar problemas entre versiones, se utiliza un flujo de trabajo basado en ramas y *Pull Requests*. Este enfoque reduce el impacto de los errores y facilita la localización de problemas cuando aparecen.

3.6. Gestión de la configuración

La gestión de la configuración se ha orientado al control sistemático de los cambios introducidos en el código fuente, los recursos del proyecto y la documentación asociada. Para ello, se ha utilizado Git como sistema de control de versiones, organizando el trabajo mediante un flujo basado en ramas (*Gitflow*) que separa la versión estable del proyecto, la línea de desarrollo y las ramas destinadas a funcionalidades concretas. En la Figura 3.7, se puede ver un diagrama de flujo de *Gitflow*.

De este modo, la rama principal se ha reservado para las versiones estables del producto, mientras que la rama de desarrollo ha permitido integrar progresivamente nuevas funcionalidades antes de su consolidación. A su vez, cada cambio relevante se ha implementado en ramas específicas de tipo *feature*, lo que ha facilitado el trabajo iterativo,

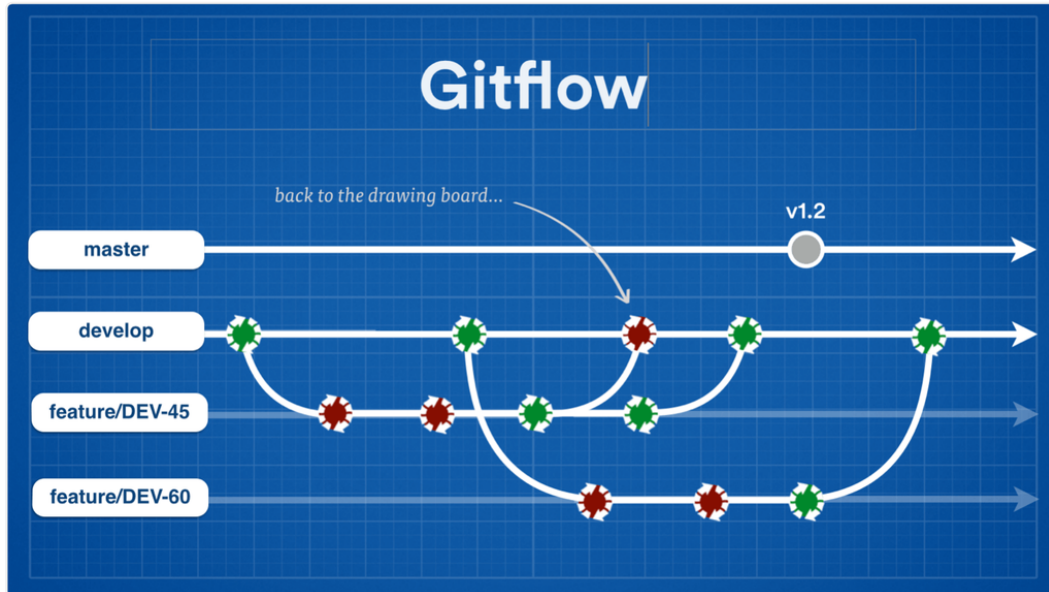


Figura 3.7: Diagrama de flujo de Gitflow

la revisión de modificaciones y la incorporación controlada de nuevas mecánicas al proyecto.

El uso de Git ha ido acompañado de una política de *commits* frecuentes y descriptivos, con el objetivo de registrar de forma trazable la evolución del proyecto y poder recuperar estados anteriores en caso de incidencias. Asimismo, cuando ha sido necesario validar cambios antes de su integración definitiva, se han empleado *Pull Requests* como mecanismo de revisión y control, usando más de 50 Pull Requests a lo largo del proyecto. En la Figura 3.8 se puede ver un ejemplo de Pull Request del proyecto.

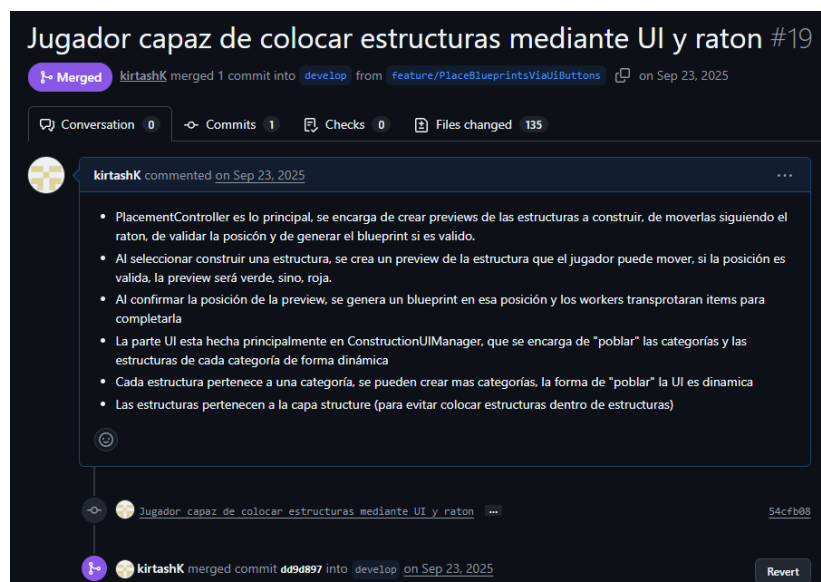


Figura 3.8: Ejemplo de Pull Request

Además del código fuente, se han versionado cuidadosamente los recursos del proyecto, como prefabs, escenas, *ScriptableObjects*, materiales, animaciones e interfaces. Esta

gestión resulta especialmente importante en Unity, ya que tanto la lógica de ejecución como los activos de contenido forman parte del producto final y deben mantenerse coherentes entre sí. La organización del repositorio y la separación funcional entre carpetas han contribuido igualmente a preservar la claridad estructural del proyecto y a reducir el riesgo de errores derivados de cambios no controlados.

3.7. Aseguramiento de calidad

El aseguramiento de calidad se ha basado en una combinación de buenas prácticas de diseño, validación continua y revisión manual del comportamiento del juego a medida que se incorporaban nuevas funcionalidades. Debido a la naturaleza del proyecto, la calidad no se ha limitado a comprobar que el código compila correctamente, sino que ha requerido verificar de forma explícita la interacción entre sistemas, la coherencia de los estados internos y la estabilidad general del prototipo durante la ejecución.

Desde el punto de vista del desarrollo, se han seguido principios de diseño orientados a la mantenibilidad y a la reducción de dependencias, aplicando criterios compatibles con los principios SOLID [7] siempre que ha sido posible. En particular, se ha priorizado la separación de responsabilidades, el uso de componentes reutilizables y la comunicación entre sistemas mediante interfaces, eventos y gestores centralizados. Esta organización facilita la detección de errores, reduce el impacto de los cambios y mejora la capacidad de evolución del proyecto.

Una parte importante del control de calidad ha consistido en validar los mecanismos diseñados para evitar inconsistencias en tiempo de ejecución. Entre ellos destacan el sistema de reservas, la prevención de condiciones de carrera, la correcta liberación de recursos al cancelar tareas, la gestión de la destrucción de objetos y la sincronización de atributos y modificadores. También se ha revisado que las interfaces se actualicen automáticamente cuando cambia el estado del sistema.

Además de estas comprobaciones funcionales, se han utilizado escenas de prueba y entornos reducidos para aislar subsistemas concretos antes de integrarlos en el juego completo. Esta práctica ha resultado especialmente útil en sistemas con fuerte interdependencia, como la recolección y el transporte de recursos y la producción, donde un fallo en un módulo puede afectar a varios componentes simultáneamente.

La gestión de la configuración, basada en control de versiones, también ha formado parte del aseguramiento de calidad, ya que ha permitido registrar cambios de forma trazable, revisar iteraciones anteriores y recuperar estados estables cuando ha sido necesario. De forma complementaria, el código fuente se ha documentado con comentarios y descripciones de propósito en los casos en que resultaba útil para facilitar su mantenimiento futuro.

Como criterio de aceptación, un subsistema se ha considerado correcto cuando cumple su función prevista, mantiene la coherencia con el resto de la arquitectura y no introduce errores visibles en la ejecución.

Parte II

Desarrollo

4. Requisitos del Sistema

En este capítulo se definen los objetivos y los requisitos del videojuego desarrollado, entendido como un prototipo de estrategia y gestión en 3D. Dado que no se parte de una solución informática previa que se pretenda adaptar o sustituir, el análisis de la técnica se utiliza principalmente como referencia conceptual para identificar mecánicas habituales en juegos del mismo género y trasladarlas a una arquitectura propia.

4.1. Objetivos de Negocio

Aunque el proyecto no responde a una necesidad organizativa externa, sí persigue una serie de objetivos de producto que desempeñan una función equivalente a los objetivos de negocio en una solución software convencional.

1. Obtener un prototipo jugable que permita al usuario gestionar una base mediante construcción, producción, investigación, economía y combate.
2. Disponer de una base técnica suficientemente sólida como para poder ampliar el juego en futuras iteraciones sin necesidad de reestructurar por completo la arquitectura interna. Para ello, el sistema debe separar claramente los datos configurables de la lógica de ejecución, centralizar la gestión de tareas y mantener una comunicación coherente entre subsistemas.
3. Demostrar la viabilidad de un enfoque modular aplicado al desarrollo de videojuegos de estrategia, donde múltiples sistemas independientes, como unidades, recursos, investigación, estructuras e interfaz, deban convivir y coordinarse de forma estable.

4.2. Objetivos del Sistema

El sistema desarrollado persigue como objetivo principal ofrecer un prototipo jugable de estrategia y gestión que permita al usuario construir una base, organizar recursos, producir unidades, investigar mejoras y defenderse de amenazas mediante un control directo e intuitivo de las principales mecánicas del juego. Para ello, la solución debe integrar en una misma arquitectura varios subsistemas especializados que colaboren de forma coherente durante la partida.

A nivel funcional, el sistema debe cubrir los siguientes objetivos:

- **Gestionar recursos, inventarios y almacenes de forma coherente**, permitiendo almacenar, transportar, reservar y consumir ítems sin generar inconsistencias entre subsistemas.

- **Permitir la construcción y gestión de estructuras**, incluyendo tanto edificios de producción y apoyo como estructuras defensivas y elementos especializados de la base.
- **Gestionar unidades con comportamientos diferenciados**, de forma que puedan existir trabajadores, soldados y enemigos con funciones y lógicas propias.
- **Habilitar la producción de unidades y el procesamiento de recursos**, mediante recetas configurables que transformen materiales de entrada en resultados útiles para el jugador.
- **Permitir la investigación de mejoras**, de modo que el progreso del jugador desbloquee nuevo contenido o modifique las características de unidades, estructuras y sistemas globales.
- **Ofrecer un sistema de selección y órdenes de tipo RTS**, que permita al jugador controlar unidades individuales o grupos de forma cómoda y consistente.
- **Integrar combate, defensa y retroalimentación visual**, garantizando que los enfrentamientos entre unidades y estructuras sean comprensibles y respondan correctamente a las acciones del jugador.
- **Proporcionar una interfaz de usuario clara y funcional**, que facilite el acceso a las interfaces de construcción, investigación, producción de unidades, inventario y tienda.

Además de los objetivos jugables, el sistema debe cumplir una serie de objetivos técnicos orientados a la calidad del software. En particular, la arquitectura debe ser **extensible**, para permitir añadir nuevas mecánicas o contenido sin reescribir partes importantes del código; **mantenible**, para facilitar la evolución del proyecto; y **robusta**, para gestionar correctamente reservas, cancelaciones, cambios de estado y situaciones de error sin comprometer la estabilidad general del juego.

4.3. Requisitos funcionales

A continuación se recogen los requisitos funcionales principales del sistema.

Tabla 4.1

Requisitos funcionales del sistema

RF-01	Gestión de trabajadores. El sistema deberá incluir trabajadores capaces de ejecutar tareas de transporte, construcción, investigación y apoyo a otros subsistemas de forma automática y autónoma.
RF-02	Asignación de tareas. Las tareas deberán gestionarse de forma centralizada, permitiendo registrarlas, cancelarlas y reasignarlas en función de la disponibilidad, prioridad y distancia de las entidades.
RF-03	Transporte de objetos. El sistema deberá permitir mover objetos entre el mundo, los almacenes y los consumidores de recursos, respetando reservas y evitando conflictos entre tareas concurrentes.

RF-04	Inventario y almacenes. El sistema deberá almacenar objetos por tipo, permitir consultas agregadas de stock y gestionar almacenes con capacidad limitada y categorías aceptadas.
RF-05	Construcción de estructuras. El sistema deberá permitir al jugador colocar y rotar estructuras en el terreno mediante un sistema de previsualización, validación de posición y confirmación de construcción.
RF-06	Gestión de planos. Las estructuras deberán existir inicialmente como planos y, una vez completadas las condiciones necesarias, convertirse en la estructura final.
RF-07	Selección de unidades y estructuras. El sistema deberá permitir la selección de una o varias entidades del juego para mostrar su información y emitir órdenes o acciones asociadas.
RF-08	Emisión de órdenes RTS. El jugador deberá poder ordenar movimientos y otras acciones a unidades seleccionadas, incluyendo desplazamiento en grupo y distribución de posiciones mediante formación.
RF-09	Procesado de recursos. El sistema deberá permitir transformar recursos de entrada en productos de salida siguiendo recetas configurables y tiempos de proceso definidos.
RF-10	Gestión de umbrales. El sistema deberá permitir definir umbrales de producción o recogida para que determinados procesos se detengan o reanuden en función del stock disponible.
RF-11	Investigación. El jugador deberá poder iniciar investigaciones que desbloqueen contenido nuevo o apliquen modificadores sobre estructuras, unidades o atributos globales.
RF-12	Sistema de atributos. El sistema deberá soportar atributos base y modificadores derivados de investigaciones, mejoras u otras fuentes, recalculando los valores finales cuando cambien los modificadores.
RF-13	Producción de unidades. El sistema deberá permitir registrar órdenes de producción en estructuras específicas, consumir recursos asociados y generar unidades tras completar el proceso correspondiente.
RF-14	Combate y defensa. Las unidades y estructuras deberán poder atacar objetivos enemigos, recibir daño y mostrar retroalimentación visual asociada al combate.
RF-15	Sistema de objetivos hostiles. El juego deberá identificar objetivos hostiles válidos según su categoría, facción y estado de vida para ser usados por torres, soldados, enemigos y agentes.
RF-16	Interfaz de usuario. El sistema deberá ofrecer paneles para los sistemas fundamentales, así como retroalimentación contextual para el jugador.
RF-17	Control de la velocidad del juego. El sistema deberá permitir pausar el juego y modificar su velocidad de ejecución mediante controles de interfaz o teclado.

4.4. Requisitos no funcionales

A continuación se describen los principales requisitos no funcionales del sistema.

Tabla 4.2

Requisitos no funcionales del sistema

RNF-01	Mantenibilidad. El código deberá organizarse en módulos desacoplados, con responsabilidades claras y una separación adecuada entre datos, lógica, interfaz y sistemas auxiliares.
RNF-02	Extensibilidad. La arquitectura deberá permitir añadir nuevo contenido o mecánicas sin reescribir por completo los subsistemas existentes.
RNF-03	Robustez. El sistema deberá gestionar correctamente casos límite como cancelaciones de tareas, destrucción de entidades, pérdida de referencias o cambios de estado mientras una acción está en curso.
RNF-04	Consistencia. Las reservas, los contadores internos, los umbrales y los estados de las tareas deberán mantenerse coherentes entre sí para evitar duplicidades, pérdidas de recursos o bloqueos.
RNF-05	Rendimiento. El sistema deberá evitar comprobaciones innecesarias por fotograma siempre que sea posible, priorizando eventos, temporizadores y actualizaciones selectivas de estado.
RNF-06	Usabilidad. La interfaz deberá presentar la información de forma clara y permitir al jugador interactuar con el juego de manera comprensible, especialmente en paneles de construcción, investigación y producción.
RNF-07	Portabilidad. El proyecto deberá ejecutarse dentro del entorno objetivo de Unity y poder compilarse en una plataforma de escritorio estándar sin depender de herramientas externas no justificadas.
RNF-08	Escalabilidad de contenido. Las mecánicas basadas en datos deberán poder ampliarse con nuevos <i>ScriptableObjects</i> , <i>prefabs</i> y configuraciones sin necesidad de modificar el código fuente.
RNF-09	Trazabilidad. Los sistemas principales deberán dejar suficiente rastro como para poder identificar con facilidad qué componente controla cada funcionalidad o de dónde provienen errores y advertencias gracias a los registros generados.
RNF-10	Calidad visual y de retroalimentación. Las acciones relevantes, como daño, curación, construcción completada o cambio de estado, deberán contar con algún tipo de respuesta visual o de interfaz que ayude al jugador a entender lo que ocurre.

4.5. Requisitos de información

El sistema deberá gestionar, como mínimo, los siguientes tipos de información:

- Datos de unidades: salud, daño, velocidad, alcance, *cooldown*, facción y configuración asociada.
- Datos de estructuras: categoría, coste, requisitos de construcción, atributos y comportamiento.
- Datos de investigación: nombre, descripción, efectos, prerequisites y estado de completado.
- Datos de recursos: tipo de objeto, categoría, valor, *prefab*, peso y restricciones de compra/venta.
- Datos de recetas: entradas, salidas, cantidades, tiempos de proceso, umbrales y consumo de combustible.
- Datos de tareas: tipo de trabajo, prioridad, posición, rango de interacción y estado de ejecución.
- Datos de interfaz: paneles, entradas del usuario, estados visibles y elementos desbloqueados.

Esta información se organizará principalmente mediante activos configurables del motor, especialmente *ScriptableObjects*, de forma que los valores de diseño puedan modificarse sin alterar la lógica interna del juego.

4.6. Reglas de negocio

A efectos del proyecto, las reglas de negocio se interpretan como reglas de juego y restricciones funcionales que el sistema debe respetar para mantener la coherencia entre sus distintos subsistemas.

- Una estructura no podrá colocarse si la posición no es válida o si colisiona con otro elemento del escenario.
- Algunas estructuras podrán estar limitadas a una única instancia por jugador.
- Una estructura no podrá construirse si no se han entregado los recursos necesarios.
- Un trabajador no podrá ejecutar simultáneamente dos tareas incompatibles.
- Una tarea reservada no podrá ser asignada a otro trabajador mientras la reserva siga activa.
- Un objeto no podrá pertenecer a más de un almacén o reserva de transporte al mismo tiempo.
- Un almacén solo podrá aceptar objetos compatibles con las categorías que tenga configuradas y siempre dentro de su capacidad disponible.
- Un almacén no podrá superar su capacidad efectiva, teniendo en cuenta tanto los objetos almacenados como las reservas activas.
- Un recurso no podrá extraerse si ya ha alcanzado su umbral de almacenamiento.

- Un procesador de recursos no podrá continuar generando productos ni solicitando recursos de entrada si se supera el umbral definido para ese producto.
- Una investigación no podrá activarse si no se cumplen sus prerequisites.
- Solo una investigación puede estar activa a la vez.
- Solo podrán aplicarse efectos o modificadores si la investigación ha sido completada.
- Los atributos finales de una unidad o estructura deberán recalcularse cuando cambien los modificadores activos.
- Una unidad no podrá atacar objetivos de su misma facción.
- Un rival no podrá ser atacado si su salud es igual o menor a 0.
- Una torre, soldado o enemigo solo podrá atacar si el objetivo está dentro de su rango y es válido para el sistema de objetivos.
- Una orden de producción no podrá avanzar si no se dispone de los recursos necesarios o si el edificio no puede producir en ese momento.
- Una orden o proceso cancelado deberá liberar las reservas y el estado asociado.
- Las interfaces de construcción, investigación, producción e inventario deberán reflejar siempre el estado real del sistema.

Estas reglas son esenciales para mantener la coherencia entre los distintos subsistemas del juego y evitar estados inválidos durante la partida.

5. Análisis del Sistema

Este capítulo cubre el análisis del sistema de información a desarrollar, haciendo uso del lenguaje de modelado UML.

5.1. Modelo Conceptual

En la Figura 5.1 se puede observar el modelo conceptual; se apoya en una arquitectura basada en datos, donde la información configurable se separa de la lógica de ejecución. En este apartado se representan únicamente las entidades de dominio del videojuego y sus relaciones estáticas.

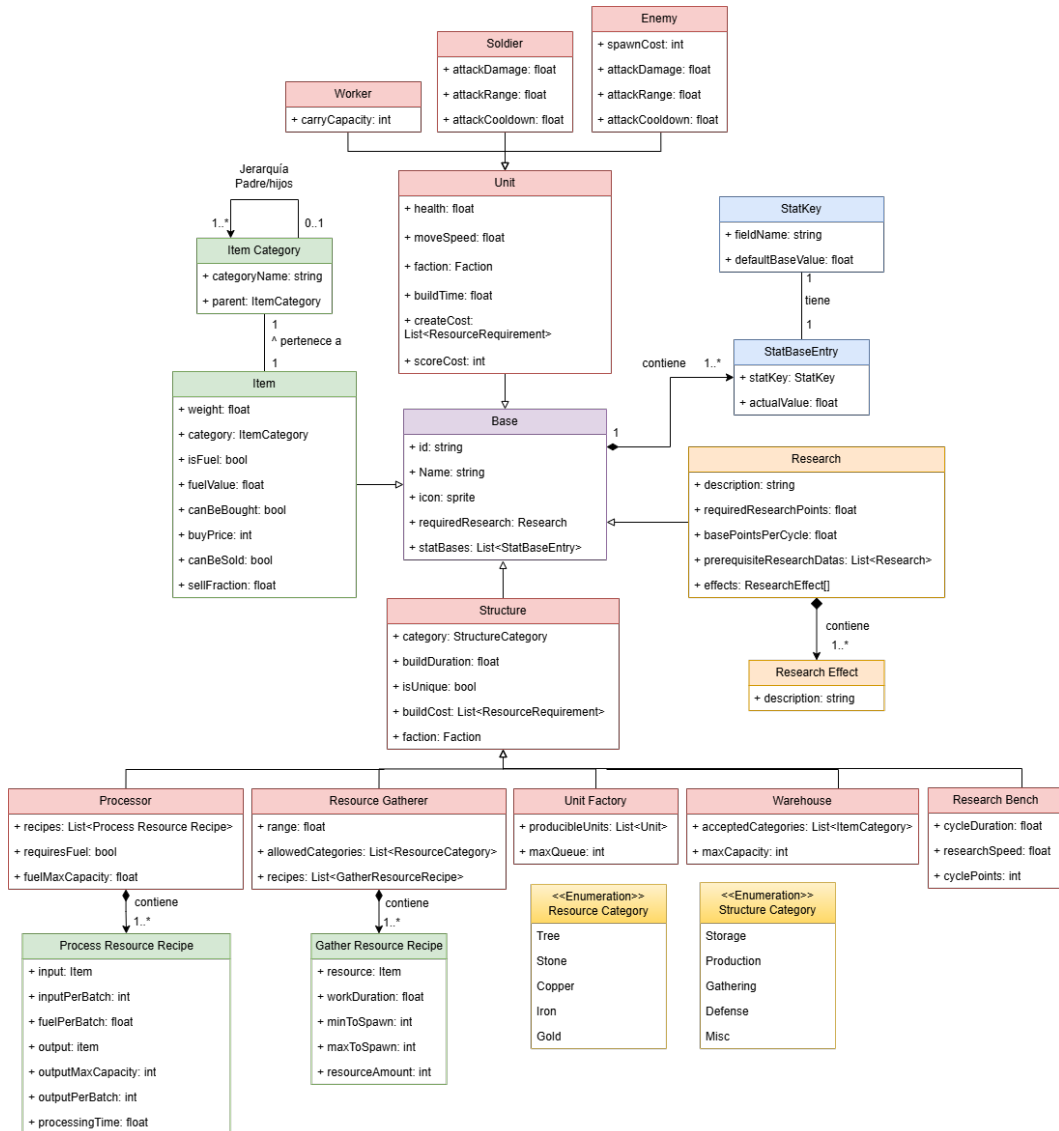


Figura 5.1: Modelo conceptual

5.2. Modelo de Casos de Uso

En la Figura 5.2 se puede observar el diagrama de casos de uso, donde el usuario/jugador es el actor principal

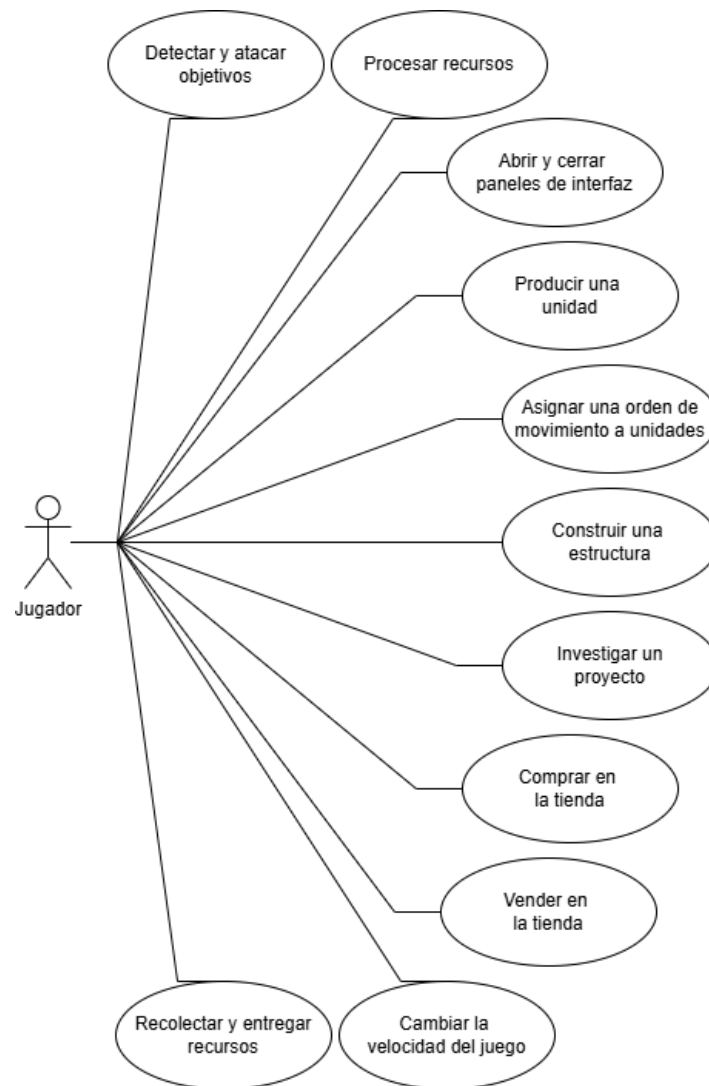


Figura 5.2: Diagrama de caso de uso

A continuación, se describen los casos de uso más representativos del sistema.

CU-01. Construir una estructura

Actor principal: Jugador

Actores secundarios: Trabajador, sistema de tareas

Precondiciones:

- El jugador dispone de la estructura deseada en el panel de construcción.

Postcondiciones:

- La estructura colocada está construida.

Escenario principal:

1. El **jugador** abre el panel de construcción.
2. El **sistema** muestra la interfaz con las estructuras disponibles, separadas por categorías.
3. El **jugador** selecciona una estructura disponible.
4. El **sistema** muestra una previsualización sobre el terreno.
5. El **jugador** confirma la colocación y rotación.
6. El **sistema** valida la posición, crea el plano correspondiente y registra la tarea de construcción.
7. El **sistema de tareas** asigna la tarea de construcción a los trabajadores más cercanos.
8. Los **trabajadores** transportan los recursos necesarios.
9. Los **trabajadores** construyen la estructura una vez el plano tiene todos los recursos necesarios.
10. La construcción finaliza y el plano se sustituye por la estructura terminada.

Escenarios alternativos:

- Si la posición no es válida, la estructura no se puede colocar.

CU-02. Asignar una orden de movimiento a unidades

Actor principal: Jugador

Actores secundarios: Unidad/es seleccionada/s, sistema de órdenes

Precondiciones:

- Existe al menos una unidad seleccionable.

Postcondiciones:

- La unidad recibe y ejecuta la orden emitida.
- Si se selecciona un grupo, cada unidad obtiene un destino adaptado a la formación.

Escenario principal:

1. El **jugador** selecciona una o varias unidades.
2. El **sistema** muestra efectos visuales para indicar que unidades han sido seleccionadas.

3. El **jugador** emite una orden de movimiento.
4. El **sistema** calcula los destinos correspondientes, asignando una formación si hay más de una unidad.
5. El **sistema** muestra efectos visuales para indicar el destino de cada unidad.
6. Cada unidad comienza a desplazarse hacia su posición asignada.

Escenarios alternativos:

- Si el destino no es navegable, la orden se cancela.
- Si la selección no contiene unidades válidas, no se emite ninguna orden.

CU-03. Investigar un proyecto

Actor principal: Jugador

Actores secundarios: Trabajador, sistema de investigación, sistema de tareas

Precondiciones:

- Existe un proyecto de investigación disponible.

Postcondiciones:

- El proyecto de investigación está completado.
- El sistema actualiza los desbloques y modificadores correspondientes.

Escenario principal:

1. El **jugador** abre el panel de investigación.
2. El **sistema** muestra la interfaz de investigación, que muestra cada proyecto de investigación.
3. El **jugador** selecciona un proyecto de investigación disponible.
4. El **sistema de investigación** muestra información detallada (descripción, efectos y progreso) del proyecto seleccionado.
5. El **jugador** activa el proyecto seleccionado.
6. El **sistema de investigación** comprueba los prerequisites y registra la tarea de investigación.
7. El **sistema de tareas** asigna la tarea de investigación al trabajador disponible más cercano a un edificio de investigación.
8. El **trabajador** se mueve al edificio de investigación asignado.
9. El **trabajador** dedica tiempo a avanzar el proyecto de investigación.
10. Al completarse, el **sistema** aplica los efectos y actualiza el estado del juego.

Escenarios alternativos:

- Si no se cumplen los prerequisites, la investigación no puede activarse.
- Si se interrumpe la tarea o surge una tarea con mayor prioridad, el progreso parcial se guarda, pudiéndose continuar la investigación más adelante.

CU-04. Producir una unidad

Actor principal: Jugador

Actores secundarios: trabajador, sistema de producción, sistema de inventario, sistema de tareas

Precondiciones:

- Existe una estructura productora.

Postcondiciones:

- La orden de producción queda registrada.
- Los recursos necesarios quedan consumidos o reservados.
- La unidad solicitada se genera al finalizar el proceso.

Escenario principal:

1. El **jugador** selecciona una estructura de producción.
2. El **sistema** abre la interfaz con la lista de unidades producibles por esa estructura.
3. El **jugador** selecciona la unidad deseada.
4. El **sistema** verifica costes y requisitos.
5. El **sistema de producción** registra la orden de producción.
6. El **sistema de tareas** asigna una tarea de transporte de los recursos necesarios al trabajador disponible más cercano.
7. El **trabajador** recoge y entrega recursos a la estructura de producción.
8. Al recibir todos los recursos, el **sistema de producción** pasa la orden de producción al estado de listo.
9. El **sistema de producción** pasa la orden de producción al estado de procesando.
10. Al finalizar el estado, el **sistema de producción** crea la unidad seleccionada.

Escenarios alternativos:

- Si la unidad requiere una investigación no completada, no se puede seleccionar la unidad.
- Si la estructura tiene la cola de producción llena, no se puede seleccionar la unidad.

- Si la estructura tiene la cola de procesamiento llena pero la de producción no, la orden de producción tendrá que esperar a que haya hueco en la cola de procesamiento, respetando orden FIFO respecto a otras órdenes.

CU-05. Procesar recursos

Actor principal: Estructura de procesamiento

Actores secundarios: Trabajador, sistema de inventario, sistema de tareas

Precondiciones:

- Existe una estructura procesadora.

Postcondiciones:

- Los recursos de entrada son consumidos.
- Se generan los recursos de salida correspondientes.
- El sistema actualiza el estado del proceso y los umbrales.

Escenario principal:

1. La **estructura** verifica si el inventario global de uno o más de los recursos de salida procesados en la estructura es menor al umbral configurado por el jugador o por defecto.
2. La **estructura** solicita recursos de entrada.
3. El **sistema de tareas** registra y asigna la tarea de transporte al trabajador disponible más cercano.
4. El **trabajador** recoge y entrega los recursos de entrada en la estructura de procesamiento.
5. La **estructura** almacena los recursos de entrada y libera las reservas asociadas.
6. La **estructura** inicia el procesamiento de los recursos de entrada.
7. Al terminar de procesar, la **estructura** genera los recursos procesados de salida.

Escenarios alternativos:

- Si no hay recursos de entrada suficientes, el procesamiento no empieza.
- Si se alcanza el umbral configurado para los recursos de salida, la estructura deja de solicitar recursos de entrada.

CU-06. Recolectar y entregar recursos

Actor principal: Trabajador

Actores secundarios: Sistema de tareas, recurso, estructura consumidora

Precondiciones:

- Existe un recurso disponible.

Postcondiciones:

- El objeto queda almacenado o entregado al consumidor correspondiente.
- Las reservas asociadas se liberan.
- El stock global del sistema de inventario se actualiza.

Escenario principal:

1. Una **estructura consumidora** solicita un recurso.
2. El **sistema de tareas** asigna al recurso una tarea de transporte y asigna al trabajador disponible más cercano la tarea.
3. El **trabajador** recoge o recolecta el recurso.
4. El **trabajador** se mueve hasta la estructura consumidora que solicitó el recurso.
5. El **trabajador** entrega el recurso.
6. La **estructura consumidora** almacena el recurso y libera las reservas asociadas.

Escenarios alternativos:

- Si el trabajador es interrumpido o atacado mientras transporta el recurso, el recurso es soltado, las reservas asociadas se liberan y la tarea se reinicia.

CU-07. Detectar y atacar objetivos

Actor principal: Sistema de combate

Actores secundarios: Unidad, torre, objetivo hostil

Precondiciones:

- Existe una unidad o torre.
- Existe un objetivo válido dentro del rango de detección.

Postcondiciones:

- El objetivo recibe daño o es destruido.
- El sistema actualiza la salud y emite retroalimentación visual.

Escenario principal:

1. La **unidad o torre** detecta un objetivo hostil.
2. La **unidad o torre** comprueba que el objetivo está dentro de rango y está vivo.
3. La **unidad o torre** ejecuta un ataque.
4. El **sistema** reduce la salud del objetivo en base al daño del ataque.

5. El **sistema** actualiza los eventos visuales y de estado.

Escenarios alternativos:

- Si no hay objetivos válidos o se alejan del rango de detección, la unidad permanece en espera a nuevos objetivos.
- Si al recibir daño, la salud del objetivo baja a 0, el objetivo cuenta como eliminado.

CU-08. Abrir y cerrar paneles de interfaz

Actor principal: Jugador

Actores secundarios: Sistema de interfaz

Precondiciones:

- Existe un panel compatible con la acción solicitada.

Postcondiciones:

- El panel solicitado queda visible o cerrado.
- La interfaz refleja el estado actual del sistema.

Escenario principal:

1. El **jugador** pulsa la tecla correspondiente o el elemento de la interfaz correspondiente.
2. El **sistema** muestra el panel asociado a la tecla o elemento.
3. El **jugador** interactúa con su contenido, específico a cada panel.
4. Al acabar, el **jugador** desea cerrar el panel mediante la tecla global correspondiente o pulsando fuera del panel abierto.
5. El **sistema** oculta el panel.

Escenarios alternativos:

- Si el panel ya estaba abierto, la acción oculta el panel.
- Si el sistema detecta una entrada inválida, no se produce ningún cambio.

CU-09. Comprar en la tienda

Actor principal: Jugador

Actores secundarios: Tienda, sistema de inventario, sistema de puntuación

Precondiciones:

- La interfaz de la tienda está abierta.

Postcondiciones:

- La puntuación del jugador se reduce en el importe correspondiente.
- El objeto comprado se añade al almacén de la tienda.
- La interfaz actualiza la cantidad a comprar, el coste total y el estado de la operación.

Escenario principal:

1. El **jugador** selecciona la tienda.
2. El **sistema** muestra la interfaz de la tienda, presentando un listado dinámico de recursos que se pueden comprar y/o vender.
3. El **jugador** introduce una cantidad a comprar de uno o varios recursos.
4. El **sistema** comprueba si el recurso se puede comprar.
5. El **sistema** actualiza la cantidad seleccionada y el precio total.
6. El **jugador** confirma la operación.
7. El **sistema** verifica que el jugador dispone de puntuación suficiente.
8. El **sistema** reduce la puntuación del jugador en función del coste total.
9. El **sistema** genera los recursos en el almacén de la tienda.
10. El **sistema** refresca la interfaz.

Escenarios alternativos:

- Si el jugador no dispone de saldo suficiente, la compra no se puede completar.
- Si el objeto no puede comprarse, se muestra un aviso en la interfaz de la tienda explicándolo.

CU-10. Vender en la tienda

Actor principal: Jugador

Actores secundarios: Tienda, sistema de inventario, almacenes, sistema de puntuación

Precondiciones:

- La interfaz de la tienda está abierta.

Postcondiciones:

- La cantidad vendida se elimina de los almacenes.
- El jugador recibe la puntuación correspondiente.
- La interfaz actualiza el stock, el precio de venta y el estado de la operación.

Escenario principal:

1. El jugador selecciona la tienda.
2. El **sistema** muestra la interfaz de la tienda, mostrando un listado dinámico de recursos que se pueden comprar y/o vender.
3. El **jugador** introduce una cantidad a vender de uno o varios recursos.
4. El **sistema** comprueba si el recurso se puede vender.
5. El **sistema** actualiza la cantidad seleccionada y el valor total.
6. El **jugador** confirma la operación.
7. El **sistema** comprueba que el objeto puede venderse.
8. El **sistema** comprueba que existe stock suficiente en los almacenes del jugador.
9. El **sistema** retira las unidades correspondientes de los almacenes del jugador.
10. El **jugador** recibe el valor de la venta en su puntuación.
11. El **sistema** refresca la interfaz.

Escenarios alternativos:

- Si el objeto no puede venderse, la operación no se puede completar.
- Si no existe stock suficiente, la operación no se puede completar.

CU-11. Cambiar la velocidad del juego

Actor principal: Jugador

Actores secundarios: Sistema de velocidad

Precondiciones:

- El juego se encuentra en ejecución.

Postcondiciones:

- La velocidad de simulación del juego se actualiza.
- La interfaz refleja el nuevo estado de pausa o aceleración.

Escenario principal:

1. El **jugador** pulsa el botón en la interfaz o la tecla correspondiente al cambio de velocidad.
2. El **sistema de velocidad** actualiza el valor de velocidad actual del juego al solicitado, modifica la velocidad de simulación, la cual avanza más rápido, más lento o queda en pausa según la opción seleccionada.
3. La **interfaz** muestra el estado actualizado, haciendo más visible la velocidad seleccionada y haciendo más transparentes las no seleccionadas.

Escenarios alternativos:

- Si el jugador selecciona la misma velocidad que ya está activa, el sistema no realiza cambios adicionales.
- Si el juego se encuentra en pausa, al reanudar se vuelve al modo de velocidad previo a la pausa.

5.3. Modelo de Interfaz de Usuario

La interfaz del sistema debe representarse mediante un prototipo de baja fidelidad y un diagrama de navegación entre pantallas. En este proyecto, la navegación puede organizarse alrededor de un HUD principal desde el que el jugador accede a distintos paneles especializados. Cabe destacar que múltiples elementos de la interfaz, por ejemplo, la interfaz de investigación, muestran de forma dinámica otros elementos, de forma que al añadir nuevo contenido, por ejemplo, otro proyecto de investigación, no hace falta modificar la interfaz, sino que el sistema añade el contenido a la interfaz automáticamente.

Las pantallas o paneles principales son:

- **Interfaz principal:** en la Figura 5.3 se puede ver información general como recursos almacenados (inventario), velocidad del juego, enemigos activos, aviso de oleadas y acceso a menús. Esta interfaz es la que el jugador va a ver durante gran parte del juego, por lo que se ha procurado dejar el centro de la pantalla libre, excepto para el aviso de oleadas, que aparece brevemente antes de cada oleada y permite que el jugador prepare sus tropas.
- **Interfaz de construcción:** en la Figura 5.4 se puede ver la lista de estructuras disponibles, separadas por categorías, además de la interfaz principal.
- **Interfaz de investigación:** en la Figura 5.5 se puede ver la lista de proyectos de investigación, al seleccionar un proyecto se muestra información detallada para ese proyecto, como la descripción y la lista de efectos al completarse, además de una barra con el progreso del proyecto y un botón para marcar el proyecto como activo.
- **Interfaz de producción de unidades:** en la Figura 5.6 se puede ver la lista de unidades disponibles por esa estructura, la cola de unidades en producción junto con sus estados (En producción, esperando hueco, esperando materiales).
- **Interfaz de la tienda:** en la Figura 5.7 se puede ver la lista de recursos disponibles para comprar y/o vender, cada recurso muestra el coste en puntuación para comprar y vender, además de botones para aumentar o decrementar la cantidad a comprar/vender y un cuadro para introducir la cantidad por texto. También se muestra el balance total de la operación, mensajes de error en caso de existir y botones para confirmar la operación o devolverla al estado inicial.

En la Figura 5.8 se puede observar el diagrama de navegación. Al lanzar la aplicación, primero se muestra el logo de Unity y, a continuación, el menú principal, donde el jugador puede salir del juego, abrir los ajustes, crear una partida nueva o cargar una partida. Una vez cargada la partida, tanto al seleccionar estructuras concretas como al



Figura 5.3: Mockup interfaz principal

interactuar con los botones de la interfaz principal se pueden abrir las interfaces más específicas vistas en los mockups anteriormente. Para cerrar estas interfaces, el jugador puede pulsar fuera del área de la interfaz o pulsar un botón.

5.4. Modelo de Comportamiento

A continuación, se muestran los diagramas de secuencia para algunos de los casos de uso considerados más interesantes y únicos, en concreto los casos de uso CU-01, CU-03, CU-05 y CU-11.

Construir una estructura

En la Figura 5.9 se puede ver que el jugador selecciona una estructura en el panel de construcción, la coloca en el terreno y confirma la posición. El controlador de colocación valida la posición, crea el plano y registra la nueva tarea de construcción. El sistema de tareas asigna la tarea de construcción a los trabajadores, quienes entregan los recursos necesarios y el plano se convierte en la estructura final.

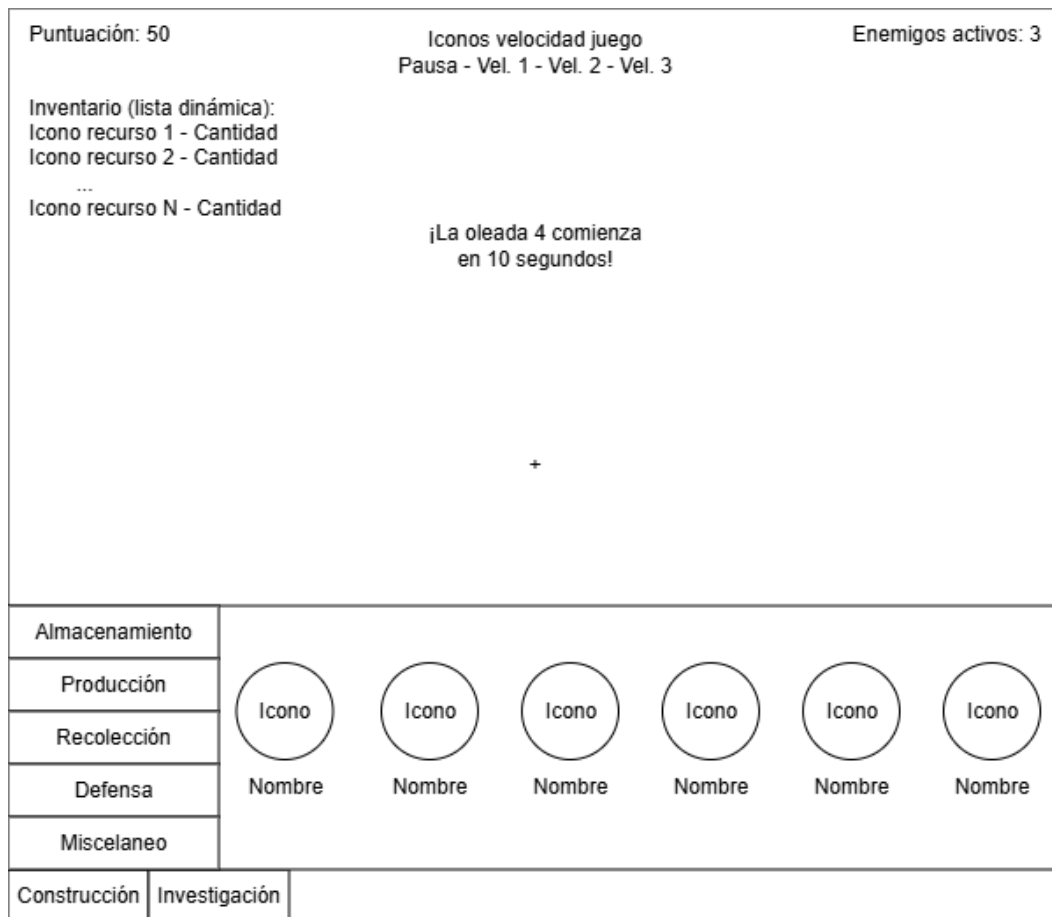


Figura 5.4: Mockup interfaz de construcción

Investigar un proyecto

En la Figura 5.10 se puede ver que el jugador selecciona un proyecto de investigación en el panel de investigación. El sistema verifica los prerequisites del proyecto y muestra información detallada. El jugador activa el proyecto, el sistema de investigación registra la tarea de investigación. El sistema de tareas asigna la tarea de investigación a los trabajadores, quienes investigan en las estructuras correspondientes hasta completar el proyecto, momento en el que el sistema de investigación aplica los efectos del proyecto.

Procesar recursos

En la Figura 5.11 se puede ver como una estructura de procesamiento verifica si el inventario global de un recurso de salida es menor que el configurado en su umbral, de ser así, solicita recursos de entrada para procesarlos. El sistema de tareas registra y asigna la tarea de recogida y transporte a los trabajadores más cercanos, los cuales recogen y entregan los recursos de entrada a la estructura de procesamiento. La estructura almacena los recursos y libera las reservas asociadas, luego inicia el procesamiento de los recursos de entrada. Al acabar de procesar, la estructura genera los recursos de salida.

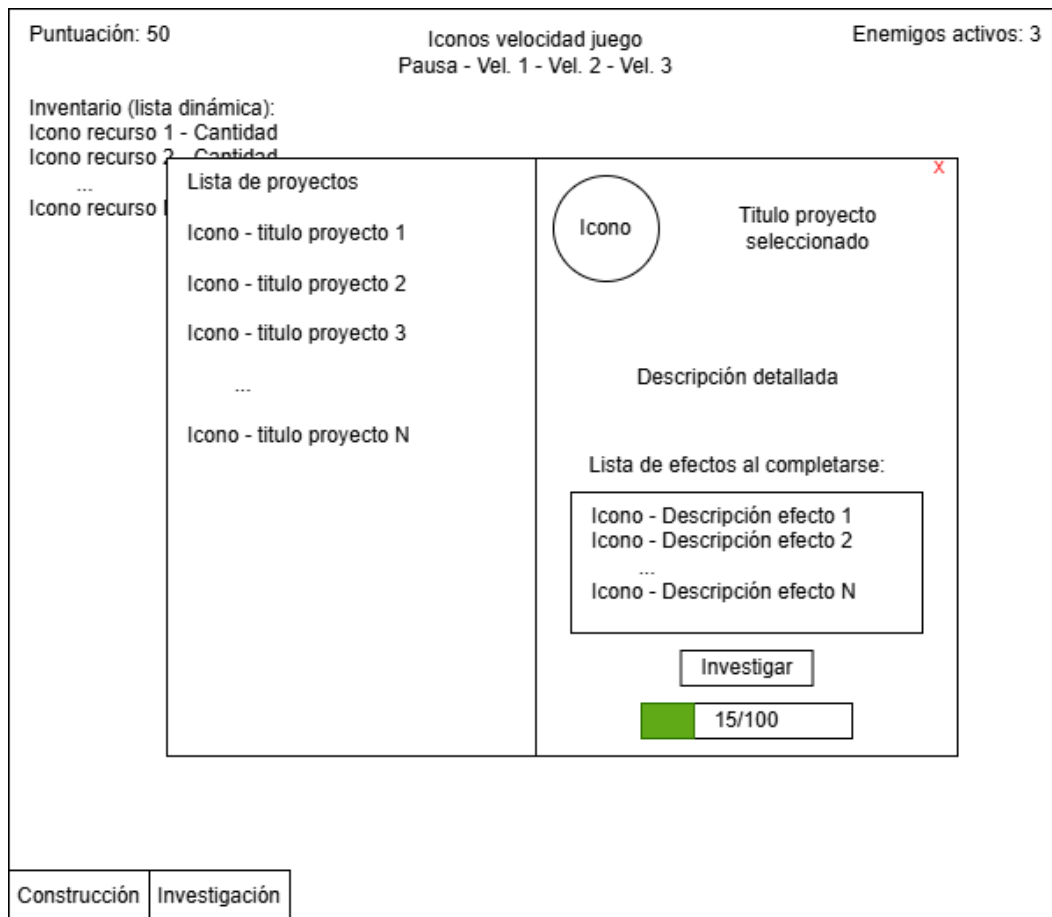


Figura 5.5: Mockup interfaz de investigación

Cambiar la velocidad del juego

En la Figura 5.12 se puede ver que el jugador solicita cambiar la velocidad del juego mediante un botón de la interfaz o la tecla correspondiente. El sistema de velocidad almacena la velocidad actual (para reanudar en caso de pausa) y actualiza la velocidad de la simulación. La interfaz encargada de mostrar la velocidad actualiza el estado actual, haciendo visible al jugador que el cambio solicitado se ha cumplido.

Puntuación: 50

Iconos velocidad juego
Pausa - Vel. 1 - Vel. 2 - Vel. 3

Enemigos activos: 3

Inventario (lista dinámica):
Icono recurso 1 - Cantidad
Icono recurso 2 - Cantidad
...
Icono recurso N - Cantidad

¡La oleada 4 comienza
en 10 segundos!

+

Cola de producción			Nombre de la estructura	
Icono Soldado En producción	Icono Trabajador Esperando hueco	Icono Soldado Esperando materiales	Capacidad: 3/4 En proceso: 1/1	
			Lista de unidades disponibles	
			Icono Unidad 1	Icono Unidad N

Construcción

Investigación

Figura 5.6: Mockup interfaz de producción de unidades

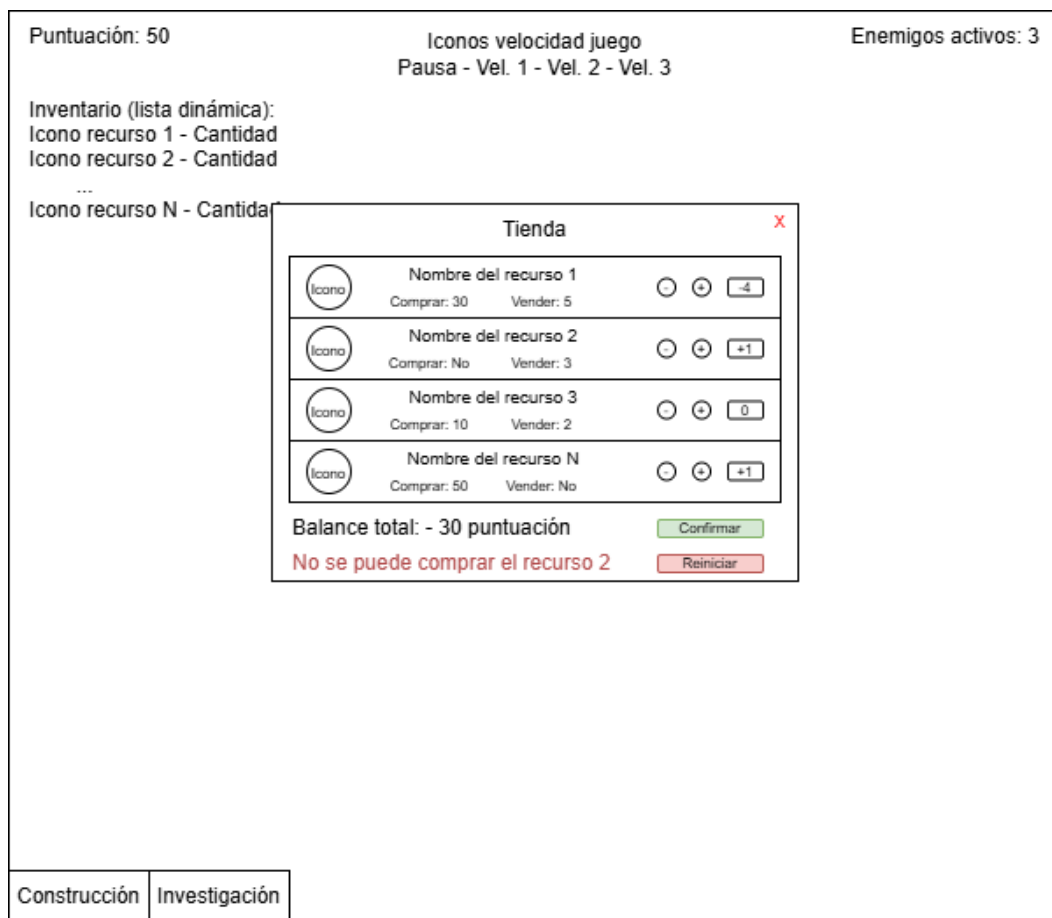


Figura 5.7: Mockup interfaz de la tienda

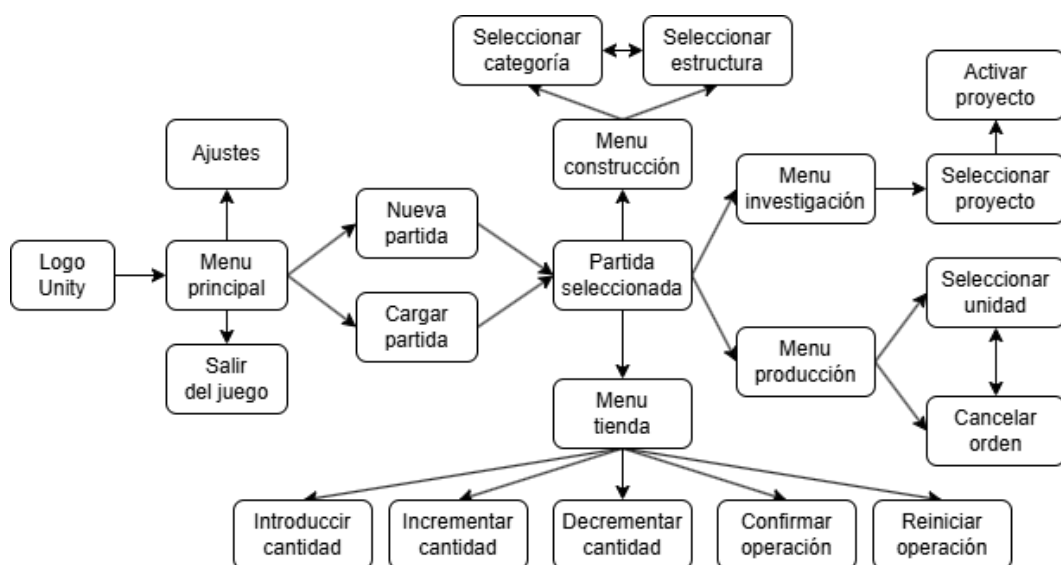


Figura 5.8: Diagrama de navegación

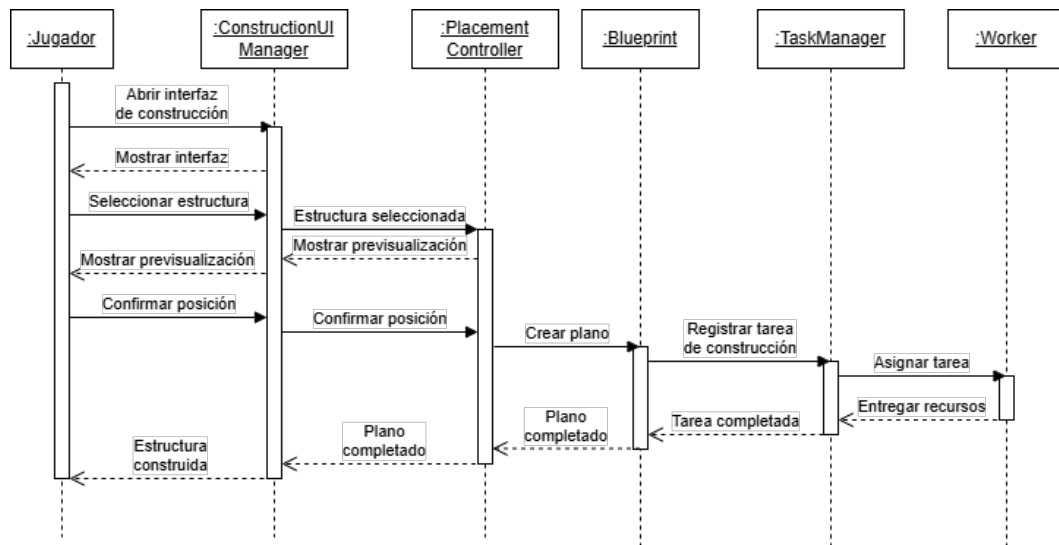


Figura 5.9: Diagrama de secuencia para construir una estructura

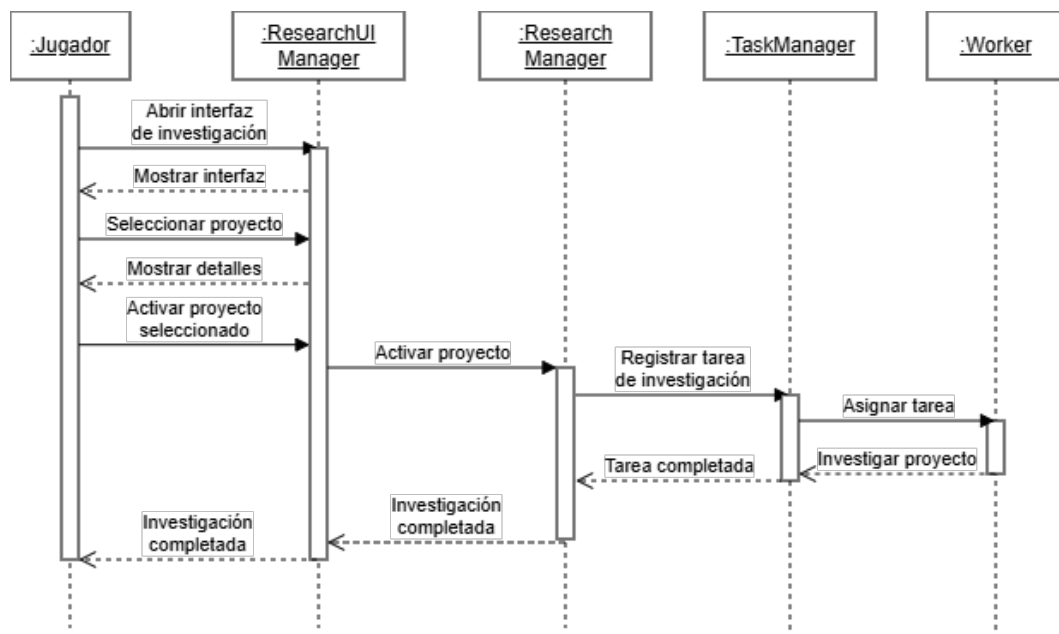


Figura 5.10: Diagrama de secuencia para investigar un proyecto

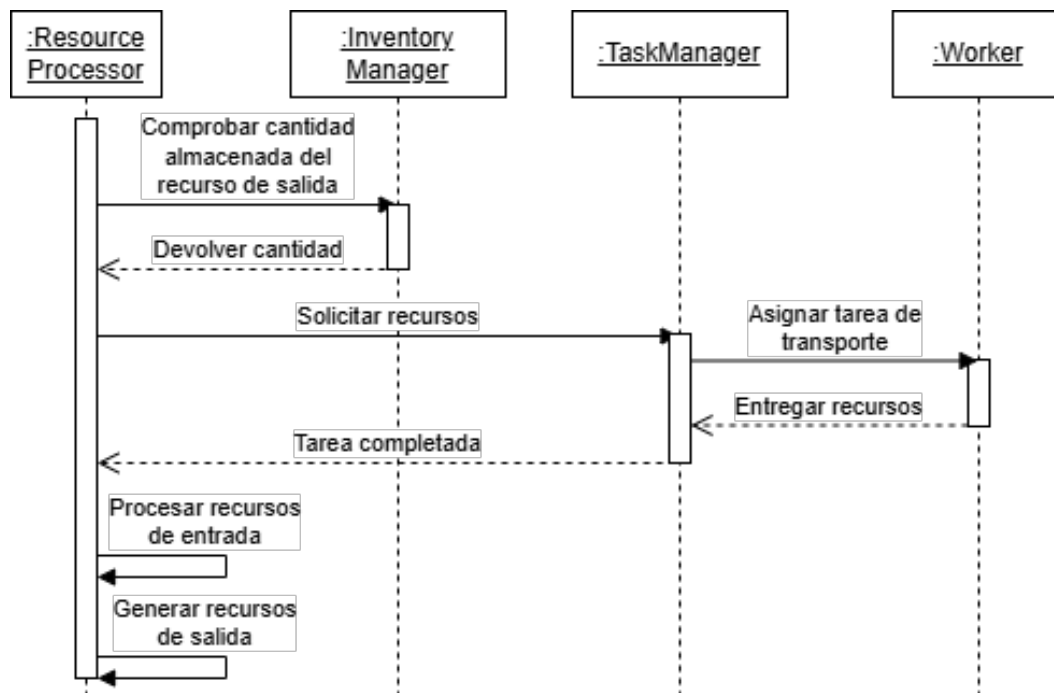


Figura 5.11: Diagrama de secuencia para procesar recursos

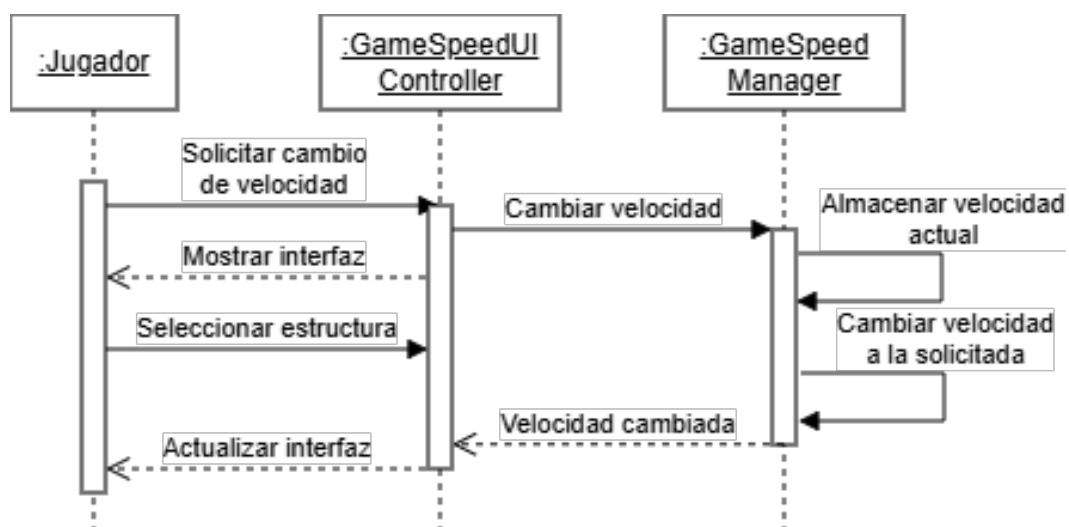


Figura 5.12: Diagrama de secuencia para cambiar la velocidad del juego

6. Diseño del Sistema

En este capítulo se recoge el diseño de la arquitectura del sistema informático, el diseño físico de datos y el diseño detallado de la interfaz de usuario.

6.1. Arquitectura del Sistema

La arquitectura del sistema se ha diseñado siguiendo un enfoque modular y orientado a componentes, propio de Unity, en el que cada subsistema asume una responsabilidad concreta y se comunica con el resto mediante eventos, interfaces y gestores centralizados. Esta organización permite separar de forma clara las principales preocupaciones del proyecto: datos configurables, lógica de juego, inteligencia de las unidades, interfaz de usuario y sistemas de soporte.

Desde un punto de vista conceptual, la arquitectura puede representarse mediante un modelo similar al propuesto por C4. En el nivel más alto, el sistema está formado por el jugador, la escena de juego y los distintos subsistemas que componen la experiencia jugable. En un nivel intermedio, pueden distinguirse los siguientes bloques funcionales:

- **Datos y configuración:** activos *ScriptableObject* que definen unidades, estructuras, ítems, recetas, investigaciones, atributos y categorías.
- **Lógica de juego:** sistemas de construcción, tareas, inventario, almacenes, procesamiento de recursos, investigación, combate y producción.
- **Control y selección:** gestión de la selección de entidades, emisión de órdenes y control de la cámara.
- **Interfaz de usuario:** paneles de construcción, investigación, producción, inventario, depuración y elementos de retroalimentación visual.
- **Soporte transversal:** gestores globales de tareas, selección, inventario, atributos, puntuación, velocidad del juego y depuración.

La arquitectura lógica del proyecto no sigue un patrón clásico en capas propio de aplicaciones empresariales, sino una estructura híbrida adaptada a un videojuego en tiempo real. La capa de datos queda separada de la capa de comportamiento mediante *ScriptableObjects*, mientras que la lógica en escena se implementa mediante componentes reutilizables que pueden asociarse a prefabs de unidades y estructuras. Los gestores globales centralizan aquellos aspectos que requieren una visión común del estado del juego, como la gestión de tareas o el inventario.

Esta organización presenta varias ventajas. Por un lado, favorece la reutilización de componentes y la extensibilidad del proyecto. Por otro lado, permite introducir nuevas mecánicas sin modificar en profundidad los sistemas ya existentes. Por ejemplo, una nueva unidad o estructura puede incorporarse definiendo un nuevo activo de datos y

asociando los componentes de comportamiento necesarios, sin necesidad de alterar el resto de la arquitectura.

6.1.1. Diseño de alto nivel

En la Figura 6.1 se puede observar el diagrama de contexto según [4]. El sistema se despliega como una aplicación de escritorio autónoma, por lo que el único actor externo relevante es el jugador, que interactúa con el videojuego a través de la interfaz gráfica y los controles de entrada, el videojuego usa el motor gráfico Unity.

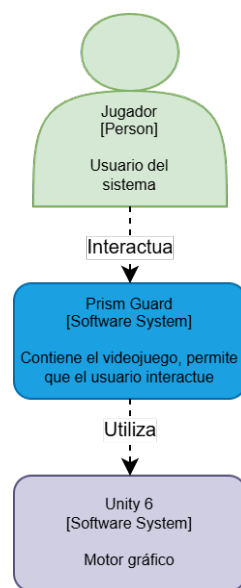


Figura 6.1: Diagrama de contexto

En la Figura 6.2 se puede observar el diagrama de componentes de alto nivel. A continuación, se explica cada capa:

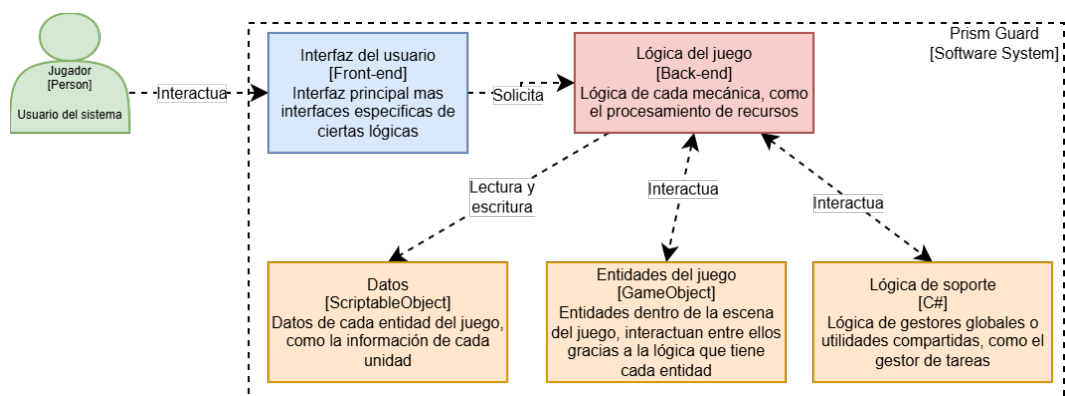


Figura 6.2: Diagrama de componentes de alto nivel

Capa de presentación

Esta capa se encarga de recibir las acciones del jugador y mostrar el estado del juego de forma clara y comprensible, además de comunicarse con la capa de la lógica del juego para enviar las acciones del jugador. Incluye la cámara, la selección de entidades, los paneles de interfaz, los *tooltips* y los elementos de retroalimentación visual.

Capa de lógica de juego

Esta capa ejecuta las reglas del juego y actualiza el estado de las entidades, comunicándose con la capa de las entidades del juego y la capa de soporte, y leyendo y escribiendo en la capa de datos.

Capa de las entidades del juego

Agrupar los componentes de las escenas del juego que gobiernan el comportamiento del mundo jugable, como los trabajadores, soldados, enemigos, torres, muros, planos de construcción, almacenes, procesadores de recursos, edificios de investigación, estructuras productoras de unidades y más.

Capa de datos

Contiene los activos configurables que describen el contenido del juego, como atributos base, tipos de recursos, recetas, investigaciones, categorías, estructuras y unidades. El uso de datos externos permite ajustar valores sin recompilar la lógica principal, además de reducir la memoria usada, teniendo los datos de cada entidad en un solo sitio, en vez de cada entidad con sus propios datos duplicados.

Capa de soporte

Incluye gestores globales y utilidades compartidas, como el sistema de tareas, el gestor de selección, el sistema de velocidad de juego, la gestión de puntuación, la retroalimentación de combate y los sistemas de depuración.

La comunicación entre estas capas se realiza principalmente mediante interfaces y eventos. Por ejemplo, una unidad puede notificar que ha recibido daño sin conocer qué elemento de la interfaz mostrará ese cambio; del mismo modo, una estructura puede registrarse como consumidora de tareas sin depender directamente del origen de dichas tareas.

6.1.2. Diseño detallado

A la hora de diseñar el juego, se ha procurado que el flujo general del juego se organice en torno a varios ciclos principales. El jugador selecciona una acción en la interfaz; la

acción se traduce en una orden o en la creación de una entidad, el sistema registra el cambio de estado y los componentes implicados actualizan su comportamiento a partir de ello. Por ejemplo, una investigación completada puede activar modificadores de atributos, lo que obliga a recalcular los valores finales de las entidades afectadas mediante eventos.

Esta separación entre comportamiento, datos y control facilita que los sistemas se comuniquen sin dependencias innecesarias. Así, una modificación en la lógica de producción no obliga a reescribir la interfaz, y un cambio en la interfaz no obliga a alterar el modelo de datos.

En la Figura 5.1 vista anteriormente, se puede ver el modelo conceptual con las asociaciones entre clases. A continuación, se explican algunas de las entidades más relevantes:

- **BaseData**: clase base de los datos del juego, de la que derivan unidades, estructuras y otros elementos configurables.
- **StatKey**, **StatBaseEntry** y **StatModifierManager**: definen y calculan los atributos de las entidades y permiten aplicar modificadores dinámicos.
- **Unit** y **Structure**: representan entidades instanciadas en escena, con salud, facción, combate y eventos asociados.
- **Worker**: componente de las unidades de trabajo, encargado de ejecutar tareas y gestionar el inventario personal.
- **TaskManager** e **ITask**: coordinan las tareas disponibles y la asignación de trabajadores.
- **Blueprint**: representa una estructura en fase de construcción y registra la tarea asociada.
- **ResourceGatherer**, **ResourceProcessor**, **Warehouse**, **UnitFactory** y **ResearchBench**: estructuras que consumen, producen o transforman recursos y trabajo.
- **Sensor**, **ITarget** y **TargetSearchUtility**: proporcionan el sistema de detección de objetivos para combate y defensa.
- **SelectionManager**, **OrderManager** y componentes relacionados: gestionan la selección múltiple y las órdenes RTS.
- **CombatFeedbackManager** y **CombatText**: muestran retroalimentación visual de daño y curación.

6.2. Diseño Físico de Datos

Dado que el proyecto no utiliza una base de datos relacional tradicional, el diseño físico de datos se ha adaptado al modelo de almacenamiento propio de Unity. La información del juego se representa principalmente mediante activos persistentes, prefabs y componentes serializados.

Los elementos de datos más importantes son:

- **ScriptableObjects:** definen los datos de las unidades, estructuras, objetos, recetas, investigaciones, atributos y categorías.
- **Prefabs:** representan instancias reutilizables de unidades, estructuras, proyectiles, elementos de interfaz y retroalimentación visual.
- **Componentes serializados:** almacenan parámetros de comportamiento específicos de cada entidad en la escena.
- **Jerarquía de escena:** organiza contenedores, almacenes, puntos de generación de recursos, puntos de generación de las oleadas de enemigos, paneles de la interfaz y elementos funcionales.

La ventaja de este enfoque es que permite separar claramente el contenido del comportamiento. Los datos pueden modificarse desde el editor sin tocar la lógica central, y los sistemas en escena leen esa información en tiempo de ejecución para adaptar su funcionamiento.

La principal ventaja de usar *ScriptableObjects* es centralizar los datos comunes; esto evita duplicar en cada instancia los datos comunes. Esto también se conoce como *Flyweight Pattern* [33].

En el caso de los atributos, el modelo de datos se apoya en una entidad base común y en tablas de valores base por atributo, de forma que el valor final de una unidad o estructura puede calcularse a partir de su definición y de los modificadores activos en ese momento, sin necesidad de que cada entidad almacene los valores finales.

6.3. Diseño de la Interfaz de Usuario

La interfaz de usuario se ha diseñado para ofrecer acceso rápido a las principales mecánicas del juego, manteniendo al mismo tiempo una navegación clara y coherente. Visualmente, se ha priorizado la legibilidad y la respuesta inmediata ante las acciones del jugador, eligiendo una interfaz más operativa que decorativa.

Para el conjunto de estilos, se ha elegido usar colores poco llamativos, como tonos grises y blancos, para evitar cansar la vista. La iconografía consiste en iconos simples pero reconocibles rápidamente por el jugador. La tipografía consiste en la fuente *Liberation-Sans* [1] y en colores visibles respecto al fondo. En ciertos casos, como en la interfaz de la tienda, el color varía en función del texto, lo cual favorece que el jugador sea capaz de identificar fácilmente las partes que le interesan en cada momento. Por ejemplo, si al jugador solo le interesa vender recursos la próxima vez que visite la tienda, solo tendrá que mirar los textos del color asociado a la venta.

En las siguientes Figuras (6.3, 6.4, 6.5, 6.6 y 6.7) se muestran los prototipos de cada mockup mostrado anteriormente. Cabe destacar que los botones de construcción e investigación de la interfaz principal, vistos en la Figura 6.3, usan colores más llamativos para resaltar la importancia de las mecánicas asociadas y diferenciarlos del resto de las interfaces más específicas.

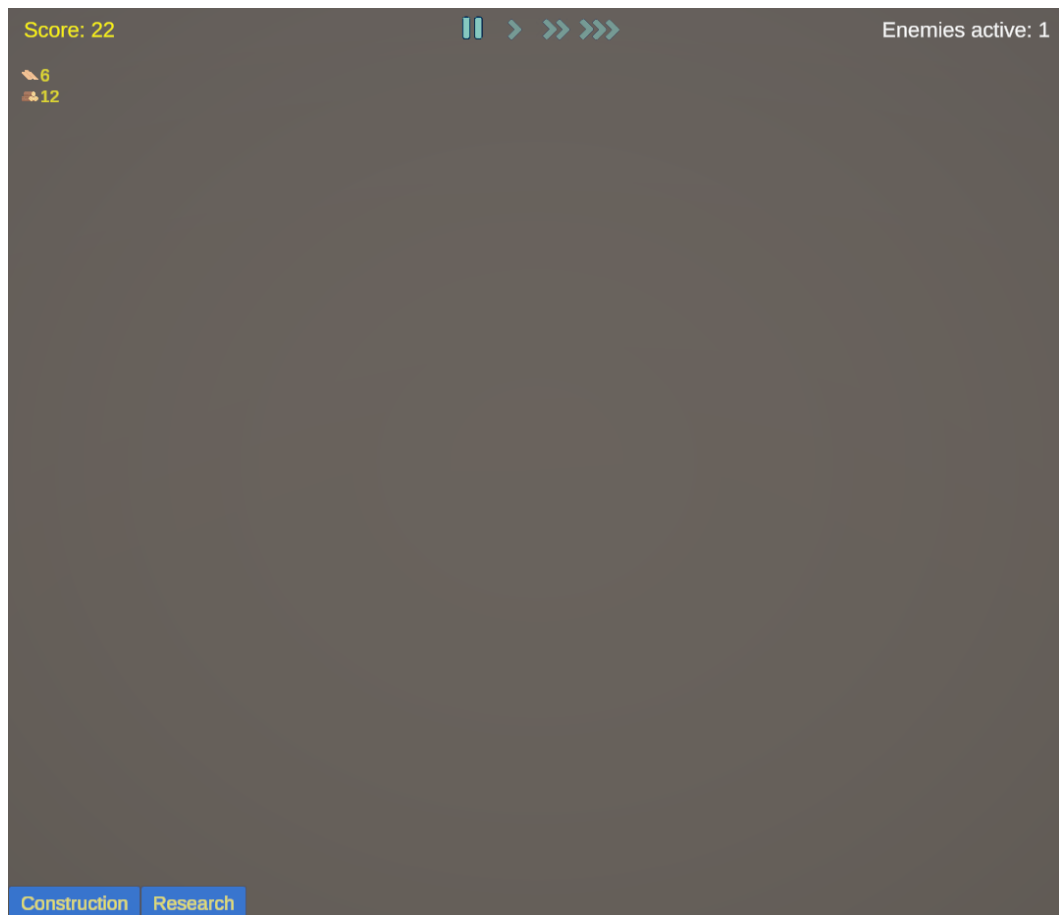


Figura 6.3: Prototipo de la interfaz principal



Figura 6.4: Prototipo de la interfaz de construcción

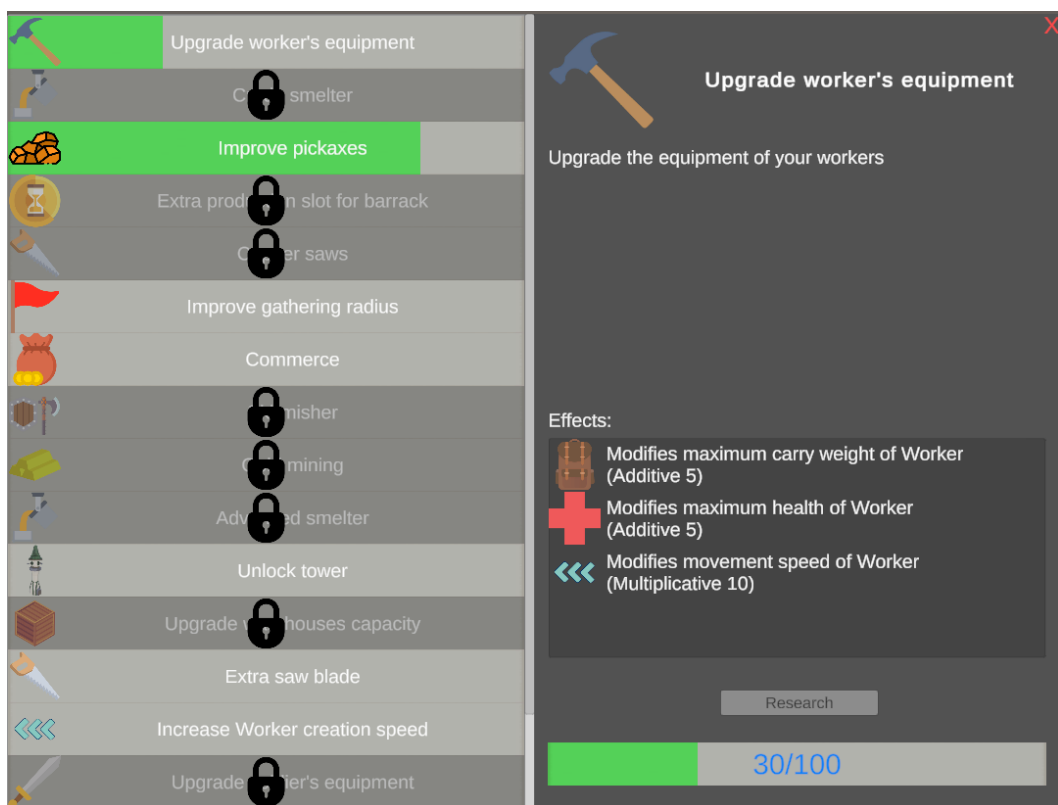


Figura 6.5: Prototipo de la interfaz de investigación

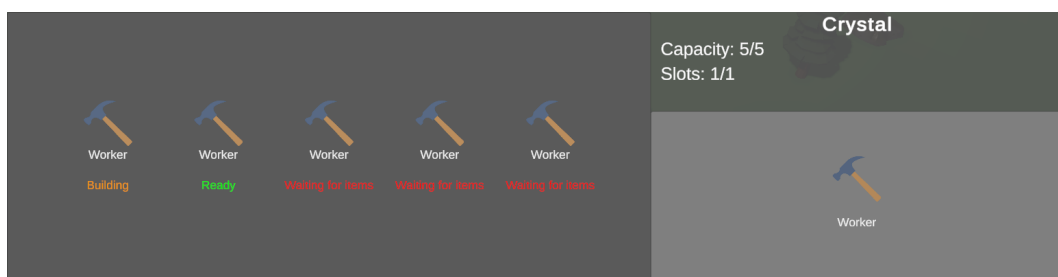


Figura 6.6: Prototipo de la interfaz de producción de unidades



Figura 6.7: Prototipo de la interfaz de la tienda

7. Construcción del Sistema

Este capítulo trata sobre todos los aspectos relacionados con la implementación del sistema en código, haciendo uso de un determinado entorno tecnológico.

7.1. Entorno de Construcción

El desarrollo del proyecto se ha realizado utilizando el motor de videojuegos Unity, empleando el lenguaje de programación C# para la implementación de la lógica de juego. Este entorno ha permitido trabajar con una arquitectura orientada a componentes y aprovechar de forma directa las herramientas nativas del motor para navegación, interfaz de usuario, animación y gestión de recursos.

Las principales herramientas utilizadas durante el desarrollo han sido las siguientes:

- **Unity 6** [25], como motor principal de desarrollo.
- **Microsoft Visual Studio** [21], como entorno de desarrollo integrado (*IDE*) para la edición y depuración del código fuente.
- **Git** [18], como sistema de control de versiones.
- **GitHub** [13], como repositorio remoto principal del proyecto.
- **Fork** [5], como cliente gráfico para la gestión de ramas, revisiones y *pull requests*.
- **Unity NavMesh** [32], para la navegación de unidades.
- **Unity Input System** [34], para la gestión de entradas de usuario.
- **TextMeshPro** [37], para la construcción de interfaces de usuario.
- **Addressables** [38], para la gestión y carga flexible de recursos.
- **ML-Agents** [35], para la experimentación con agentes inteligentes basados en aprendizaje automático.

Para la organización del trabajo, se ha seguido una estrategia basada en *Gitflow*, separando la línea estable del proyecto de la rama de desarrollo y de las ramas específicas de funcionalidad. En particular, se ha trabajado con una rama principal para las versiones estables, una rama de desarrollo para la integración progresiva de cambios y ramas de tipo *feature* para implementar cada nueva funcionalidad de forma aislada. Este enfoque ha facilitado la revisión de cambios, la reducción de conflictos y la incorporación controlada de mejoras al proyecto.

Para la construcción y despliegue, basta con indicar a Unity que inicie el proceso; la primera vez hace falta configurar el perfil de construcción, por ejemplo, para indicar el sistema operativo, en este caso, Windows 11. Las siguientes veces, basta con pulsar

Construir perfiles”. Esto crea una carpeta con el ejecutable del juego y los demás archivos necesarios.

7.2. Código Fuente

La Figura 7.1 muestra la organización general de los directorios de código del proyecto, organizados en directorios. En las siguientes subsecciones se habla de cada directorio en más detalle y, por último, se incluyen algunos extractos significativos de código.

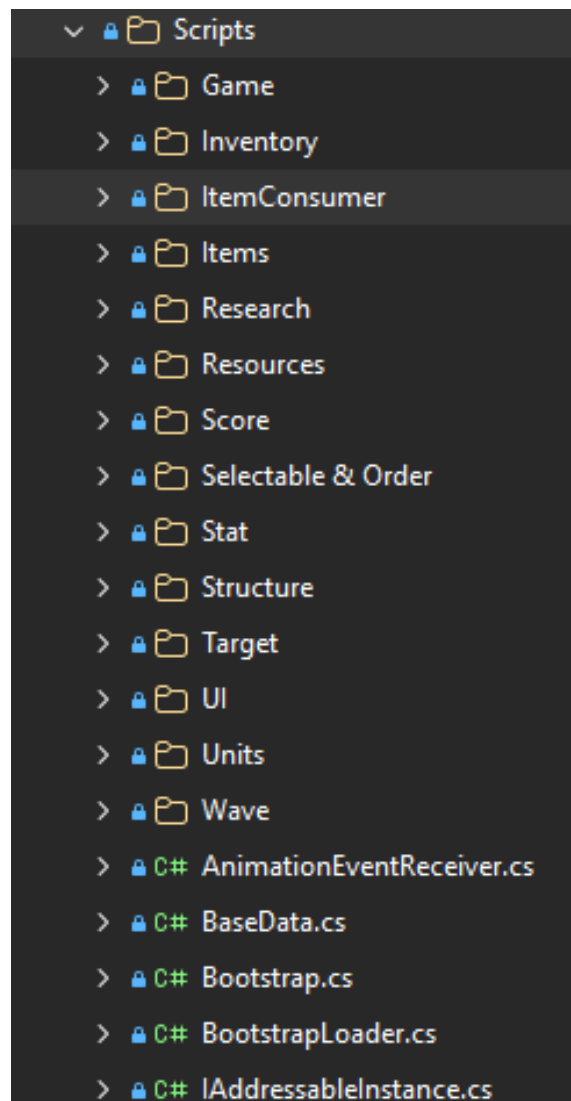


Figura 7.1: Organización general de los directorios

7.2.1. Archivos sueltos

En la Figura 7.2 se pueden observar los scripts sueltos que no pertenecen a ningún directorio.

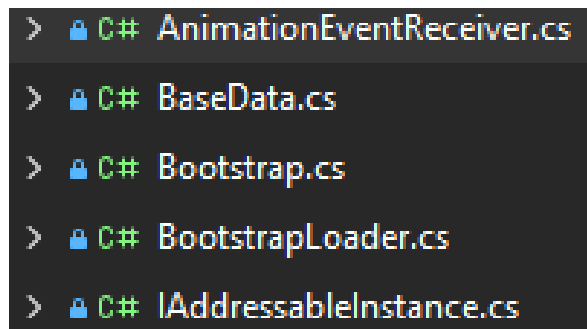


Figura 7.2: Scripts sueltos

- **BaseData:** ScriptableObject abstracto base que define los datos del juego comunes a los demás datos derivados. Agrupa propiedades comunes como el nombre, el identificador único, el icono, el requisito de investigación y la lista de atributos base.
- **AnimationEventReceiver:** componente auxiliar que recibe eventos de animación y los reenvía a la unidad situada en la jerarquía de padres más cercana. Se utiliza para conectar eventos como la finalización de una animación, un impacto de ataque o el final de una habilidad con la lógica de la unidad correspondiente.
- **Bootstrap:** clase estática que actúa como punto de entrada del juego. Antes de cargar la primera escena, comprueba si la escena de *Bootstrap* ya está cargada y, en caso contrario, la carga de forma aditiva para inicializar el conjunto de gestores globales.
- **BootstrapLoader:** componente encargado de completar la inicialización de la aplicación una vez que los gestores ya han sido preparados. Según la escena activa, decide si debe cargar el menú principal o descargar la escena de *Bootstrap*, evitando así que el jugador permanezca en la escena de arranque.
- **IAddressableInstance:** interfaz que define el contrato necesario para almacenar la referencia a una instancia cargada mediante *Addressables*. Permite que ciertos objetos puedan conservar el *handle* devuelto por la carga asíncrona y gestionarlo posteriormente.

7.2.2. Directorio Game

En la Figura 7.3 se pueden observar los scripts del directorio *Game*, que agrupa varios componentes transversales encargados del control general de la partida, la cámara, la velocidad de simulación y utilidades de depuración.

- **CameraController:** componente encargado de controlar la cámara principal durante la partida. Gestiona el desplazamiento, la rotación y otros comportamientos básicos de visión sobre el escenario.
- **DebugMenuManager:** gestor del menú de depuración. Permite modificar valores de prueba durante la ejecución, como la puntuación, los modificadores globales o los estados relacionados con las oleadas. Usado para facilitar las pruebas.

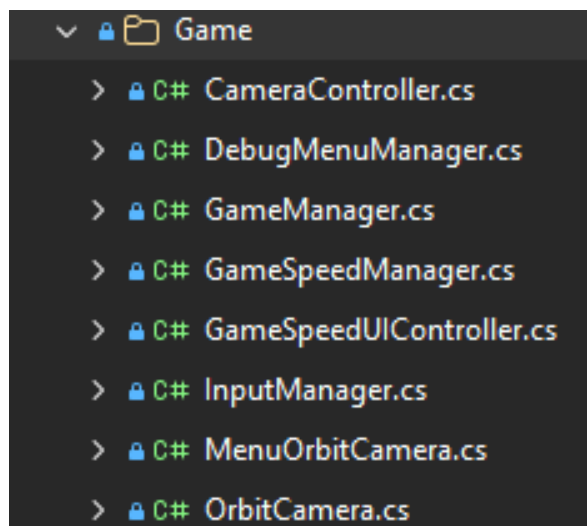


Figura 7.3: Organización del directorio Game

- **GameManager:** Coordina el estado global de la partida y actúa como punto de apoyo para funcionalidades transversales que no pertenecen a un subsistema concreto.
- **GameSpeedManager:** gestor de la velocidad de simulación. Permite pausar el juego o modificar su ritmo de ejecución, actualizando el comportamiento temporal de la partida.
- **GameSpeedUIController:** controlador de la interfaz asociada a la velocidad del juego. Conecta los elementos visuales de la UI con las acciones proporcionadas por *GameSpeedManager*.
- **InputManager:** gestor de entrada de usuario. Centraliza la lectura de acciones procedentes del jugador, usando *Input System* nativo de Unity.
- **OrbitCamera:** controlador de cámara orbital utilizado para escenas o contextos en los que se requiere una vista giratoria alrededor de un punto de interés.
- **MenuOrbitCamera:** variante del controlador orbital destinada a la cámara del menú principal, ya que el menú principal carece de ciertos elementos y es más simple.

7.2.3. Directorio Inventory

En la Figura 7.4 se pueden observar los scripts del directorio *Inventory*, que agrupa los componentes encargados de gestionar los umbrales de producción, el stock global y la interfaz asociada al inventario.

- **IThresholdProvider:** interfaz que define el comportamiento básico de cualquier sistema capaz de exponer y modificar umbrales. Permite consultar los elementos afectados y ajustar el valor límite de cada uno de ellos.
- **ThresholdRowUI:** elemento visual que representa una fila individual dentro del

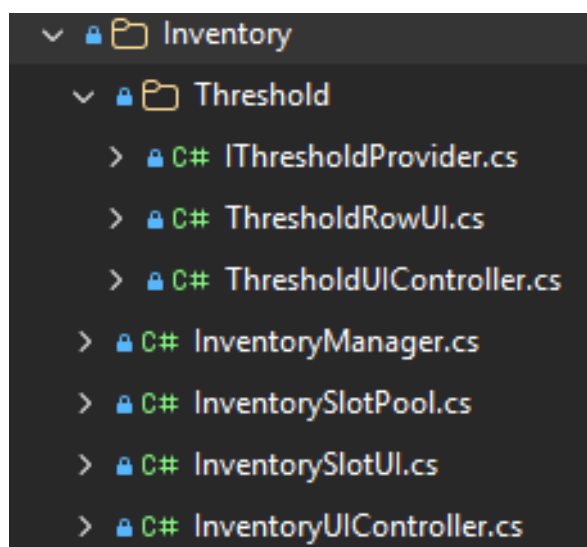


Figura 7.4: Organización del directorio Inventory

panel de umbrales. Muestra la información asociada a un recurso o producto y permite modificar su valor desde la interfaz.

- **ThresholdUIController**: controlador principal del panel de umbrales. Se encarga de construir la interfaz, cargar los datos del proveedor correspondiente y sincronizar los cambios realizados por el jugador.
- **InventoryManager**: gestor central del inventario global. Mantiene el stock acumulado de los distintos recursos del juego y actualiza sus valores cuando se almacenan o retiran objetos de los contenedores.
- **InventorySlotPool**: sistema de reutilización de elementos visuales del inventario. Permite crear y reciclar ranuras de interfaz para evitar instanciar objetos de forma innecesaria.
- **InventorySlotUI**: componente visual que representa una ranura individual del inventario. Muestra la información de un recurso concreto, como su icono y su cantidad total.
- **InventoryUIController**: controlador principal de la interfaz de inventario. Se encarga de mostrar el stock actual, organizar las ranuras visibles y actualizar la información mostrada al jugador.

7.2.4. Directorio ItemConsumer

En la Figura 7.5 se pueden observar los scripts del directorio *ItemConsumer*.

- **ItemConsumer**: interfaz que define el comportamiento básico de cualquier elemento capaz de recibir ítems. Establece las operaciones necesarias para comprobar si puede aceptar un ítem, reservarlo, liberarlo y confirmar su recepción.
- **ItemConsumerManager**: gestor encargado de registrar y coordinar todos los consumidores de ítems disponibles en la escena. Permite localizar candidatos válidos.

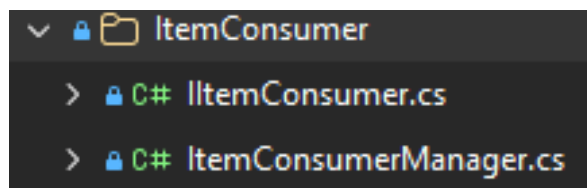


Figura 7.5: Organización del directorio ItemConsumer

dos para tareas de transporte, ordenar dichos candidatos según criterios de prioridad y facilitar la asignación de recursos a almacenes, procesadores o cualquier otra entidad consumidora.

7.2.5. Directorio Items

En la Figura 7.6 se pueden observar los scripts del directorio *Items*, que agrupa los componentes relacionados con la definición, instanciación y gestión de los ítems.

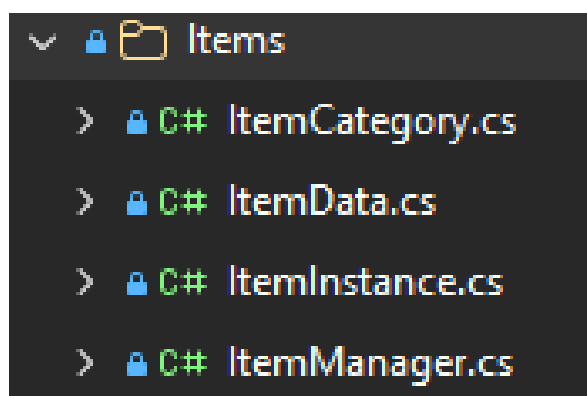


Figura 7.6: Organización del directorio Items

- **ItemCategory**: clase que representa la categoría de un ítem. Permite organizar los objetos en una jerarquía de categorías y subcategorías, lo que facilita la clasificación y el filtrado de recursos en los almacenes.
- **ItemData**: ScriptableObject derivado de BaseData que define los datos de un ítem, como su nombre, icono, prefab, categoría, peso y restricciones de compra o venta.
- **ItemInstance**: componente asociado al prefab de un ítem en escena. Almacena la referencia al dato correspondiente y, en caso necesario, información adicional sobre la entidad que lo transporta.
- **ItemManager**: gestor encargado de crear instancias de ítems en la escena a partir de sus prefabs. Se utiliza para generar recursos y objetos producidos que deben aparecer en la escena durante la partida.

7.2.6. Directorio Research

En la Figura 7.7 se pueden observar los scripts del directorio *Research*, que agrupa los componentes relacionados con la investigación, la aplicación de efectos al completar proyectos de investigación y la interfaz asociada.

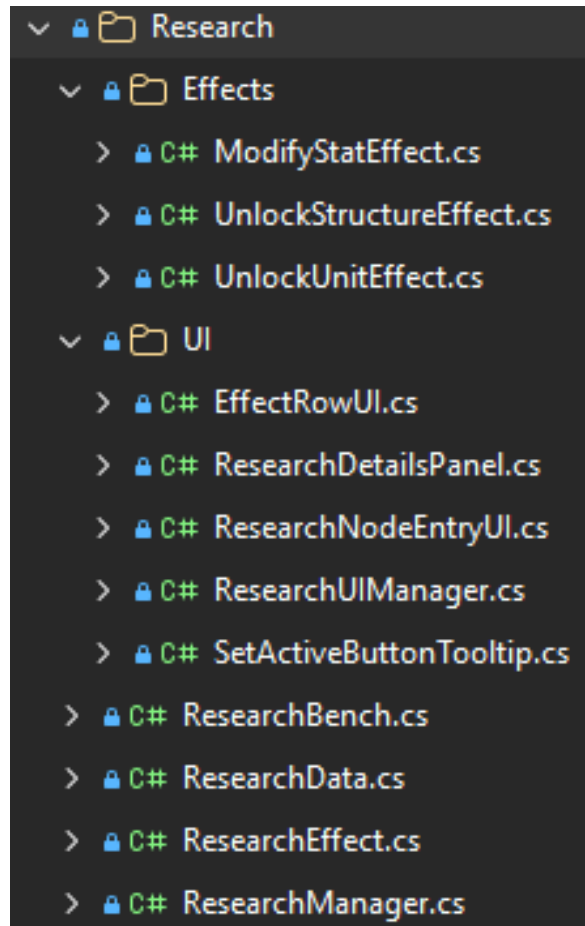


Figura 7.7: Organización del directorio Research

- **ResearchData:** ScriptableObject derivado de BaseData que define los datos de una investigación concreta. Incluye información como el nombre, la descripción, los prerequisites y la lista de efectos.
- **ResearchManager:** gestor central del sistema de investigación. Controla el progreso de las investigaciones activas, aplica sus efectos y actualiza el estado de desbloqueo de contenido.
- **ResearchBench:** estructura en la que los trabajadores ejecutan tareas de investigación. Registra la tarea asociada y coordina la producción de progreso.
- **ResearchEffect:** clase base que representa un efecto aplicado al completar una investigación. Sirve como fundamento para los efectos concretos que modifican el comportamiento del sistema.
- **ModifyStatEffect:** efecto de investigación que modifica uno o varios atributos del sistema al completarse la investigación correspondiente.

- **UnlockStructureEffect**: efecto de investigación que desbloquea nuevas estructuras para el jugador.
- **UnlockUnitEffect**: efecto de investigación que desbloquea nuevas unidades para el jugador.
- **ResearchUIManager**: controlador principal de la interfaz de investigación. Se encarga de construir la lista de investigaciones, mostrar sus detalles y actualizar su estado según el progreso del sistema.
- **ResearchNodeEntryUI**: elemento visual que representa una investigación en la interfaz principal, mostrando su estado, nombre y disponibilidad.
- **ResearchDetailsPanel**: panel de interfaz encargado de mostrar la información detallada de la investigación seleccionada, incluyendo su icono, descripción, efectos y requisitos.
- **EffectRowUI**: componente visual que representa una fila individual dentro del panel de detalles de investigación. Muestra de forma resumida el efecto aplicado por una investigación.
- **SetActiveButtonTooltip**: componente auxiliar que muestra un mensaje explicativo cuando el botón de activación de una investigación no puede utilizarse, normalmente por faltar prerequisites.

7.2.7. Directorio Resources

En la Figura 7.8 se pueden observar los scripts del directorio *Resources*, que agrupa los componentes relacionados con la gestión de recursos, su recolección y procesamiento.

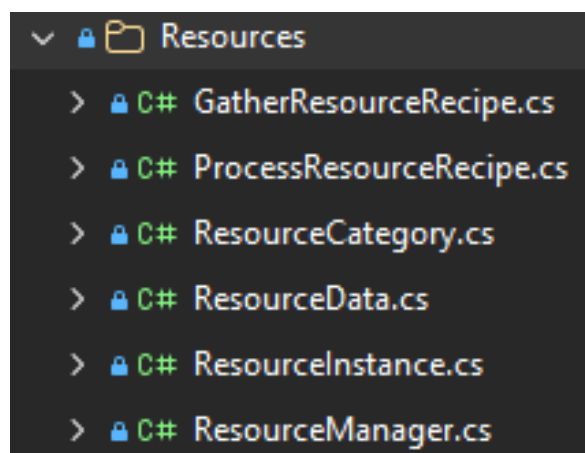


Figura 7.8: Organización del directorio Resources

- **ResourceData**: ScriptableObject derivado de BaseData que define los datos de un nodo de recursos (como un árbol o mena de hierro). Incluye información como su nombre, icono, prefab, categoría y propiedades asociadas.
- **ResourceCategory**: Enumerado que representa la categoría de un nodo de recursos.

- **ResourceManager**: gestor principal de nodos de recursos. Se encarga de coordinar la creación, registro y seguimiento de los nodos de recursos presentes en el mundo.
- **ResourceInstance**: componente asociado a una instancia concreta de recurso en escena. Mantiene la referencia al dato correspondiente y a la entidad física que representa el recurso.
- **GatherResourceRecipe**: clase que define la configuración necesaria para recolectar un recurso, incluyendo el tiempo necesario para recolectarlo, la cantidad (mínimo/máximo) de ítems a crear al completar la recolección y la cantidad máxima de ítems que puede contener un recurso. Usado por banderas de recolección.
- **ProcessResourceRecipe**: clase que describe una receta de procesado. Establece los recursos de entrada, los recursos de salida, combustible (opcional), las cantidades necesarias y el tiempo necesario para completar el procesamiento. Usado por estructuras de procesamiento como el aserradero.

7.2.8. Directorio Score

En la Figura 7.9 se pueden observar los scripts del directorio *Score*.

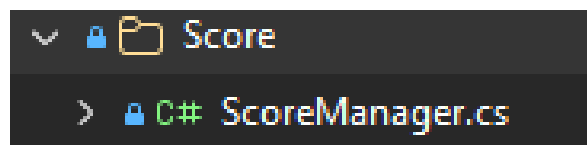


Figura 7.9: Organización del directorio Score

- **ScoreManager**: gestor encargado de almacenar, modificar y consultar la puntuación del jugador.

7.2.9. Directorio Selectable & Order

En la Figura 7.10 se pueden observar los scripts del directorio *Selectable & Order*, que agrupa los componentes relacionados con la selección de entidades, la emisión de órdenes y la coordinación de movimientos de las unidades controladas por el jugador.

- **ISelectable**: interfaz que define el comportamiento básico de cualquier entidad seleccionable. Establece las operaciones necesarias para mostrar o ocultar su estado de selección.
- **Selectable**: componente asociado a una entidad seleccionable. Permite integrar dicha entidad en el sistema de selección y gestionar su estado visual o funcional cuando pasa a estar seleccionada o deseleccionada.
- **SelectionManager**: gestor principal de la selección. Se encarga de registrar las entidades seleccionadas en cada momento y de coordinar el comportamiento general del sistema de selección múltiple.

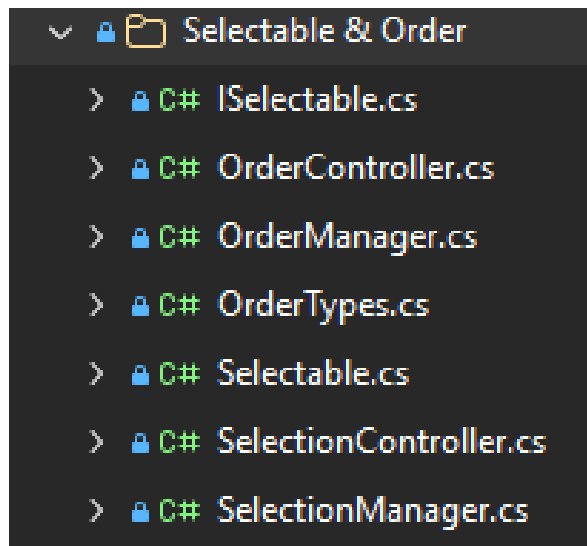


Figura 7.10: Organización del directorio Selectable & Order

- **SelectionController**: controlador de la interacción del jugador con el sistema de selección. Procesa las entradas del usuario y traduce dichas acciones en cambios sobre la selección activa.
- **OrderTypes**: enumeración que define los tipos de órdenes disponibles en el sistema, como movimiento y vigilar áreas.
- **OrderManager**: gestor encargado de coordinar la emisión y ejecución de órdenes sobre las unidades seleccionadas. Actúa como punto central de comunicación entre la interfaz, la selección y la lógica de comportamiento de las unidades.
- **OrderController**: componente que interpreta las acciones del jugador para generar órdenes concretas y enviarlas al gestor correspondiente. Se utiliza como intermediario entre la entrada del usuario y la ejecución de comandos en el juego.

7.2.10. Directorio Stat

En la Figura 7.11 se pueden observar los scripts del directorio *Stat*, que agrupa los componentes relacionados con la definición de atributos, el cálculo de modificadores y la actualización de valores en las entidades del juego.

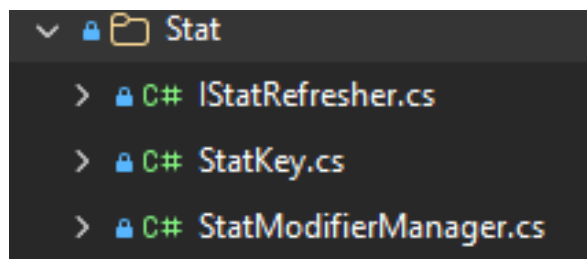


Figura 7.11: Organización del directorio Stat

- **StatKey**: ScriptableObject que define los datos de un atributo concreto. Contiene el nombre, tipo y valor por defecto. Se utiliza como referencia común para consultar, modificar y aplicar valores sobre atributos como salud, velocidad o daño.
- **StatModifierManager**: gestor central de modificadores de atributos. Se encarga de almacenar y aplicar los cambios procedentes de investigaciones que alteran el valor final de un atributo.
- **IStatRefresher**: interfaz que define el comportamiento de las entidades capaces de actualizar sus atributos cuando se producen cambios en los modificadores activos.

7.2.11. Directorio Structure

En la Figura 7.12 se pueden observar los scripts del directorio *Structure*, en concreto, los scripts sueltos que no pertenecen a ningún subdirectorio.

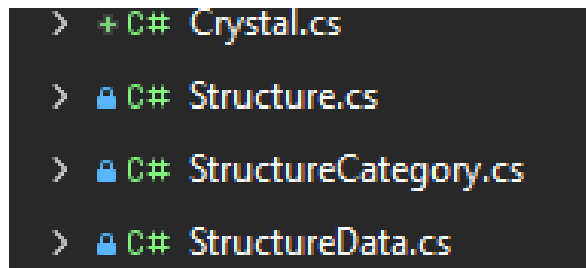


Figura 7.12: Scripts sueltos del directorio Structure

- **StructureData**: ScriptableObject base derivado de BaseData que define los datos para las estructuras. Incluye información como el tiempo necesario para construirla, requisitos de recursos, si permite rotación y si es una estructura única.
- **StructureCategory**: Enumerado utilizado para agrupar estructuras en la interfaz de construcción.
- **Structure**: componente principal de toda estructura. Gestiona información común a todas las estructuras, como la salud, destrucción y curación por oleada.
- **Crystal**: componente del cristal, estructura que el jugador ha de proteger. Al ser destruido notifica al sistema correspondiente.

Subdirectorio Construction

En la Figura 7.13 se pueden observar los scripts del subdirectorio *Construction*, que agrupa los componentes relacionados con la colocación de estructuras, la gestión de planos y la interfaz de usuario asociada.

- **PlacementController**: componente encargado de gestionar la previsualización y colocación de estructuras en el terreno. Valida la posición seleccionada, controla la confirmación de construcción y crea el plano correspondiente.

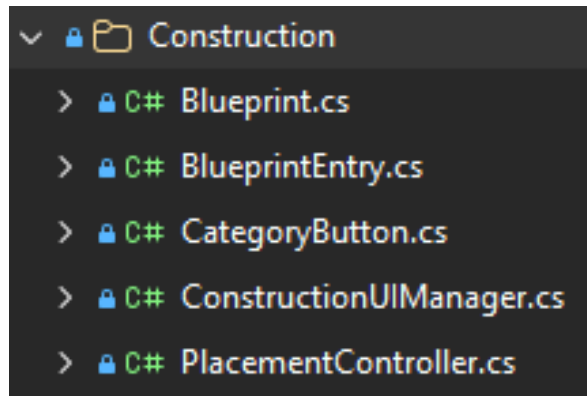


Figura 7.13: Organización del subdirectorio Construction

- **Blueprint**: representa una estructura en fase de construcción. Mantiene la información necesaria para completar la edificación y coordina la tarea asociada hasta su finalización.
- **ConstructionUIManager**: controlador principal de la interfaz de construcción. Se encarga de mostrar las estructuras disponibles, organizar los elementos visuales y gestionar la interacción del jugador con este sistema.
- **CategoryButton**: componente de interfaz que representa un botón de categoría dentro del panel de construcción. Permite agrupar y filtrar estructuras por tipo.
- **BlueprintEntry**: elemento visual que representa una estructura concreta en la interfaz de construcción. Muestra su información básica y permite iniciar el proceso de colocación.

Subdirectorio Defenses

En la Figura 7.14 se pueden observar los scripts del subdirectorio *Defenses*.

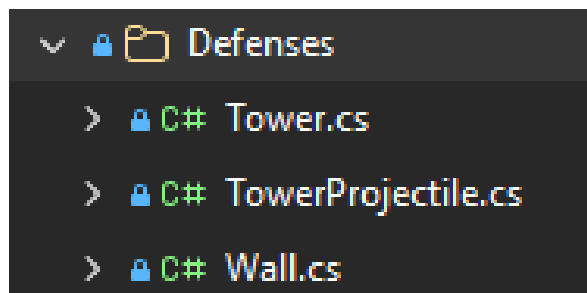


Figura 7.14: Organización del subdirectorio Defenses

- **Tower**: estructura defensiva que utiliza el sistema de detección de objetivos para localizar enemigos dentro de su alcance y ejecutar ataques de forma automática.
- **TowerProjectile**: componente asociado a los proyectiles disparados por las torres. Gestiona el desplazamiento, la detección de impactos y la aplicación de daño sobre los objetivos alcanzados.

- **Wall:** estructura defensiva pasiva cuya función principal es bloquear el avance de los enemigos y proteger otras estructuras.

Subdirectorio Resource

En la Figura 7.15 se pueden observar los scripts del subdirectorio *Resource*, que agrupa las estructuras de recolección y procesamiento junto con sus datos.

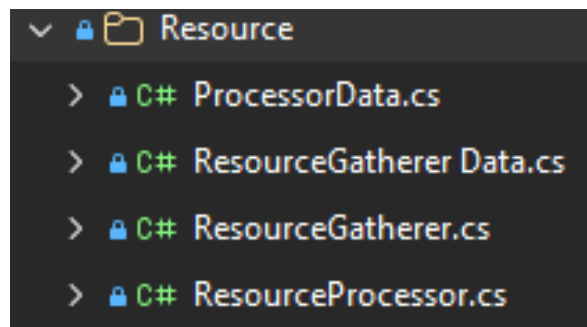


Figura 7.15: Organización del subdirectorio Resource

- **ProcessorData:** ScriptableObject derivado de StructureData que define los datos de una estructura de procesamiento de recursos. Almacena las recetas de procesamiento disponibles, así como parámetros relacionados con el uso de combustible, incluyendo si la estructura requiere combustible para funcionar y su capacidad máxima de almacenamiento.
- **ResourceProcessor:** Componente encargado de gestionar el funcionamiento de las estructuras de procesamiento. Controla la recepción y reserva de materiales de entrada, la ejecución simultánea de lotes de producción, el consumo de combustible cuando es necesario, la generación de recursos de salida y la aplicación de umbrales para evitar producir un producto más de lo necesario.
- **ResourceGathererData:** ScriptableObject que configura las estructuras de recolección de recursos. Define las categorías de recursos que la estructura puede detectar y explotar, permitiendo especializar distintos recolectores para diferentes tipos de materias primas.
- **ResourceGatherer:** Componente responsable de localizar automáticamente nodos de recursos dentro de un radio determinado y generar las tareas necesarias para su recolección. Mantiene un registro de los nodos detectados, actualiza dinámicamente su área de influencia en función de los atributos activos y coordina la creación de tareas de recolección, respetando los umbrales de almacenamiento definidos por el jugador, evitando recolectar un nodo de recurso más de lo necesario.

Subdirectorio Shop

En la Figura 7.16 se pueden observar los scripts del subdirectorio *Shop*.

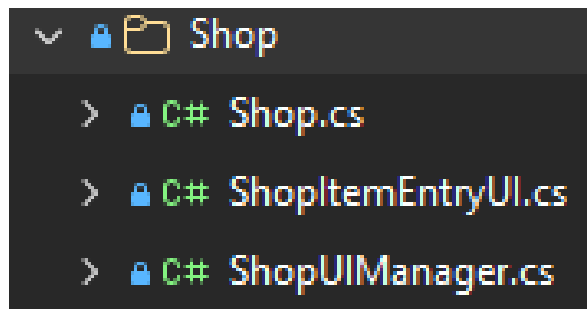


Figura 7.16: Organización del subdirectorio Shop

- **Shop**: componente asociado a la estructura física de la tienda, encargado de actualizar los atributos cuando se aplican cambios, además de almacenar las referencias a los datos correspondientes.
- **ShopItemEntryUI**: componente visual que representa una entrada individual dentro del panel de la tienda. Muestra información de un ítem, como el icono, nombre, coste de compra y venta, además de botones para aumentar/decrementar la cantidad y un campo para introducir la cantidad por texto.
- **ShopUIManager**: controlador principal de la interfaz de la tienda. Se encarga de construir la lista de ítems, mostrar sus detalles y actualizar el balance total.

Subdirectorio UnitFactory

En la Figura 7.17 se pueden observar los scripts del subdirectorio *UnitFactory*.

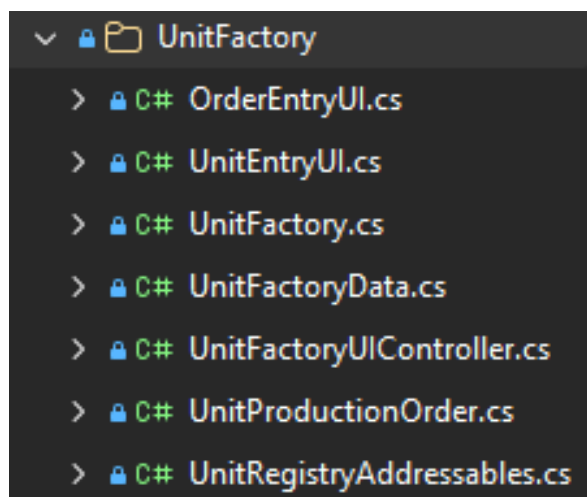


Figura 7.17: Organización del subdirectorio UnitFactory

- **UnitFactoryData**: ScriptableObject derivado de StructureData que define los datos para las fábricas de unidades. Incluye información como la lista de unidades que se pueden producir en esa fábrica, en caso de querer limitarla, y el tamaño máximo de la cola de producción.

- **UnitFactory**: componente asociado a la estructura de producción de unidades. Responsable de actualizar los atributos, almacenar las referencias a los datos correspondientes y gestionar las órdenes de producción.
- **UnitEntryUI**: componente visual que representa una unidad disponible a producir en la fábrica de unidades. Muestra el icono, nombre y coste (mediante *tooltip*) de la unidad, si los prerequisites de investigación no están completados, muestra un candado y no permite crear órdenes de producción para esa unidad.
- **OrderEntryUI**: componente visual que representa la orden de producción en progreso de una unidad en la fábrica de unidades. Muestra el icono, el nombre y el estado actual (listo para producir, esperando hueco en cola, en producción).
- **UnitProductionOrder**: representa la lógica de una orden de producción, al ser un *ItemConsumer*, solicita automáticamente ítems para poder producir la unidad. Se encarga de gestionar el estado de la orden de producción.
- **UnitFactoryUIController**: controlador principal de la interfaz de producción de unidades, se encarga de mostrar el panel asociado a la estructura de producción de unidades, creando la lista de unidades disponibles, órdenes en producción e información de la estructura, como el tamaño de la cola de producción y el número de órdenes que se pueden procesar a la vez.
- **UnitRegistryAddressables**: componente encargado de cargar todos los posibles *UnitData* mediante *Addressables*. Es necesario para crear la lista de unidades disponibles automáticamente.

Subdirectorio Warehouses

En la Figura 7.18 se pueden observar los scripts del subdirectorio *Warehouses*.

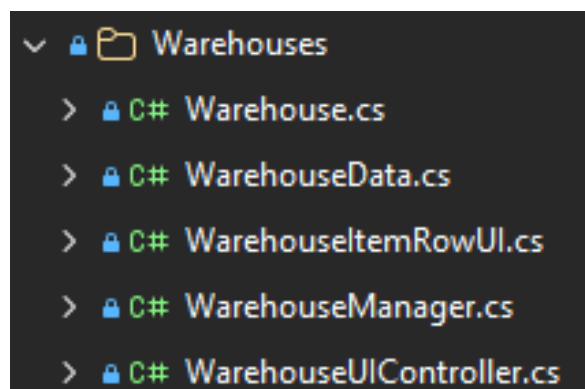


Figura 7.18: Organización del subdirectorio Warehouses

- **WarehouseData**: *ScriptableObject* derivado de *StructureData* que define los datos para los almacenes. Incluye información como la lista de categorías aceptadas.
- **Warehouse**: componente asociado al almacén correspondiente. Se encarga de actualizar los atributos, gestionar la lógica de reserva y cancelación tanto para

entregas como para recibir ítems y almacenar ítems de las categorías aceptadas.

- **WarehouseManager**: gestor principal de los almacenes. Se encarga de coordinar la creación, registro y seguimiento de todos los almacenes presentes en el mundo. Permite buscar ítems específicos en todos los almacenes, además de devolver la lista de almacenes registrados.
- **WarehouseUIController**: controlador principal de la interfaz de almacenes, se encarga de mostrar el panel asociado al almacén cuando es seleccionado. Muestra el nombre del almacén, el espacio ocupado, el espacio máximo posible, además de crear la lista de recursos almacenados.
- **WarehouseItemRowUI**: componente visual que representa un recurso almacenado en el almacén seleccionado. Muestra el icono, nombre y cantidad almacenada.

7.2.12. Directorio Target

En la Figura 7.19 se pueden observar los scripts del directorio *Target*.

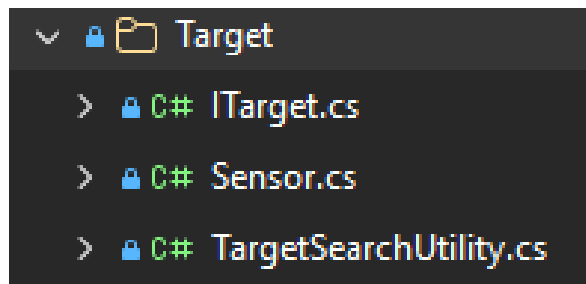


Figura 7.19: Organización del directorio Target

- **ITarget**: interfaz que define el comportamiento básico de cualquier elemento capaz de ser un objetivo. Establece las operaciones necesarias para comprobar la posición, facción, categoría (unidad o estructura), salud actual, salud máxima, curar al objetivo y dañar al objetivo.
- **Sensor**: componente asociado a un elemento presente en el mundo capaz de detectar a objetivos. Tiene un modo para detectar enemigos y otro para detectar aliados. Para evitar comprobaciones constantes, detecta objetivos cuando entran en un área y deja de detectarlos cuando salen, almacenando las referencias mientras están dentro del área.
- **TargetSearchUtility**: clase estática de utilidad con métodos usados para devolver el objetivo más cercano al solicitante o para ordenar listas de objetivos según distancia.

7.2.13. Directorio UI

En la Figura 7.20 se pueden observar los scripts del directorio *UI*.

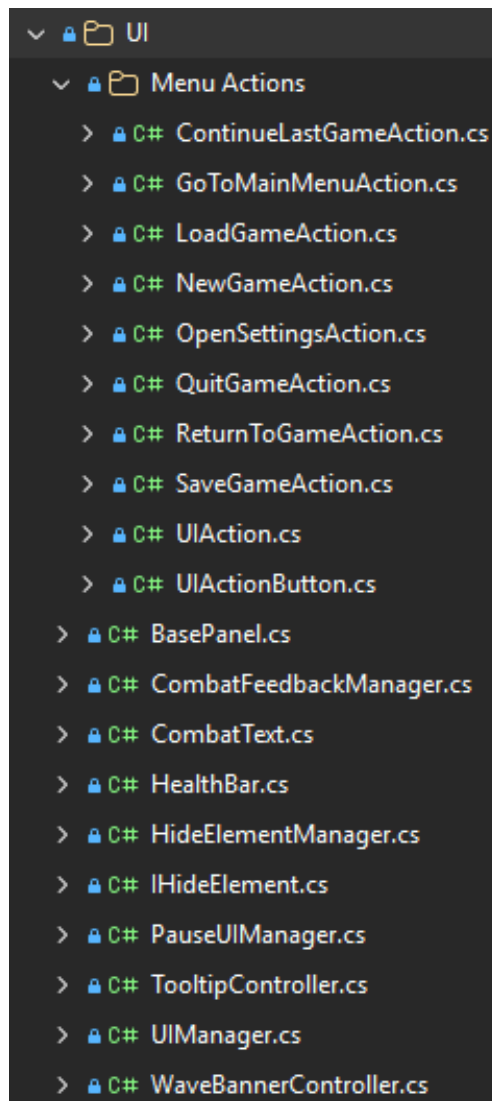


Figura 7.20: Organización del directorio UI

- **UIAction:** ScriptableObject abstracto que representa una acción de UI. Las subclases concretas implementan `Execute()` (Nueva partida, Guardar, Salir, etc...). Esto evita duplicar código si varios menús tienen los mismos botones, por ejemplo el menú principal y el menú de pausa ambos tienen botones para salir del juego.
- **UIActionButton:** Componente que enlaza un botón de la interfaz con un `UIAction`. Al pulsarse el botón, se ejecuta la acción asignada.
- **ContinueLastGameAction, GoToMainMenuAction, LoadGameAction, OpenSettingsAction, QuitGameAction, ReturnToGameAction, SaveGameAction:** ScriptableObjects derivados de `UIAction`. Se encargan de ejecutar la acción asociada al nombre cuando se pulsa el `UIActionButton` correspondiente.
- **BasePanel:** Clase base abstracta para todos los paneles de la interfaz. Gestiona el registro en `HideElementManager`, el modo de input y los métodos de cierre del panel.
- **CombatFeedbackManager:** Gestor principal encargado de la retroalimenta-

ción visual de combate. Registra fuentes de daño y curación, y muestra textos flotantes mediante un *pool* de `CombatText`.

- **CombatText**: Componente visual de un texto flotante de combate. Anima el texto con movimiento ascendente hacia los lados (con variación aleatoria) y *fade out*, devolviéndose al *pool* al terminar para evitar el coste de rendimiento de instanciar/destruir.
- **HealthBar**: Componente visual asociado a un `ITarget` en el mundo del juego. Consiste en una barra que actualiza su relleno y color según la facción cuando se reciben eventos de daño y curación.
- **IHideElement**: Interfaz con un único método `HidePanel()`, implementada por todos los paneles que pueden ser ocultados por `HideElementManager`.
- **HideElementManager**: Gestor principal encargado de mantener un registro de todos los `IHideElement` activos. Permite mostrar solo un panel a la vez, ocultando el resto u ocultar todos los paneles.
- **PauseUIManager**: Controlador principal del menú de pausa (distinto de cambiar la velocidad de simulación a 0). Pausa y reanuda el juego al mostrarse u ocultarse, al mostrarse, se muestran `UIButton` como reanudar la partida o salir del juego. Mientras este menú está abierto, la cámara orbita alrededor lentamente, al cerrarse, la cámara vuelve al estado anterior a orbitar.
- **TooltipController**: Controlador principal encargado de mostrar un *tooltip* de texto junto al cursor, ajustando su tamaño al contenido y manteniéndolo dentro de los límites de la pantalla. El texto mostrado depende del contenido que lo haya llamado; por ejemplo, en la interfaz de construcción, al mover el cursor sobre una estructura, se muestra el coste de construcción.
- **UIManager**: Gestor principal monolítico, usado en las primeras fases de desarrollo, conforme avanzaba el desarrollo, se ha ido refactorizando y metiendo partes del monolito en módulos más escalables e independientes.
- **WaveBannerController**: Componente visual que muestra un texto con animación de *fade in/out* para notificar eventos de oleada al jugador. Usado para avisar al jugador cuando una oleada está cerca de ocurrir.

7.2.14. Directorio Units

En la Figura 7.21 se pueden observar los scripts del directorio *Units*, en concreto, los scripts sueltos que no pertenecen a ningún subdirectorio.

- **UnitData**: `ScriptableObject` base derivado de `BaseData` que define los datos para las unidades. Incluye información como tiempo de producción, coste en puntuación, lista de costes en ítems y facción.
- **Unit**: componente principal de toda unidad. Gestiona información común a todas las unidades, como la salud, velocidad de movimiento, animaciones, destrucción y curación por oleadas.

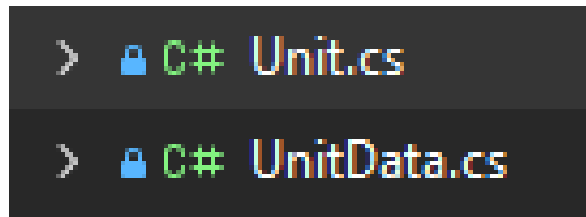


Figura 7.21: Scripts sueltos del directorio Unit

Subdirectorio Agent

En la Figura 7.22 se pueden observar los scripts del subdirectorio *Agent*, que agrupa los componentes relacionados con las unidades controladas mediante *ML-Agents*, su entrenamiento y las habilidades especiales disponibles para este tipo de soldados.

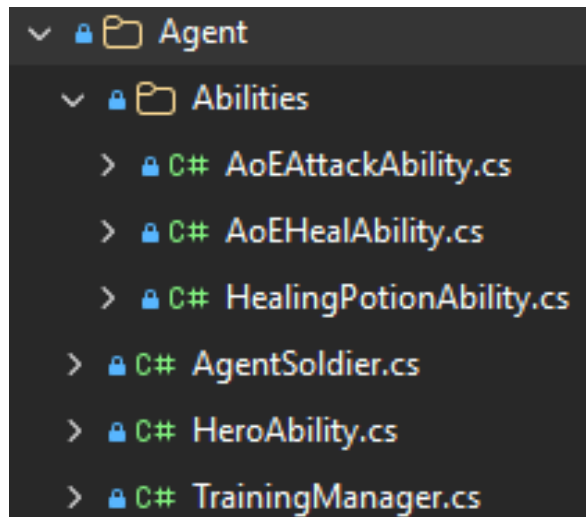


Figura 7.22: Organización del subdirectorio Agent

- **AgentSoldier**: agente de *Unity ML-Agents* que controla el comportamiento de un soldado especial. Define las observaciones que se proporcionan a la red neuronal, interpreta las acciones discretas generadas por el modelo y coordina el movimiento, el combate, el uso de habilidades y el comportamiento defensivo de la unidad. Además, sincroniza sus atributos con el sistema de atributos y utiliza sensores para localizar enemigos y aliados cercanos.
- **TrainingManager**: gestor encargado de coordinar las sesiones de entrenamiento de los agentes. Se ocupa de reiniciar episodios, posicionar agentes y enemigos, repartir recompensas compartidas y terminar la partida de entrenamiento cuando se cumplen las condiciones definidas.
- **HeroAbility**: *ScriptableObject* base abstracto que define la estructura común de las habilidades especiales disponibles para los agentes. Establece la información general de cada habilidad, como su nombre, icono y tiempo de reutilización, y obliga a implementar la recogida de observaciones, la ejecución de la habilidad, la comprobación de uso y el cálculo de recompensa asociada.

- **AoEAttackAbility**: ScriptableObject derivado de HeroAbility. Habilidad ofensiva de área que permite al agente dañar simultáneamente a varios objetivos cercanos. Calcula el centro de impacto, detecta enemigos en un radio determinado y aplica daño a todas las entidades válidas que cumplan las condiciones de facción y estado de vida.
- **AoEHealAbility**: ScriptableObject derivado de HeroAbility. Habilidad de curación de área que restaura salud tanto al propio agente como a las unidades aliadas cercanas. Evalúa qué objetivos pueden beneficiarse de la curación, aplica la cantidad correspondiente y mide la curación efectiva y la parte desperdiciada para ajustar la recompensa.
- **HealingPotionAbility**: ScriptableObject derivado de HeroAbility. Habilidad de curación individual. Permite al agente recuperar su salud y calcula la recompensa asociada según la cantidad de vida realmente restaurada, penalizando desperdicios.

Subdirectorio Enemy

En la Figura 7.23 se pueden observar los scripts del subdirectorio *Enemy*, que agrupa la lógica de comportamiento de las unidades enemigas y su máquina de estados de combate usando *State Pattern* [36].

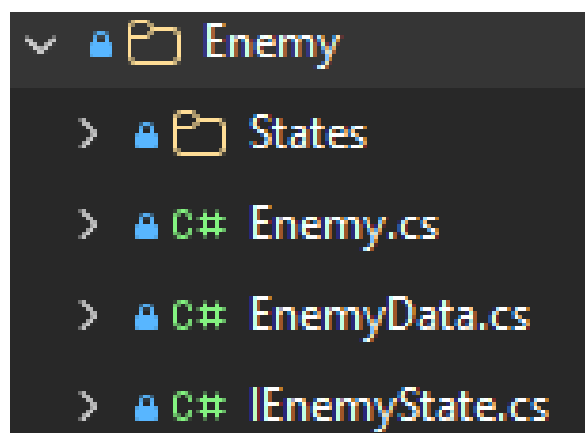


Figura 7.23: Organización del subdirectorio Enemy

- **EnemyData**: ScriptableObject base derivado de UnitData que define los datos de un enemigo concreto. Incluye información sobre el coste de creación para WaveManager, el radio de vigilancia y los retrasos entre patrullas, además de atributos como salud, daño, velocidad de movimiento, etc.
- **Enemy**: componente principal que representa a un enemigo instanciado en la escena. Gestiona su vida, facción, referencia a los sistemas de combate y coordinación con la máquina de estados que determina su comportamiento.
- **IEnemyState**: interfaz que define el comportamiento común de los distintos estados del enemigo. Establece las operaciones básicas que debe implementar cada estado, como la entrada al estado, su actualización y su salida.

- **EnemyIdleState**: estado de reposo. Se utiliza cuando la unidad no ha detectado ningún objetivo válido y permanece a la espera de cambios en su entorno.
- **EnemyGuardState**: estado de vigilancia. Permite que el enemigo patrulle un área mientras espera a que aparezcan objetivos cercanos.
- **EnemyChaseState**: estado de persecución. Se activa cuando se detecta un objetivo válido y el enemigo debe desplazarse hacia él para entrar en rango de ataque.
- **EnemyAttackState**: estado de ataque. Se encarga de ejecutar el daño sobre el objetivo detectado, respetando el alcance, el tiempo de reutilización y las condiciones de validez del combate.

Subdirectorio Soldier

En la Figura 7.24 se pueden observar los scripts del subdirectorio *Soldier*, que agrupa la lógica de comportamiento de los soldados del jugador y su máquina de estados de combate usando *State Pattern* [36].

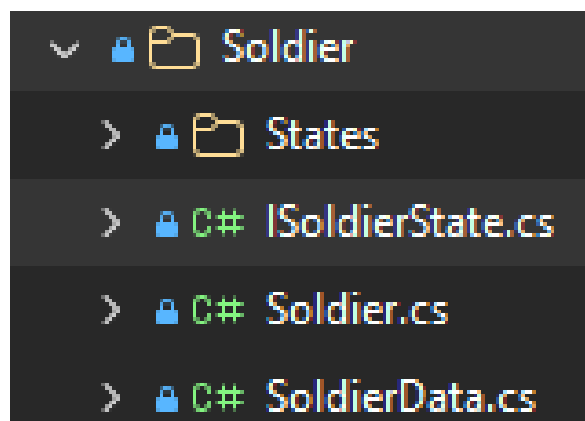


Figura 7.24: Organización del subdirectorio Soldier

- **SoldierData**: ScriptableObject derivado de *UnitData* que define los datos de un soldado concreto. Incluye información de los atributos, como la salud, daño, velocidad de movimiento, etc.
- **Soldier**: componente principal que representa a un soldado instanciado en escena. Gestiona sus atributos, su sensor de detección, la integración con el sistema de órdenes y el cambio entre estados de movimiento, persecución, ataque e inactividad.
- **ISoldierState**: interfaz que define el comportamiento común de los distintos estados del soldado. Establece las operaciones básicas de entrada, actualización y salida de cada estado.
- **SoldierIdleState**: estado de reposo. Mantiene la unidad parada mientras no exista un objetivo válido y realiza la transición a persecución cuando detecta un enemigo cercano.

- **SoldierMoveState**: estado de desplazamiento. Se utiliza cuando la unidad recibe una orden de movimiento o de ataque. Controla la llegada al destino, movimiento en formación si se han seleccionado varias unidades y la asignación de un punto de guardia.
- **SoldierChaseState**: estado de persecución. Ajusta el destino de la unidad hacia la posición de ataque del objetivo y cambia al estado de ataque cuando la distancia es suficiente para atacarlo.
- **SoldierAttackState**: estado de combate. Mantiene la unidad en rango, orienta al personaje hacia el objetivo y ejecuta ataques al objetivo.

Subdirectorio Worker

En la Figura 7.25 se pueden observar los scripts del subdirectorio *Worker*, que agrupa la lógica de comportamiento de los trabajadores del jugador y su máquina de estados de tareas usando *State Pattern* [36].

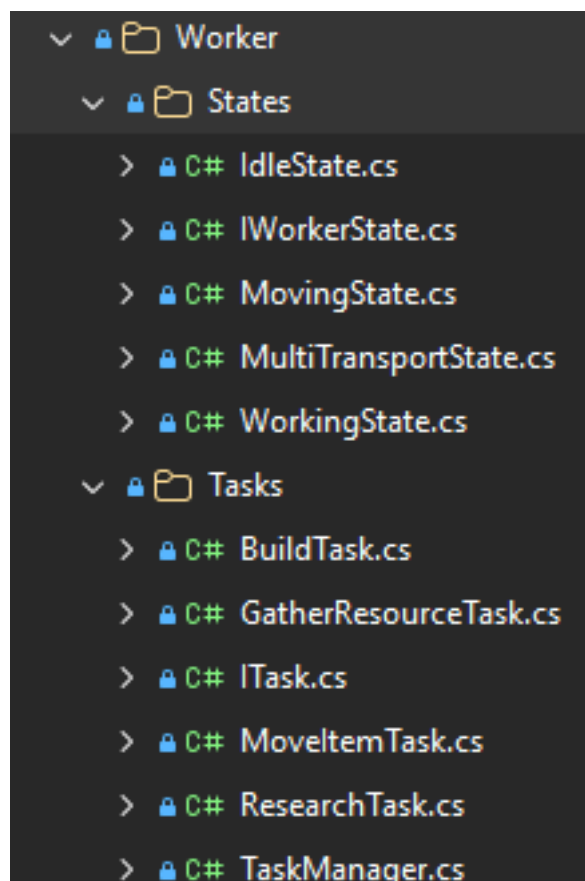


Figura 7.25: Organización del subdirectorio Worker

- **WorkerData**: ScriptableObject derivado de UnitData que define los datos de un trabajador concreto. Incluye información de los atributos, como la salud, la capacidad del inventario, la velocidad de movimiento, etc.

- **Worker:** componente principal que representa a un trabajador instanciado en la escena. Gestiona sus atributos, su inventario, el cambio entre estados de comportamiento y la ejecución de tareas.
- **IWorkerState:** interfaz que define el comportamiento común de los distintos estados del trabajador. Establece las operaciones básicas de entrada, actualización y salida que deben implementar todos los estados.
- **IdleState:** estado de inactividad. Mientras no tenga una tarea asignada, consulta a *TaskManager* para localizar el trabajo más adecuado y, cuando encuentra uno, pasa al estado de desplazamiento.
- **MovingState:** estado de desplazamiento. Se utiliza tanto para moverse hacia una tarea asignada como para seguir una orden de movimiento directa del jugador. Controla la llegada al destino y la transición posterior al estado de trabajo o al estado de reposo.
- **MultiTransportState:** estado especializado para el transporte de varios objetos asociados a una tarea de traslado. Coordina las fases de recogida y entrega, permitiendo recoger varios ítems cercanos con un mismo recorrido cuando comparten destino.
- **WorkingState:** estado de ejecución de la tarea asignada. Inicia el trabajo correspondiente y, en función del tipo de tarea, puede delegar el comportamiento en un estado específico como el transporte múltiple.
- **TaskManager:** gestor central de tareas disponibles en la escena. Registra, elimina y selecciona tareas en función de su prioridad y de la distancia al trabajador que las solicita.
- **ITask:** interfaz que define el contrato común de cualquier tarea. Incluye la posición de trabajo, la posición de referencia, la prioridad, el rango de interacción, el tipo de trabajo y los métodos necesarios para ejecutar o cancelar la tarea.
- **BuildTask:** tarea asociada a un *Blueprint*. Encapsula la lógica necesaria para construir una estructura y delega la ejecución y la cancelación sobre el propio plano de construcción.
- **GatherResourceTask:** tarea asociada a un nodo de recursos. Al activarse, genera los recursos configurados en la receta correspondiente y registra las nuevas instancias para que puedan ser transportadas por los trabajadores.
- **MoveItemTask:** tarea asociada a un objeto transportable. Gestiona la búsqueda de un consumidor válido, la reserva del destino, la entrega del objeto y la recuperación del estado en caso de cancelación o destrucción del consumidor.
- **ResearchTask:** tarea asociada a un edificio de investigación. Representa el trabajo que debe realizar un trabajador en la *ResearchBench* para avanzar en un proyecto de investigación concreto.

7.2.15. Directorio Wave

En la Figura 7.26 se pueden observar los scripts del directorio *Wave*, que agrupa los componentes relacionados con la generación de oleadas enemigas, la selección de puntos de aparición y la recompensa asociada a la superación de cada oleada.

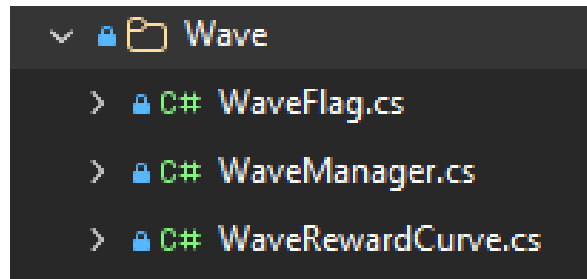


Figura 7.26: Organización del directorio Wave

- **WaveManager**: gestor central del sistema de oleadas. Se encarga de cargar las definiciones de enemigos mediante *Addressables*, clasificarlas según su coste de aparición en categorías barata, media y cara, seleccionar un punto de aparición válido, generar cada oleada en función de un presupuesto que crece progresivamente y notificar a la interfaz cuando una oleada empieza o termina. Además, coordina la activación del indicador visual del punto de aparición y otorga una recompensa adicional de puntuación al completar cada oleada.
- **WaveFlag**: componente visual asociado a un punto de aparición de oleadas. Permite activar o desactivar el modelo gráfico que indica al jugador qué punto está activo en ese momento.
- **WaveRewardCurve**: activo de datos visual que se puede modificar en el editor que define, mediante una curva de valores, la recompensa en puntuación que se obtiene al completar una oleada en función de su número. Permite ajustar fácilmente el progreso de las recompensas sin modificar el código fuente.

7.2.16. Extractos significativos de códigos

En las siguientes subsecciones se muestran algunos extractos de código representativos del proyecto. No se pretende reproducir todo el código fuente, sino ilustrar aquellas partes que resultan más relevantes desde el punto de vista de la arquitectura, la extensibilidad y la coordinación entre subsistemas.

State Pattern

El patrón *State Pattern* [36] se ha utilizado para modelar el comportamiento de las unidades del juego. Este patrón es útil cuando una entidad puede encontrarse en distintos estados de comportamiento con transiciones bien definidas, como reposo, movimiento, persecución, ataque o ejecución de una tarea. Cada estado tiene un método que se lanza al entrar en el estado, un método que se actualiza en cada frame y un método que se

lanza al salir del estado. Cada estado contiene solo la lógica que le interesa, quedando aislado en un módulo, lo cual hace más fácil mantener y escalar el sistema, ya que un nuevo estado no modifica el código existente.

En el caso de *Soldier*, el comportamiento se organiza mediante una interfaz común (*ISoldierState*) y varias clases concretas que representan cada estado. La clase principal mantiene una referencia al estado actual y delega en él la lógica de actualización. Al crearse o cargarse el soldado por primera vez, la clase principal *Soldier* activa el estado *Idle*, el cual se encarga de detectar enemigos y, entonces, cambiar al siguiente estado. Cuando se produce una transición, el estado actual finaliza su ejecución mediante *ExitState*, se sustituye por el nuevo estado mediante *ChangeState* y se ejecuta su método *EnterState*.

En los siguientes extractos se pueden ver los estados Idle, Chase y Attack. Cuando no se detectan enemigos, *HandleCombatEnd()* cambia el estado del soldado a Idle, además de reiniciar las referencias a objetivos.

Extracto de código 7.1: Extracto de SoldierIdleState

```
1 public class SoldierIdleState : ISoldierState
2 {
3     public void EnterState(Soldier soldier)
4     {
5         soldier.unit.agent.isStopped = true;
6     }
7
8     public void UpdateState(Soldier soldier)
9     {
10         ITarget target = soldier.FindNearestEnemyTarget();
11
12         if (target != null && target.IsAlive)
13         {
14             soldier.unit.target = target;
15             soldier.ChangeState(new SoldierChaseState());
16         }
17     }
18
19     public void ExitState(Soldier soldier)
20     {
21
22     }
23 }
```

Extracto de código 7.2: Extracto de SoldierChaseState

```
1 public class SoldierChaseState : ISoldierState
2 {
3     public void EnterState(Soldier soldier)
4     {
5         soldier.unit.agent.isStopped = false;
6         soldier.unit.agent.stoppingDistance = soldier.unit.
            attackRange;
```

```

7      }
8
9      public void UpdateState(Soldier soldier)
10     {
11         if (soldier.unit.target == null || !soldier.unit.target.
            IsAlive)
12         {
13             soldier.HandleCombatEnd();
14             return;
15         }
16
17         soldier.unit.agent.SetDestination(soldier.unit.
            GetTargetAttackPosition(soldier.unit.target));
18
19         float distance = Vector3.Distance(soldier.transform.
            position, soldier.unit.GetTargetAttackPosition(soldier.
            unit.target));
20         if (distance <= soldier.unit.attackRange)
21         {
22             soldier.ChangeState(new SoldierAttackState());
23         }
24     }
25
26     public void ExitState(Soldier soldier)
27     {
28         soldier.unit.agent.ResetPath();
29     }
30 }

```

Extracto de código 7.3: Extracto de SoldierAttackState

```

1 public class SoldierAttackState : ISoldierState
2 {
3     public void EnterState(Soldier soldier)
4     {
5         soldier.unit.agent.isStopped = true;
6     }
7
8     public void UpdateState(Soldier soldier)
9     {
10         if (soldier.unit.target == null || !soldier.unit.target.
            IsAlive)
11         {
12             soldier.HandleCombatEnd();
13             return;
14         }
15
16         float distance = Vector3.Distance(soldier.transform.
            position, soldier.unit.GetTargetAttackPosition(soldier.
            unit.target));
17         if (distance > soldier.unit.attackRange + 0.1f)

```

```

18         {
19             soldier.ChangeState(new SoldierChaseState());
20             return;
21         }
22
23         soldier.unit.FaceTarget(soldier.unit.target.Position, 720f)
24             ;
25
26         if (Time.time >= soldier.unit.nextAttackTime)
27         {
28             soldier.unit.Attack(soldier.unit.target);
29
30             soldier.unit.nextAttackTime = Time.time + soldier.unit.
31                 attackCooldown;
32         }
33     }
34
35     public void ExitState(Soldier soldier)
36     {
37         soldier.unit.agent.isStopped = false;
38     }
39 }

```

Singleton Pattern

El patrón *Singleton* [12] se ha empleado en varios gestores globales del proyecto, como *TaskManager*, para garantizar que exista una única instancia accesible desde cualquier parte del sistema. Este enfoque es habitual en Unity para sistemas transversales que deben coordinar el comportamiento de múltiples entidades sin necesidad de pasar referencias manualmente por toda la jerarquía de objetos.

En *TaskManager*, la instancia se almacena en una propiedad estática *Instance* y se inicializa en el método *Awake*. Si ya existe otra instancia activa, el nuevo objeto se destruye, lo que evita duplicidades. A partir de ese punto, cualquier trabajador puede consultar el gestor central para solicitar tareas, registrarlas o eliminarlas. Este patrón simplifica el acceso global al sistema de tareas y evita dependencias innecesarias entre unidades.

En los siguientes extractos se puede ver cómo se garantiza que solo exista una única instancia del Singleton y el método *RequestTask*, usado por trabajadores en estado *Idle* para solicitar tareas, priorizando tareas cercanas con mayor prioridad.

Extracto de código 7.4: Extracto de *TaskManager* mostrando como se garantiza que solo exista una única instancia

```

1 public class TaskManager : MonoBehaviour
2 {
3     public static TaskManager Instance { get; private set; }
4
5     private void Awake()

```

```

6      {
7          if (Instance != null && Instance != this)
8          {
9              Destroy(gameObject);
10             return;
11         }
12         else
13         {
14             Instance = this;
15         }
16     }
17
18     // Resto del codigo...
19 }

```

Extracto de código 7.5: Extracto de TaskManager mostrando el método RequestTask

```

1 private readonly List<ITask> availableTasks = new();
2
3 public ITask RequestTask(Vector3 workerPosition)
4 {
5     if (availableTasks.Count == 0)
6     {
7         return null;
8     }
9
10    ITask bestTask = null;
11    int highestPriority = int.MinValue;
12    float closestDistance = float.MaxValue;
13
14    foreach (ITask task in availableTasks)
15    {
16        if (task == null)
17        {
18            Debug.LogWarning($"{name}: null {nameof(task)}");
19
20            UnregisterTask(task);
21            continue;
22        }
23
24        int taskPriority = task.Priority;
25        float taskDistance = GetPathLength(workerPosition, task.
            TaskPosition);
26
27        if (taskDistance < 0f)
28        {
29            continue;
30        }
31
32        if (taskPriority > highestPriority)
33        {

```

```

34         highestPriority = taskPriority;
35         closestDistance = taskDistance;
36         bestTask = task;
37     }
38     else if (taskPriority == highestPriority && taskDistance <
39             closestDistance)
40     {
41         closestDistance = taskDistance;
42         bestTask = task;
43     }
44
45     if (bestTask != null)
46     {
47         availableTasks.Remove(bestTask);
48     }
49
50     return bestTask;
51 }

```

Eventos

Los eventos se han utilizado como un mecanismo de comunicación entre subsistemas, permitiendo desacoplar emisores y receptores. En lugar de que un componente conozca directamente a todos los que deben reaccionar a un cambio, se notifica el evento correspondiente y cada subsistema interesado se suscribe a él. Esto mejora la modularidad del proyecto, ya que evita dependencias directas entre sistemas que evolucionan de manera independiente.

Este enfoque se ha usado en gran parte del proyecto. Algunos ejemplos son *StatModifierManager*, que expone el evento *OnModifiersChanged*, notificando a los suscriptores cuando un modificador afecta a sus atributos. *InventoryManager* publica *OnInventoryChanged* para informar a la interfaz y a otros sistemas del cambio de stock. *GameSpeedManager* emite *OnGameSpeedChanged* para sincronizar la interfaz con el ritmo actual de simulación.

En los siguientes extractos se puede ver cómo *StatModifierManager* notifica a los suscriptores que se ha añadido un modificador. Uno de los suscriptores, *Enemy*, registra *HandleModifiersChanged()* al activarse como el método lanzado cuando se recibe el evento.

Extracto de código 7.6: Extracto de *StatModifierManager* mostrando el uso del evento *OnModifiersChanged*

```

1 public event Action<string, string> OnModifiersChanged;
2
3 public void AddModifier(string sourceId, string targetId, StatKey
4     statKey, ModifierKind kind, float value)
5 {

```

```

5      // Se mete un modificador en la lista de modificadores activos.
        Lo cual requiere que los atributos se refresquen
6
7      // Resto del codigo...
8
9      // Se lanza el evento OnModifiersChanged
10     OnModifiersChanged?.Invoke(targetKey, statKey.id);
11 }

```

Extracto de código 7.7: Extracto de Enemy mostrando la suscripción al evento OnModifiersChanged

```

1 private void OnEnable()
2 {
3     // Se suscribe al evento OnModifiersChanged cuando el
        componente se activa
4     // Cuando el evento se lance, HandleModifiersChanged es
        ejecutado
5     StatModifierManager.Instance.OnModifiersChanged +=
        HandleModifiersChanged;
6 }
7
8 void HandleModifiersChanged(string targetId, string statKeyId)
9 {
10     if (targetId == data.id)
11     {
12         RefreshStats();
13     }
14     // Global modifier:
15     else if (string.IsNullOrEmpty(targetId))
16     {
17         RefreshStats();
18     }
19 }
20
21 // RefreshStats() comprueba los atributos asociados al componente y
    los refresca

```

Flyweight Pattern

El patrón *Flyweight* [33] se ha aplicado de forma natural en el proyecto mediante el uso de *ScriptableObjects* para almacenar la información compartida entre numerosas instancias del juego. Este enfoque resulta adecuado en Unity cuando existen muchas entidades que comparten la misma configuración base, pero difieren en su estado en tiempo de ejecución, como ocurre con unidades, estructuras, recursos, investigaciones o recetas.

En este caso, clases como *BaseData*, *UnitData*, *StructureData*, *ItemData* o *ResearchData* contienen el estado intrínseco, es decir, los datos comunes e inmutables del ele-

mento. Las instancias creadas en escena a partir de estos datos mantienen solo el estado extrínseco, como la posición, la vida actual o el progreso de una tarea. De este modo, varias entidades pueden referenciar el mismo activo de datos sin duplicar información innecesariamente.

Este diseño reduce el consumo de memoria, facilita la edición de contenido desde el editor y simplifica la extensión del proyecto. Por ejemplo, todas las instancias de un mismo tipo de recurso o de una misma unidad comparten la misma definición base, mientras que cada objeto en escena conserva su propio estado particular. La separación entre ambos niveles permite crear grandes cantidades de contenido manteniendo una arquitectura limpia y escalable.

En los siguientes extractos se puede observar que el ScriptableObject BaseData, usado como base de numerosos ScriptableObjects, contiene información común, como el nombre o el identificador único. También se puede observar cómo ResearchData hereda de BaseData, evitando duplicar datos como el nombre o el icono, puesto que ya existen en BaseData.

Extracto de código 7.8: Extracto de BaseData

```
1 public abstract class BaseData : ScriptableObject
2 {
3     [Header("Base")]
4
5     public string Name;
6     public string id;
7     public Sprite icon;
8
9     [Tooltip("Optional, if filled, this asset will require this
10         research as requisite to be unlocked/usable")]
11     public ResearchData requiredResearch;
12
13     [Header("Stats")]
14     public List<StatBaseEntry> statBases = new();
15 }
```

Extracto de código 7.9: Extracto de ResearchData

```
1 public class ResearchData : BaseData
2 {
3     [Header("Research")]
4
5     [TextArea(3, 8)]
6     public string description;
7
8     [Tooltip("Total research points required to complete this node")]
9     public float requiredResearchPoints = 100f;
10
11     [Tooltip("Base points awarded per bench cycle")]
12     public float basePointsPerCycle = 10f;
13 }
```

```
14     [Tooltip("ResearchDatas that must be researched before this  
15         research is available")]  
16     public List<ResearchData> prerequisiteResearchDatas;  
17  
18     [Tooltip("Effects that will be applied once this research  
19         completes")]  
18     public ResearchEffect[] effects;  
19 }
```

7.3. Scripts de Base de Datos

El sistema desarrollado no emplea una base de datos relacional ni un sistema gestor de bases de datos externo.

La persistencia de la información se realiza mediante los mecanismos proporcionados por Unity, principalmente:

- ScriptableObjects para la definición de datos configurables.
- Prefabs para la definición de entidades reutilizables.
- Componentes serializados para almacenar parámetros de ejecución.
- Escenas de Unity para la organización del entorno de juego.

Debido a esta arquitectura, no ha sido necesario implementar tablas, procedimientos almacenados, disparadores ni scripts SQL, sino que la estructura física de datos está definida mediante los activos y recursos gestionados por el propio motor Unity.

8. Pruebas del Sistema

En este capítulo se presenta el plan de pruebas del sistema de información, incluyendo los diferentes tipos de pruebas que se han llevado a cabo.

8.1. Estrategia

La estrategia de pruebas adoptada para este proyecto se ha basado en una combinación de validación manual en el editor de Unity y comprobación funcional en escenas de prueba representativas. Dado que se trata de un videojuego en desarrollo continuo, el objetivo principal no ha sido únicamente verificar que el proyecto compila, sino asegurar que los sistemas principales interactúan correctamente entre sí y que los cambios introducidos en una iteración no rompen el comportamiento previamente validado.

El alcance de las pruebas incluye tanto los sistemas de núcleo jugable como los sistemas de apoyo. Se han realizado pruebas de funcionamiento básico, pruebas de casos límite y pruebas de regresión tras introducir cambios estructurales.

Las pruebas se han interpretado de manera pragmática: un sistema se considera correctamente validado cuando responde de forma estable a las entradas previstas, mantiene la coherencia de sus estados internos y no genera errores de integración con otros subsistemas. En un proyecto de este tipo, la ausencia de errores visibles no es suficiente; también es necesario comprobar que las reservas, los contadores, las cancelaciones y los cambios de estado se resuelven de forma consistente.

El procedimiento de regresión ha consistido en repetir las comprobaciones más sensibles cada vez que se ha modificado un subsistema compartido. Cuando un cambio afectaba a varios módulos, se repetían manualmente las escenas de prueba relacionadas para verificar que el comportamiento siguiera siendo correcto.

Los perfiles implicados en las pruebas han sido, en la práctica, dos. Por un lado, el propio desarrollador, que ha asumido la validación técnica del sistema. Por otro lado, el usuario final simulado, utilizado para revisar la usabilidad de la interfaz, la claridad de la retroalimentación y la coherencia de la navegación entre paneles.

Los entornos utilizados para las pruebas han sido los mismos que los del desarrollo: un equipo de escritorio con Unity como entorno principal, escenas dedicadas a comprobaciones funcionales y el sistema de ejecución del editor para observar el comportamiento en tiempo real. En aquellos casos en que se necesitaba aislar un comportamiento concreto, se han empleado escenas reducidas o configuraciones temporales específicas.

8.2. Pruebas Unitarias

En el contexto de este proyecto, las pruebas unitarias se han aplicado sobre componentes concretos cuya validación puede aislarse con relativa facilidad. Dado que parte de la lógica depende del estado de la escena y de la interacción entre prefabs, gestores y activos de datos, en varios casos las comprobaciones se han realizado como pruebas de componente en escenas controladas, manteniendo siempre un alcance reducido y reproducible. El criterio de aceptación fue que el componente produjera siempre el mismo resultado ante las mismas condiciones de entrada y que no generara efectos secundarios no deseados sobre otros subsistemas.

A continuación, se explica cada prueba unitaria realizada:

Sistema de atributos.

Objetivo: Verificar que el sistema de atributos recalcula correctamente los valores finales cuando se modifica alguno de los modificadores activos.

Procedimiento: Se añade un modificador sobre distintas entidades, comprobando que el valor resultante se actualiza inmediatamente y coincide con el valor esperado.

Gestor de tareas

Objetivo: Verificar que el sistema de asignación selecciona la tarea más adecuada para cada trabajador según los criterios definidos.

Procedimiento: Se registran varias tareas con diferentes prioridades y distancias respecto a distintos trabajadores. Posteriormente, se verifica que la tarea asignada corresponde a la opción más adecuada según el algoritmo implementado.

Inventario y almacenes

Objetivo: Verificar la correcta gestión del almacenamiento y retirada de recursos.

Procedimiento: Se añaden y retiran distintas cantidades de recursos en varios almacenes, comprobando tras cada operación que los valores almacenados y el inventario global se actualicen correctamente.

Sistema de investigación

Objetivo: Verificar que las investigaciones progresen correctamente y se apliquen sus efectos al completarse.

Procedimiento: Se activa un proyecto de investigación, se completa su progreso y se comprueba que los desbloques y modificadores asociados se apliquen correctamente en el sistema.

Sistema de combate

Objetivo: Verificar el comportamiento básico de ataque y recepción de daño entre entidades.

Procedimiento: Se enfrentan distintas unidades en una escena controlada y se verifica que los ataques solo se producen dentro del alcance configurado y que el daño aplicado coincide con los valores esperados.

Órdenes RTS

Objetivo: Verificar la correcta recepción y ejecución de órdenes emitidas por el jugador.

Procedimiento: Se selecciona una o varias unidades y se envían órdenes de movimiento a diferentes posiciones del mapa, verificando que las unidades alcancen correctamente sus destinos.

Producción de unidades

Objetivo: Verificar el funcionamiento completo del sistema de producción de unidades.

Procedimiento: Se añaden órdenes de producción a una estructura capaz de fabricar unidades y se comprueba que los recursos se consumen correctamente y que la unidad aparece al finalizar el proceso.

Procesamiento de recursos

Objetivo: Verificar la transformación de recursos mediante recetas configurables.

Procedimiento: Se ejecutan diferentes recetas de procesamiento y se comprueba que los recursos de entrada se consumen correctamente y que se generan los productos de salida previstos.

Sistema de reservas

Objetivo: Verificar que varios trabajadores no puedan utilizar simultáneamente un mismo recurso o destino reservado.

Procedimiento: Se crean varias tareas concurrentes que intentan acceder a un mismo recurso o almacenamiento y se verifica que las reservas impiden conflictos y se liberan al finalizar la tarea.

Sistema de oleadas

Objetivo: Verificar la correcta generación y progresión de las oleadas enemigas.

Procedimiento: Se fuerza el inicio de varias oleadas y se comprueba que los enemigos aparecen en los puntos previstos y en la cantidad configurada para cada una de ellas.

Sistema de retroalimentación visual del combate

Objetivo: Verificar la generación de elementos visuales asociados al daño y a la curación.

Procedimiento: Se provocan distintos eventos de combate y se verifica que los textos flotantes muestren la información correcta y desaparezcan una vez transcurrido el tiempo configurado.

Sistema de velocidad

Objetivo: Verificar el funcionamiento del control de velocidad del juego.

Procedimiento: Se alterna entre los distintos niveles de velocidad disponibles, incluyendo la pausa, verificando que la simulación responda correctamente a cada cambio.

Sistema de selección

Objetivo: Verificar la correcta gestión de la selección de entidades.

Procedimiento: Se realizan selecciones individuales y múltiples sobre unidades y estructuras, comprobando que la información mostrada en la interfaz coincide con los elementos seleccionados.

Sistema de tienda

Objetivo: Verificar las operaciones de compra y venta de recursos.

Procedimiento: Se realizan varias transacciones de compra y venta, y se comprueba que tanto el saldo disponible como el inventario reflejan correctamente los cambios producidos.

8.3. Pruebas de Integración

Las pruebas de integración han sido una parte esencial del proceso, ya que el proyecto está compuesto por subsistemas fuertemente interdependientes.

A diferencia de las pruebas unitarias, aquí no se valida el comportamiento aislado de un único componente, sino el flujo completo de información entre varios sistemas. El criterio de aceptación fue que la interacción entre los módulos produjera siempre el resultado esperado, sin generar bloqueos, inconsistencias ni estados inválidos.

Entrega de recursos a consumidores.

Objetivo: Verificar que los trabajadores pueden transportar recursos hasta consumidores válidos, respetando las reservas, la capacidad disponible y la prioridad de las tareas.

Procedimiento: Se registraron tareas de transporte hacia distintos consumidores y se asignaron trabajadores disponibles. Posteriormente, se comprobó que la entrega se completaba correctamente, que el destino aceptaba el recurso y que las reservas se liberaban al finalizar la tarea.

Construcción y ejecución de planos.

Objetivo: Verificar la interacción entre el sistema de construcción, el sistema de tareas y el sistema de recursos.

Procedimiento: Se colocó una estructura en modo plano, se asignó una tarea de construcción y se comprobaron las fases de avance hasta su finalización.

Órdenes RTS y comportamiento de unidades.

Objetivo: Verificar que las unidades respondan correctamente a las órdenes del jugador sin interferir en sus comportamientos automáticos.

Procedimiento: Se seleccionaron varias unidades y se emitieron órdenes de movimiento. Durante la prueba, se observó si la orden activa cancelaba y reemplazaba correctamente la acción previa y si la unidad ejecutaba la nueva instrucción sin perder coherencia en su estado interno.

Investigación y modificación de atributos.

Objetivo: Verificar que una investigación completada afecte correctamente a las unidades y estructuras relacionadas.

Procedimiento: Se inició un proyecto de investigación, se completó su progreso y se comprobó si los atributos modificados por sus efectos se recalculaban en las entidades afectadas. También se verificó que la interfaz reflejase correctamente el estado de desbloqueo.

Sistema de oleadas y combate.

Objetivo: Verificar que la generación de enemigos, la detección de objetivos y el combate funcionan de forma conjunta.

Procedimiento: Se forzó la aparición de una oleada y se verificó que los enemigos se instanciaran en el punto correcto, que fueran detectados por torres o unidades del jugador, que el daño aplicado modificara correctamente su estado de vida y que la retroalimentación visual se mostrara correctamente.

Sistema de velocidad e interfaz.

Objetivo: Verificar que los cambios en la velocidad del juego se reflejan correctamente en la simulación y en la interfaz.

Procedimiento: Se alternó entre pausa, velocidad normal y aceleración, observando si la simulación respondía al cambio y si la interfaz actualizaba el estado mostrado al jugador.

8.4. Pruebas de Sistema

Las pruebas de sistema se han centrado en validar el comportamiento global del prototipo en condiciones de uso representativas. En este nivel, se ha comprobado que el juego cumple los requisitos funcionales definidos para la versión desarrollada y que los subsistemas principales operan de forma estable dentro de una partida completa o de una escena de prueba equivalente.

Pruebas funcionales

Las pruebas funcionales se han diseñado para verificar el cumplimiento de los requisitos funcionales definidos en la Tabla 4.1. Cada prueba se ha ejecutado en una escena controlada, comprobando tanto el resultado inmediato como la coherencia posterior de los sistemas implicados.

RF-01. Gestión de trabajadores

Objetivo: Verificar que los trabajadores puedan ejecutar tareas de forma autónoma.

Procedimiento: Se generan varios trabajadores y se registran tareas de construcción, transporte e investigación. Se observa la asignación automática de tareas a cada trabajador, el desplazamiento de cada trabajador a cada tarea asignada y la ejecución completa de las mismas.

RF-02. Asignación de tareas

Objetivo: Verificar la correcta asignación de tareas según prioridad y disponibilidad.

Procedimiento: Se registran múltiples tareas con distintas prioridades y ubicaciones. Se observa que las tareas se asignan respetando los criterios de prioridad y distancia establecidos por el sistema.

RF-03. Transporte de objetos

Objetivo: Validar el transporte de recursos entre productores, almacenes y consumidores.

Procedimiento: Se generan recursos en el mundo y se solicita su traslado hacia distintas estructuras consumidoras. Se observa cómo los objetos son recogidos y entregados correctamente, sin producir duplicidades ni pérdidas, y sin sobrepasar los límites de almacenamiento.

RF-04. Inventario y almacenes

Objetivo: Comprobar el almacenamiento y consulta de recursos.

Procedimiento: Se añaden y retiran recursos de distintos almacenes durante una partida. Se observa cómo el stock global refleja correctamente las operaciones realizadas y se respetan los límites de almacenamiento.

RF-05 y RF-06. Construcción y gestión de planos

Objetivo: Verificar el flujo completo de construcción.

Procedimiento: El jugador coloca una estructura, se genera el plano correspondiente y los trabajadores aportan los materiales necesarios. Se observa cómo el plano desaparece al finalizar la construcción y es sustituido por la estructura definitiva.

RF-07 y RF-08. Selección y órdenes RTS

Objetivo: Validar la selección de entidades y la emisión de órdenes.

Procedimiento: Se seleccionan unidades individuales y grupos de unidades, enviándolas a distintos puntos del mapa. Se observa cómo las unidades reciben correctamente las órdenes y se desplazan manteniendo la formación prevista.

RF-09. Procesado de recursos

Objetivo: Verificar el funcionamiento de las recetas de producción.

Procedimiento: Se suministran recursos de entrada a una estructura de procesamiento. Se observa que, al completarse el tiempo de procesamiento, se generan los recursos de salida definidos por la receta.

RF-10. Gestión de umbrales

Objetivo: Comprobar el comportamiento de los umbrales de producción.

Procedimiento: Se configuran diferentes valores límite para un recurso de salida. Se observa que la producción se detiene al alcanzar el umbral y se reanuda cuando vuelve a existir demanda.

RF-11 y RF-12. Investigación y sistema de atributos

Objetivo: Validar la aplicación de mejoras e investigaciones.

Procedimiento: Se completa un proyecto de investigación que modifica atributos de unidades o estructuras. Se observa que los atributos afectados se recalculan automáticamente, reflejando los nuevos valores.

RF-13. Producción de unidades

Objetivo: Comprobar la generación de nuevas unidades.

Procedimiento: Se inicia una orden de producción de unidades desde una estructura habilitada para ello. Se observa cómo la unidad asociada a la orden aparece correctamente en el mundo una vez finalizado el tiempo de producción.

RF-14 y RF-15. Combate y objetivos hostiles

Objetivo: Verificar los sistemas de combate y detección de enemigos.

Procedimiento: Se enfrentan unidades aliadas y enemigas en distintas situaciones de combate. Se observa cómo los objetivos válidos son detectados correctamente, se aplica daño y las entidades reaccionan según su estado.

RF-16. Interfaz de usuario

Objetivo: Validar la interacción del jugador con los distintos paneles.

Procedimiento: Se utilizan los sistemas de construcción, investigación, producción, inventario y tienda durante una partida. Se observa que la información mostrada

coincide con el estado real del sistema y que las acciones ejecutadas producen los efectos esperados.

RF-17. Control de velocidad

Objetivo: Comprobar el control temporal de la simulación.

Procedimiento: Se alterna entre pausa, velocidad normal y velocidades aceleradas. Se observa cómo todos los sistemas del juego responden de forma consistente a los cambios de velocidad.

Pruebas no funcionales

Las pruebas no funcionales han evaluado aspectos relacionados con la calidad global del sistema, teniendo en cuenta los requisitos no funcionales definidos en la Tabla [4.2](#).

RNF-01 y RNF-02. Mantenibilidad y extensibilidad

Objetivo: Verificar que el código esté organizado en módulos desacoplados y que permita incorporar contenido nuevo sin modificar la arquitectura existente.

Procedimiento: Se añaden nuevas estructuras, unidades y recetas mediante ScriptableObjects sin alterar el código fuente. Se comprueba que los sistemas las reconozcan y gestionen correctamente en tiempo de ejecución.

RNF-03 y RNF-04. Robustez y consistencia

Objetivo: Comprobar que el sistema gestiona correctamente cancelaciones, destrucciones de entidades y cambios de estado en curso, manteniendo la coherencia de reservas y contadores.

Procedimiento: Se cancelan tareas en distintas fases de ejecución, se destruyen entidades con reservas activas y se provocan transiciones de estado durante acciones en curso. Se observa que no se generan errores, recursos duplicados ni bloqueos en los sistemas implicados.

RNF-05. Rendimiento

Objetivo: Verificar que los sistemas evitan comprobaciones innecesarias de forma continua.

Procedimiento: Se revisa que los sistemas principales empleen eventos, temporizadores e intervalos en lugar de comprobaciones continuas por cada fotograma. Se ejecuta una sesión de juego con múltiples entidades activas y se observa la estabilidad del rendimiento.

RNF-06. Usabilidad

Objetivo: Validar que la interfaz presenta la información de forma clara y permite interactuar con las mecánicas principales de manera comprensible.

Procedimiento: Se utilizan todos los paneles de interfaces durante una partida completa. Se observa que la información mostrada es legible, coherente con el estado del sistema, y que el cierre de ventanas y la navegación entre paneles se comportan de forma consistente.

RNF-07. Portabilidad

Objetivo: Comprobar que el proyecto puede compilarse y ejecutarse en una plataforma de escritorio estándar.

Procedimiento: Se realiza una compilación del proyecto en la plataforma objetivo. Se verifica que la ejecución es correcta sin dependencias externas no justificadas.

RNF-08. Escalabilidad de contenido

Objetivo: Verificar que nuevas mecánicas y contenido pueden incorporarse mediante datos configurables.

Procedimiento: Se crean nuevos ScriptableObjects de cada tipo existente y se registran en el sistema mediante Addressables. Se comprueba que los sistemas los cargan y procesan correctamente sin necesidad de modificar el código fuente.

RNF-09. Trazabilidad

Objetivo: Comprobar que los sistemas permiten identificar con claridad el origen de errores y cambios de estado.

Procedimiento: Se provocan situaciones de error controladas, como referencias nulas, configuraciones incompletas en ScriptableObjects y cancelaciones inesperadas. Se observa que los registros generados identifican correctamente el componente y el contexto del problema.

RNF-10. Calidad visual y de retroalimentación

Objetivo: Validar que las acciones relevantes generan una respuesta visual comprensible para el jugador.

Procedimiento: Se provocan acciones relevantes, como daño, curación y cambio de estado de oleada. Se observa que cada evento produce la respuesta visual correspondiente, como retroalimentación visual del combate mediante textos flotantes, barras de vida y texto informando del estado de la oleada.

8.5. Pruebas de Aceptación

Las pruebas de aceptación se han entendido como la validación final de que la rama de desarrollo actual está en un estado suficientemente estable y completo como para entrar en la rama de producción. En este nivel, se ha comprobado que se puede jugar sin el editor de Unity, que se pasan con éxito todas las pruebas anteriores en la rama de producción, que las funcionalidades clave están disponibles, que la experiencia de juego es coherente y que el usuario puede interactuar con todos los sistemas correctamente.

9. Despliegue del Sistema

Este capítulo recoge la arquitectura física planteada para el sistema, las instrucciones para su despliegue y las instrucciones para la operación y mantenimiento del nivel de servicio.

9.1. Arquitectura Física

El sistema desarrollado no sigue una arquitectura cliente-servidor ni requiere una infraestructura física compleja para su ejecución. Se trata de un videojuego de escritorio desarrollado en Unity, por lo que su despliegue se realiza sobre una única máquina local en la que se ejecutan tanto el motor como la aplicación compilada.

La arquitectura física puede describirse como un despliegue monolítico en un equipo de usuario final o de desarrollo. Sobre este equipo se instalan el sistema operativo, el motor de Unity y, en fase de desarrollo, las herramientas necesarias para compilar y depurar el proyecto. Una vez generado el *Build*, el juego puede ejecutarse de forma autónoma sin depender del editor.

Los principales elementos de la arquitectura física son los siguientes:

- **Equipo de desarrollo o ejecución:** ordenador personal con capacidad suficiente para ejecutar Unity y el juego compilado.
- **Sistema operativo:** entorno de escritorio Windows.
- **Motor de juego:** Unity, encargado de ejecutar la lógica, la escena y los recursos del proyecto.
- **Componentes auxiliares:** paquetes de Unity utilizados por el proyecto, como Input System, TextMeshPro, NavMesh, Addressables y ML-Agents.

A diferencia de una aplicación empresarial tradicional, no se requiere un servidor de aplicaciones, base de datos externa ni servicios de red obligatorios para el funcionamiento básico del prototipo. Toda la lógica del sistema, así como los datos configurables, se encuentran integrados en el propio proyecto de Unity.

9.2. Instrucciones de despliegue

El despliegue del sistema consiste en abrir el proyecto en Unity, comprobar que las dependencias están correctamente importadas y generar una compilación ejecutable para el sistema operativo objetivo. El proceso es el siguiente:

1. Clonar o descargar el repositorio del proyecto.
2. Abrir el proyecto con la versión de Unity utilizada durante el desarrollo.

3. Verificar que todos los paquetes necesarios estén instalados.
4. Generar el *Build* para la plataforma de destino.
5. Ejecutar el juego compilado y validar su funcionamiento.

9.2.1. Requisitos previos

Para instalar y ejecutar correctamente el sistema, se requiere:

- Un ordenador con sistema operativo Windows 11 de escritorio.
- Una versión compatible de Unity instalada.
- El proyecto completo descargado o clonado en el equipo local.
- Espacio en disco suficiente para el editor, los paquetes y el *Build* generado.

En fase de desarrollo, también se requiere un editor de código compatible con C#, además del acceso a los paquetes de Unity utilizados por el proyecto.

9.2.2. Inventario de componentes

La versión desplegada del producto incluye los siguientes componentes principales:

- Escenas de juego y escenas auxiliares.
- Prefabs de unidades, estructuras, proyectiles, planos, elementos de interfaz y retroalimentación visual.
- ScriptableObjects con datos de unidades, estructuras, ítems, investigaciones, recetas y atributos.
- Código fuente del proyecto, organizado en carpetas funcionales.
- Recursos gráficos, animaciones, materiales y elementos de interfaz.

En una versión compilada para el usuario final, el producto contiene únicamente los componentes necesarios para ejecutar la partida, mientras que el código fuente y los recursos de edición quedan reservados al entorno de desarrollo.

9.2.3. Procedimientos de instalación

La instalación del sistema en el entorno de desarrollo consiste en abrir el proyecto en Unity y permitir la importación automática de los recursos y paquetes asociados.

Para generar la versión final del juego, se debe utilizar el sistema de compilación de Unity, seleccionar la plataforma de destino y lanzar el proceso de *Build*. El resultado será un ejecutable autónomo acompañado de los recursos necesarios para su funcionamiento.

9.2.4. Pruebas de implantación

Tras instalar o compilar el sistema, deben realizarse comprobaciones básicas para asegurar que la versión desplegada funciona de forma correcta. Las pruebas de implantación más relevantes son:

- Verificación del arranque correcto de la escena inicial.
- Comprobación de la carga de la interfaz principal.
- Validación de la entrada de teclado y ratón.
- Comprobación de la carga de datos configurables.
- Verificación de que no existen errores críticos al iniciar una partida.

Estas pruebas permiten detectar rápidamente problemas de configuración, referencias perdidas o errores de compilación que no se manifiestan hasta la ejecución del *Build*.

9.3. Instrucciones para la operación del sistema y mantenimiento del nivel de servicio

El mantenimiento del sistema se centra en preservar la estabilidad del prototipo durante su uso y en posibles ampliaciones futuras. Al tratarse de un videojuego de escritorio sin servicios de red ni persistencia remota compleja, las tareas de operación son sencillas.

Las principales medidas de operación y mantenimiento son las siguientes:

- Realizar copias de seguridad periódicas del repositorio y de los recursos del proyecto.
- Revisar la consola de Unity durante el desarrollo o la ejecución para detectar errores y advertencias.
- Mantener actualizada la configuración de escenas, prefabs y referencias serializadas.
- Comprobar que los paquetes de Unity utilizados siguen siendo compatibles con la versión del motor.
- Validar cualquier modificación importante mediante escenas de prueba antes de integrarla en la versión principal.

En caso de introducir nuevas funcionalidades, se recomienda seguir la política de mantenimiento preventivo utilizada en el desarrollo del proyecto, basada en iteraciones pequeñas, pruebas frecuentes y control de versiones. Este enfoque reduce el riesgo de romper sistemas ya validados, especialmente en un proyecto con múltiples subsistemas interdependientes.

Dado que no existe una infraestructura externa que requiera monitorización permanente, el mantenimiento del nivel de servicio se limita principalmente a garantizar que el proyecto pueda abrirse, compilarse y ejecutarse correctamente en el entorno objetivo,

así como a preservar la coherencia entre el código fuente y los datos configurables del juego.

Parte III

Epílogo

10. Conclusiones

En este último capítulo se detallan las lecciones aprendidas tras el desarrollo del presente proyecto y se identifican las posibles oportunidades de mejora sobre el software desarrollado.

10.1. Objetivos alcanzados

A lo largo del desarrollo del proyecto se han alcanzado todos los objetivos principales definidos al inicio de la memoria. De forma resumida, se ha conseguido un prototipo jugable de estrategia y gestión 3D, desarrollado en Unity, que integra mecánicas de construcción, administración de recursos, investigación, producción, combate y control de unidades.

También se ha conseguido diseñar una arquitectura modular y basada en datos, donde los sistemas principales se encuentran desacoplados y organizados en componentes reutilizables. Esto ha permitido incorporar subsistemas complejos, como el sistema de atributos o las tareas de los trabajadores, sin perder coherencia estructural.

10.2. Lecciones aprendidas

Durante el desarrollo del proyecto se han consolidado varias buenas prácticas tanto a nivel técnico como metodológico. Una de las principales ha sido la importancia de separar los datos configurables de la lógica de ejecución usando `ScriptableObjects`, ya que esta decisión simplifica la creación de contenido nuevo y reduce la necesidad de modificar el código fuente para ajustar el comportamiento del juego.

Otra lección relevante ha sido la utilidad de construir el proyecto de manera incremental, usando el sistema de ramas propuesto por *Gitflow*, donde cada rama *feature* contiene un solo cambio principal antes de ser unida a la rama de desarrollo. En un sistema con muchos subsistemas interdependientes, intentar implementar varias mecánicas complejas a la vez aumenta considerablemente la probabilidad de errores.

Otra lección ha sido la necesidad de prestar atención al desacoplamiento entre componentes. El uso de interfaces, eventos y gestores centralizados ha resultado especialmente útil para reducir dependencias innecesarias y hacer que el código sea más mantenible. Al principio, ciertos elementos, como *UIManager*, se convirtieron en clases monolíticas encargados de demasiadas funcionalidades, creando problemas a la hora de añadir nuevas funcionalidades, generando deuda técnica y forzando la creación de módulos más pequeños que dividen las tareas de *UIManager*.

Desde el punto de vista cuantitativo, el proyecto ha requerido una inversión elevada de tiempo en investigación, implementación y depuración (sobre todo investigación y

depuración). La mayor parte del esfuerzo no se ha concentrado únicamente en crear funcionalidades visibles, sino en hacer que interactuaran correctamente entre sí y en corregir casos límite que solo aparecen cuando varios sistemas funcionan al mismo tiempo.

En términos personales y técnicos, el proyecto ha servido para profundizar en Unity, en el diseño de sistemas modulares, en la gestión de datos mediante ScriptableObjects y en la construcción de mecánicas complejas propias de un videojuego de estrategia y gestión.

10.3. Trabajo futuro

Una primera línea de trabajo consistiría en ampliar el contenido jugable: más tipos de estructuras, unidades, recetas, investigaciones, enemigos y recursos. Esto permite enriquecer la experiencia, aumentar la variedad disponible para el jugador y profundizar en la progresión del jugador. Por ejemplo, aumentar la complejidad de crear soldados, haciendo falta una estructura intermedia entre el procesamiento y la creación del soldado, como una herrería, donde se producen espadas, necesarias para crear al soldado. Esta nueva estructura llevaría asociada proyectos de investigación, requeriría más espacio en la escena, forzando al jugador a expandirse y adaptarse a una estrategia más dinámica.

Otra mejora importante sería refinar el sistema de inteligencia artificial, tanto para trabajadores como para unidades de combate. En el caso de los trabajadores, podrían incorporarse estrategias más avanzadas de priorización y redistribución automática de tareas; en este momento, las tareas son asignadas únicamente por prioridad y distancia. En el caso del combate, sería interesante introducir comportamientos tácticos más avanzados que simplemente elegir el enemigo más cercano, ya que suele ocurrir que un grupo de soldados tiene como objetivo el mismo enemigo, ignorando al resto hasta que el primer enemigo muere. En el caso de enemigos básicos, suele pasar que son atacados por más soldados de los necesarios, malgastando ataques.

También se podría ampliar el sistema de interfaz con pantallas más completas, mejores indicadores de estado y paneles de información más detallados. En la versión actual, la interfaz tiene pocas animaciones o efectos y parece más un prototipo de interfaz que una interfaz final.

Otra posible mejora futura sería incorporar un sistema de guardado y carga, así como un mejor soporte para la persistencia entre sesiones.

Se podría desarrollar una capa adicional de pruebas unitarias para validar de forma menos manual el comportamiento de los sistemas más críticos.

Información sobre Licencia

La documentación de este proyecto se distribuye bajo la licencia *Creative Commons Reconocimiento–NoComercial–CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)*.

El código fuente del videojuego se distribuye bajo la licencia *MIT*.

Bibliografía

- [1] Adobe (2007). Liberation sans font. <https://www.dafont.com/liberation-sans.font>.
- [2] Adobe (2025). Mixamo - free 3d animations. <https://www.mixamo.com/#/>.
- [3] Brackeys (2025). Brackeys. <https://www.flaticon.com/>. Youtube channel with guides for making games in Unity.
- [4] Brown, Simon (2022). The c4 model for visualising software architecture. <https://c4model.com/>.
- [5] Dan Pristupov & Tanya Pristupova (2026). Fork - a fast and friendly git client. <https://git-fork.com/>.
- [6] Door 407 (2024). Diplomacy is not an option. https://store.steampowered.com/app/1272320/Diplomacy_is_Not_an_Option/. Diplomacy is Not an Option es un juego RTS donde el jugador ha de expandir su base mientras se defiende ante oleadas constantes de enemigos.
- [7] EBIS (2026). Principios solid: Guía completa. <https://www.ebiseducation.com/principios-solid-guia-completa>. Guía para los principios SOLID.
- [8] Endesa (2026). Precio de la luz hoy. <https://www.endesa.com/es/luz-y-gas/luz/precio-de-la-luz-hoy>. Coste €/kWh.
- [9] Epic Games (1998). Unreal engine. <https://www.unrealengine.com/>. Unreal Engine es un motor gráfico para videojuegos 3D.
- [10] Flaticon (2025). Vector icons and stickets. <https://www.flaticon.com/>.
- [11] GanzSe (2026). Ganzse free modular character. <https://assetstore.unity.com/packages/3d/characters/humanoids/fantasy/ganzse-free-modular-character-fantasy-low-poly-pack-321521>. Unity free asset used for the model of several units.
- [12] GeeksForGeeks (2026). Singleton pattern in unity. <https://www.geeksforgeeks.org/c-sharp/singleton-pattern-in-unity/>.
- [13] Github (2008). Github. <https://github.com/>.
- [14] itch.io (2026). Top game assets - itch.io. <https://itch.io/game-assets>. Website used to obtain 3D models.
- [15] Juan Linietsky and Ariel Manzur (2014). Godot. <https://godotengine.org/>. Godot es un motor gráfico open source para videojuegos tanto 2D como 3D.

- [16] Klei Entertainment (2019). Oxygen not included. <https://www.klei.com/games/oxygen-not-included>. Oxygen Not Included es un juego de simulacion con multiples sistemas complejos interconectados.
- [17] Leslie Lamport (1984). Latex, software system for typesetting documents. <https://www.latex-project.org/>.
- [18] Linus Torvalds (2005). Git, free and open source distributed version control system. <https://git-scm.com/>.
- [19] Ludeon Studios (2018). Rimworld. <https://store.steampowered.com/app/294100/RimWorld/>. Rimworld es un juego de simulación de colonias dirigido por una IA.
- [20] Microsoft (2000). C sharp - wikipedia. https://es.wikipedia.org/wiki/C_Sharp. Lenguaje de programación C#.
- [21] Microsoft (2026). Visual studio: Ide and code editor. <https://visualstudio.microsoft.com/>.
- [22] Obsidian (2026). Obsidian, aplication for notes. <https://obsidian.md/>.
- [23] Talent (2026). Ingeniero informático salario en españa. <https://es.talent.com/salary?job=ingeniero+inform%C3%A1tico>. Salario por hora de ingenieros informaticos juniors.
- [24] Tarn Adams & Zach Adams (2006). Dwarf fortress. . Dwarf Fortress es un juego de construcción y gestión de enanos en un entorno altamente simulado con sistemas interconectados complejos.
- [25] Unity (2005). Unity engine. <https://unity.com/>. Unity Engine es un motor gráfico para videojuegos tanto 2D como 3D.
- [26] Unity (2022). Create a c# style guide. https://cdn.bfldr.com/S5BC9Y64/at/5v473vn8kz7p85275gw8vnph/2022_WritingCleanerCodeThatScales_EBook-v6-Final.pdf.
- [27] Unity (2025a). Best practices for project organization and version control. https://cdn.bfldr.com/S5BC9Y64/at/hnnjs88z588fn62jggh9br6/Best_practices_for_project_organization_and_version_control_Unity_6_edition.pdf.
- [28] Unity (2025b). Create a modular game architecture in unity with scriptableobjects. <https://cdn.bfldr.com/S5BC9Y64/at/3fctx42nqsmx4f9jj7gmjn3q/create-modular-game-architecture-in-unity-with-scriptableobjects.pdf>.
- [29] Unity (2025c). Level up your code with design patterns and solid. https://cdn.bfldr.com/S5BC9Y64/at/2g8gww5kf6gsb2krfpjhcv/Level_up_your_code_with_design_patterns_and_SOLID-e-book.pdf.
- [30] Unity (2025d). Optimize your game performance for consoles and pcs in unity. https://cdn.bfldr.com/S5BC9Y64/at/xbhk7z8kvtttn35t3nx45mm98/Optimize_your_game_performance_for_consoles_and_PCs_in_Unity_Unity_6_edition_e-book.pdf.

- [31] Unity (2025e). Separate game data and logic with scriptableobjects. <https://unity.com/how-to/separate-game-data-logic-scriptable-objects>.
- [32] Unity (2026a). Ai navigation. <https://docs.unity3d.com/Packages/com.unity.ai.navigation@2.0/manual/CreateNavMesh.html>.
- [33] Unity (2026b). Flyweight pattern - unity learn. <https://learn.unity.com/course/design-patterns-unity-6/tutorial/flyweight-pattern>.
- [34] Unity (2026c). Input system 1.8.2. <https://docs.unity3d.com/Packages/com.unity.inputsystem@1.8/manual/index.html>.
- [35] Unity (2026d). ML-agents overview. <https://docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/index.html>.
- [36] Unity (2026e). State pattern — unity design patterns. <https://www.unitydesignpatterns.com/patterns/state>.
- [37] Unity (2026f). Textmesh pro documentation. <https://docs.unity3d.com/Packages/com.unity.textmeshpro@3.2/manual/index.html>.
- [38] Unity (2026g). Unity - manual: Addressables. <https://docs.unity3d.com/6000.4/Documentation/Manual/com.unity.addressables.html>.
- [39] Unity (2026h). Unity - manual: ScriptableObject. <https://docs.unity3d.com/6000.3/Documentation/Manual/class-ScriptableObject.html>.
- [40] Unity Learn (2025a). Create modular and maintainable code with the observer pattern. <https://learn.unity.com/course/design-patterns-unity-6/tutorial/create-modular-and-maintainable-code-with-the-observer-pattern#679b854eedbc2a061a71727e>.
- [41] Unity Learn (2025b). Use object pooling to boost performance of c# scripts in unity. <https://learn.unity.com/course/design-patterns-unity-6/tutorial/use-object-pooling-to-boost-performance-of-c-scripts-in-unity>.
- [42] WuBe Software (2020). Factorio. <https://www.factorio.com/>. Factorio es un juego de automatización donde el jugador construye infraestructura para automatizar la extracción y producción de recursos.
- [43] Xbox Game Studios (2019). Age of empires. https://store.steampowered.com/app/1017900/Age_of_Empires_Definitive_Edition/. Age of Empires es un juego RTS donde el jugador expande su base y su economía mientras dirige ejércitos.

Parte IV

Anexos

A. Manual del desarrollador

A continuación se recogen las instrucciones necesarias para evolucionar el software. Este manual está dirigido a los desarrolladores que pretenden extender o modificar el código fuente, con el fin de incorporar nuevas funcionalidades o modificar las ya existentes.

A.1. Introducción

Este manual está dirigido a desarrolladores que deseen ampliar, modificar o corregir el videojuego desarrollado en este proyecto. Su objetivo es facilitar la comprensión de la arquitectura interna del software, explicar cómo preparar el entorno de desarrollo y detallar las pautas básicas que conviene seguir para introducir nuevas funcionalidades sin comprometer la estabilidad del sistema.

A.2. Preparación del entorno de trabajo

Para trabajar en el proyecto, es necesario disponer de un entorno compatible con Unity y con el resto de herramientas utilizadas durante el desarrollo. El equipo de desarrollo debe contar, como mínimo, con un sistema operativo de escritorio compatible, espacio suficiente en disco, memoria RAM adecuada para ejecutar el editor y una tarjeta gráfica capaz de mover escenas 3D en tiempo real.

El entorno de desarrollo recomendado incluye:

- Unity con la versión empleada en el proyecto.
- Un editor de código compatible con C#, como Visual Studio.
- Git para control de versiones.
- Acceso al repositorio del proyecto.
- Los paquetes de Unity utilizados por el proyecto, como Input System, TextMesh-Pro, NavMesh, Addressables y ML-Agents.
- Sistema operativo Windows, preferiblemente Windows 11.

Para abrir correctamente el proyecto, se debe clonar o descargar el repositorio completo y abrirlo desde Unity Hub, asegurándose de que la versión de Unity coincida con la utilizada durante el desarrollo.

En cuanto al control de versiones, el repositorio debe mantenerse con una estructura limpia de ramas y *commits* descriptivos usando *Gitflow*. Cualquier modificación importante debería realizarse sobre una rama *feature* y fusionarse mediante *Pull Requests* a la rama de desarrollo únicamente cuando haya sido validada.

A.3. Consideraciones generales sobre el desarrollo

El proyecto está organizado siguiendo una arquitectura modular basada en componentes, datos configurables y gestores globales. A la hora de modificar o ampliar el código fuente, conviene respetar una serie de criterios generales:

- Mantener la separación entre datos, lógica de juego e interfaz.
- Reutilizar interfaces y eventos para comunicar sistemas siempre que sea posible.
- Evitar dependencias circulares entre gestores y componentes de escena.
- Centralizar la lógica compartida en gestores o utilidades comunes en lugar de duplicarla.
- Emplear los patrones de diseño explicados a lo largo de este documento.

En este proyecto destacan varios subsistemas que conviene tratar con especial cuidado:

- **Sistema de atributos:** cualquier nueva unidad o estructura debe enlazar correctamente sus *StatKey* y actualizar sus valores cuando cambien los modificadores.
- **Sistema de tareas:** si se introduce una tarea nueva, debe registrarse en *TaskManager* y respetar el patrón de reserva, ejecución y cancelación.
- **Inventario y almacenes:** las reservas de entrada y salida deben liberarse correctamente para evitar bloqueos o desajustes.
- **Construcción:** los planos y las sesiones de construcción requieren un manejo estricto de cancelaciones y de estados ocupados.
- **Combate:** las entidades que puedan recibir daño deben mantener sus eventos sincronizados con el sistema de retroalimentación visual.
- **Interfaz:** los paneles comparten reglas de apertura y cierre, por lo que conviene reutilizar el mismo patrón de interacción, heredando de *BasePanel*.

Como criterio de estilo, se recomienda mantener nombres descriptivos, una responsabilidad clara por clase y una estructura de métodos breve y comprensible. Los cambios que afecten a subsistemas transversales deben probarse de forma aislada antes de integrarlos en una escena completa.

A.4. Instrucciones para construcción

Para depurar el software en una máquina de desarrollo, se recomienda seguir los siguientes pasos:

1. Abrir el proyecto en Unity con la versión correspondiente.
2. Esperar a que termine la importación y compilación de scripts.
3. Verificar que no existan errores de compilación en la consola.

4. Comprobar que las escenas principales están disponibles y correctamente configuradas.
5. Ejecutar el proyecto desde el editor para validar el comportamiento en tiempo real.

Para construir el software en una máquina de desarrollo, se recomienda seguir los siguientes pasos:

1. Abrir el proyecto en Unity con la versión correspondiente.
2. Esperar a que termine la importación y compilación de scripts.
3. Verificar que no existan errores de compilación en la consola.
4. Comprobar que las escenas principales están disponibles y correctamente configuradas.
5. Acceder a la configuración de *Build* del editor.
6. Seleccionar la plataforma de destino.
7. Asegurarse de que las escenas necesarias están incluidas en la lista de escenas a construir.
8. Seleccionar *Build*, esto genera una carpeta con el ejecutable y demás archivos necesarios.

B. Manual de usuario

Las instrucciones de uso del software se detallan a continuación. Este manual se dirige al usuario final del software objetivo de este proyecto.

B.1. Introducción

Este manual está dirigido al usuario final del videojuego desarrollado en este proyecto. Su objetivo es explicar de forma clara cómo instalar y ejecutar el software, así como describir las principales acciones disponibles durante la partida.

El juego es un prototipo de estrategia y gestión 3D en el que el jugador administra una base mediante la construcción de estructuras, la asignación de unidades, la recolección y procesamiento de recursos, la investigación de mejoras y el combate contra enemigos. La experiencia se apoya en una interfaz de usuario que permite acceder a las distintas mecánicas de forma ordenada, junto con sistemas de selección, órdenes, inventario y producción.

B.2. Instalación

El software se distribuye como una aplicación de escritorio para sistemas compatibles con Unity. Para utilizarlo correctamente, se recomienda disponer de un equipo con las siguientes características mínimas:

- Sistema operativo de escritorio compatible, como Windows 11.
- Procesador y memoria suficientes para ejecutar un videojuego 3D en tiempo real.
- Tarjeta gráfica compatible con escenas 3D y efectos básicos de interfaz.
- Espacio libre en disco para la instalación del ejecutable y sus recursos asociados.

Los pasos generales para instalar el juego son los siguientes:

1. Descargar o copiar la versión compilada del juego en una carpeta local.
2. Descomprimir, si procede, el paquete de distribución.
3. Ejecutar el archivo principal de la aplicación.

B.3. Uso del sistema

Al iniciar el juego, el usuario accede al menú principal, desde donde puede comenzar una partida. Una vez cargada la partida, la interfaz principal permite gestionar las

distintas mecánicas del juego a través de paneles y controles contextuales.

Controles generales

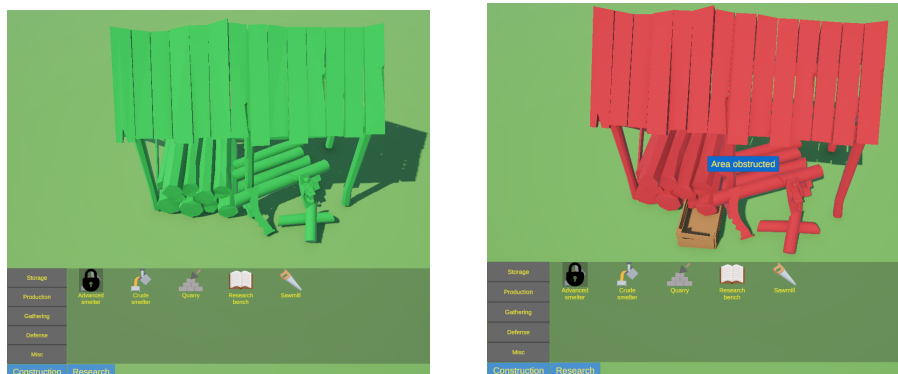
- **Movimiento de cámara:** se realiza mediante teclado y ratón, permitiendo desplazar la vista por el escenario, rotarla y ajustar la altura.
- **Selección:** el jugador puede seleccionar unidades o estructuras haciendo clic sobre ellas. Arrastrando el clic puede seleccionar varias unidades a la vez.
- **Cierre de paneles:** los paneles de interfaz pueden cerrarse con la tecla *Escape*, con clic derecho o haciendo clic fuera del panel.
- **Velocidad del juego:** el usuario puede pausar la partida o modificar su velocidad mediante los botones específicos de la interfaz o los atajos de teclado asignados (*Space*, *1*, *2*, *3*).

Construcción

Para construir una estructura, el jugador debe abrir el panel de construcción y seleccionar la estructura a construir. Aparecerá una previsualización en el terreno que permite comprobar y rotar la posición antes de confirmar. El material de la previsualización cambia según el estado: verde si es válida, rojo si es inválida y azul una vez confirmada la posición, para indicar que la estructura se ha colocado como un plano.

En la Figura B.1a se puede ver la previsualización de un aserradero en una posición válida. En la Figura B.1b se puede ver la misma previsualización, esta vez en una posición inválida, ya que la previsualización está bloqueada por otra entidad.

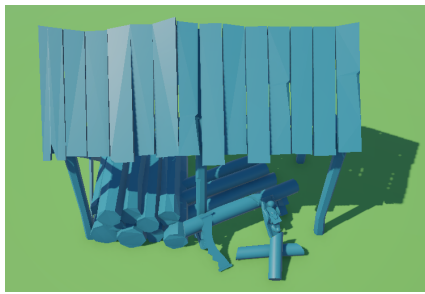
Al confirmar la colocación, si la posición es válida, se crea un plano (Figura B.2a). Después, los trabajadores pueden transportar los materiales necesarios y completar la construcción hasta convertir el plano en la estructura final, como se puede ver en la Figura B.2b.



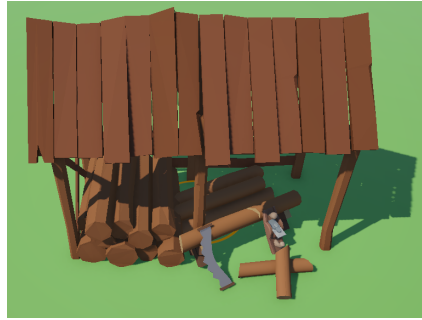
(a) Previsualización válida

(b) Previsualización inválida

Figura B.1: Previsualización de la estructura a construir



(a) Plano de la estructura



(b) Estructura construida

Figura B.2: Plano y construcción

Trabajadores y tareas

Los trabajadores son unidades encargadas de ejecutar tareas automáticamente. Pueden recolectar recursos de la naturaleza, transportar objetos, colaborar en la construcción y participar en otros procesos del juego. El sistema asigna las tareas de forma automática en función de la prioridad y la disponibilidad.

El jugador no necesita ordenar manualmente cada acción básica del trabajador, aunque sí puede intervenir indirectamente mediante la construcción de nuevas estructuras, la configuración de recursos o la activación de proyectos de investigación que afectan las tareas disponibles para los trabajadores.

En la Figura B.3a se puede ver a un trabajador completando una tarea de recolección de recursos, específicamente madera. En la Figura B.3b se puede ver a un trabajador investigando un proyecto, el cual ha sido activado por el jugador, creando la tarea de investigación de manera indirecta.



(a) Trabajador talando un árbol



(b) Trabajador investigando un proyecto

Figura B.3: Trabajador completando tareas

Recursos e inventario

Los recursos recogidos o producidos se almacenan en el inventario y en los almacenes disponibles. El jugador puede consultar el stock acumulado (Figura B.4a) y, en

determinadas estructuras, ajustar umbrales de producción o recolección para evitar sobreacumulación. (Figura B.4b).

Los almacenes aceptan distintos tipos de objetos en función de su categoría. Esto permite organizar el crecimiento de la base de forma más eficiente y controlar qué recursos se almacenan en cada momento.



(a) Inventario



(b) Umbral de una estructura de procesamiento

Figura B.4: Inventario y umbrales

Investigación

En la Figura B.5 se puede ver la interfaz de investigación. La investigación permite desbloquear contenido nuevo y aplicar mejoras permanentes a los atributos del sistema. Para iniciar una investigación, el jugador debe acceder al panel correspondiente y seleccionar la mejora disponible, siempre que cumpla con los requisitos previos.

Cuando una investigación se completa, sus efectos se aplican automáticamente y pueden modificar atributos de unidades, estructuras o sistemas globales.

Producción

Algunas estructuras permiten producir unidades o procesar recursos. En estos edificios, el jugador puede crear órdenes de producción que consumen materiales y avanzan hasta completar el resultado final, como el barracón, una estructura donde se producen unidades de combate.

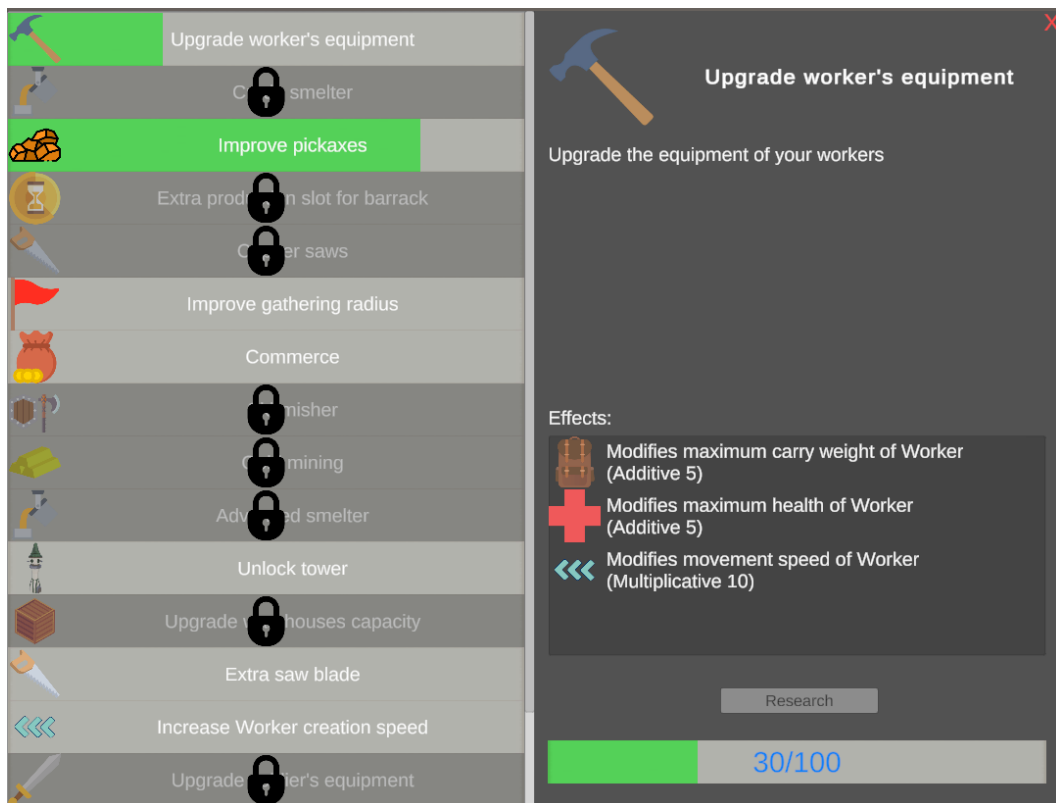


Figura B.5: Interfaz de investigación

En el caso de los procesadores de recursos, es posible definir umbrales de producción para evitar que se sigan generando objetos cuando ya existe suficiente stock disponible. Esto se puede observar en la Figura B.4b mostrada anteriormente.

Combate

Las unidades y estructuras defensivas pueden atacar a enemigos automáticamente cuando detectan objetivos hostiles dentro de su radio de acción. El jugador puede desplegar soldados, héroes, torres y muros para proteger la base.

Cuando una entidad recibe daño o se cura, el sistema muestra retroalimentación visual en forma de textos flotantes, lo que facilita la comprensión del resultado de los combates en tiempo real.

En la Figura B.6 se puede ver a una torre del jugador disparando a un enemigo, mientras que un soldado del jugador y otro enemigo combaten.

Órdenes RTS

El sistema de selección y órdenes permite controlar varias unidades al mismo tiempo. Tras seleccionar una o varias entidades, el jugador puede emitir órdenes de movimiento y vigilancia. El sistema mueve las unidades en formación y distribuye posiciones de destino para evitar que todas las unidades converjan exactamente sobre el mismo punto.

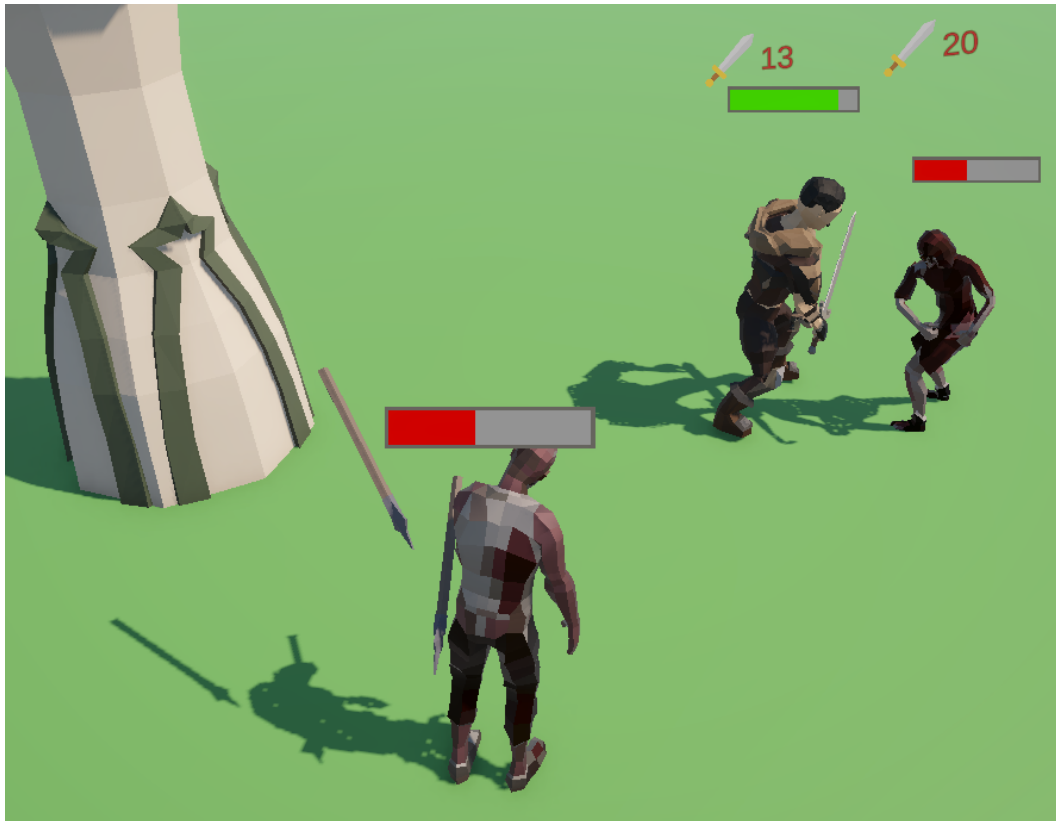


Figura B.6: Combate y retroalimentación visual

Las unidades de combate que reciben una orden de movimiento, al alcanzar el destino, vigilan el área alrededor. Si la unidad se aleja al defender el área de enemigos, una vez que no haya más enemigos, vuelve a la posición inicial.

En la Figura B.7 se puede observar a 6 unidades moviéndose en formación al destino. Cada unidad muestra un halo azul indicando que están seleccionadas, y el destino indicado por el jugador muestra un halo púrpura.

Fin de partida

Cuando el cristal del jugador es destruido, la partida se pausa, la cámara orbita alrededor del cristal, el cristal pasa a un estado destruido y se muestra la pantalla de derrota, como se puede ver en la Figura B.8.



Figura B.7: Movimiento en formación

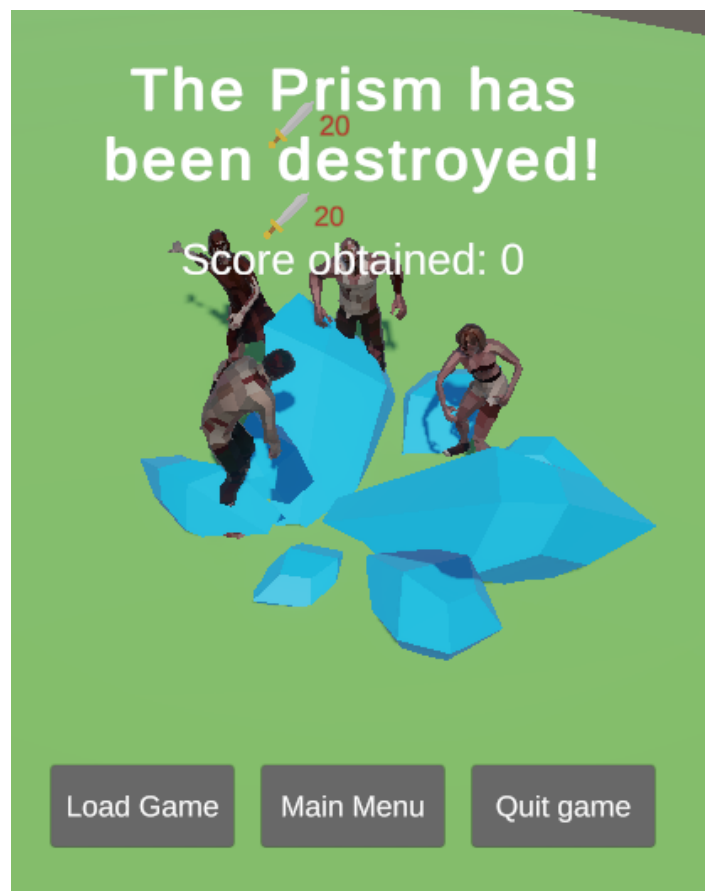


Figura B.8: Interfaz de derrota